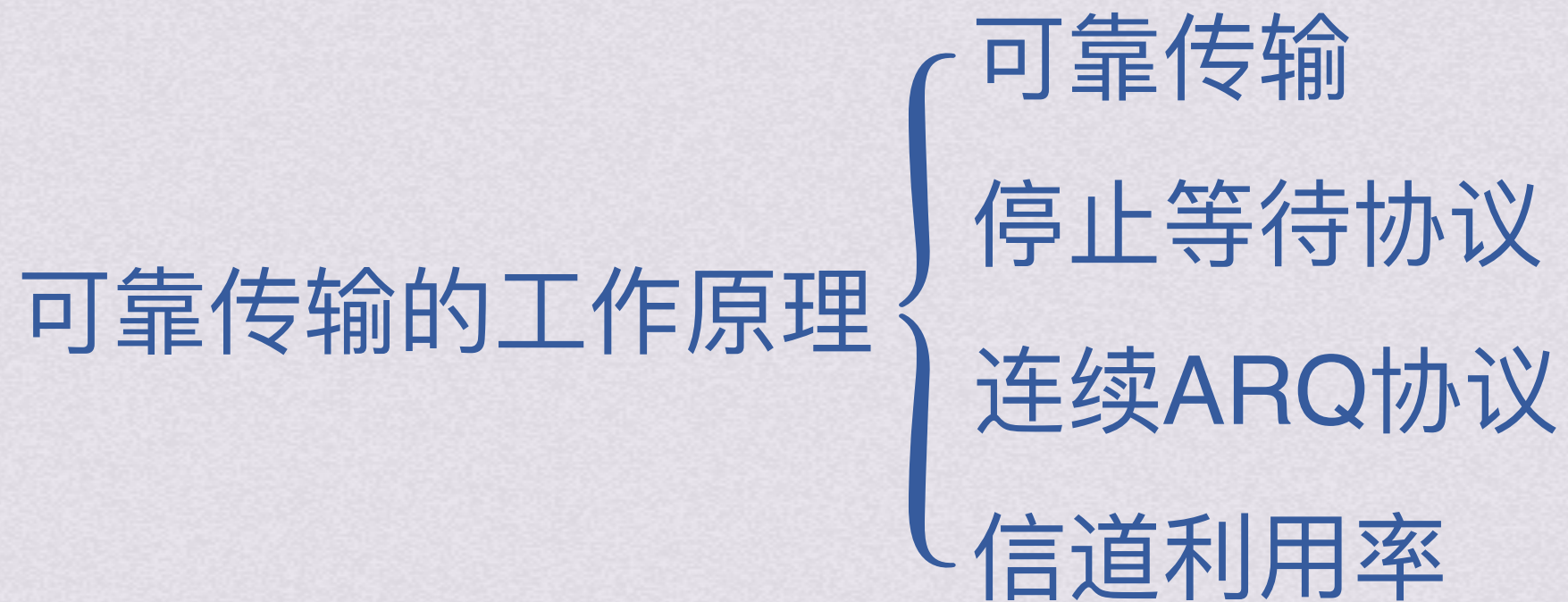


bok25

基
礎

課、計算機網絡





可靠传输





可靠传输





可靠传输



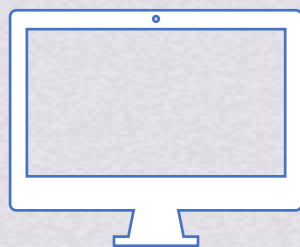


可靠传输





可靠传输



除了可能发生比特差错，
帧在传输过程中可能失序、重复、丢失等差错



可靠传输



帧2

帧1

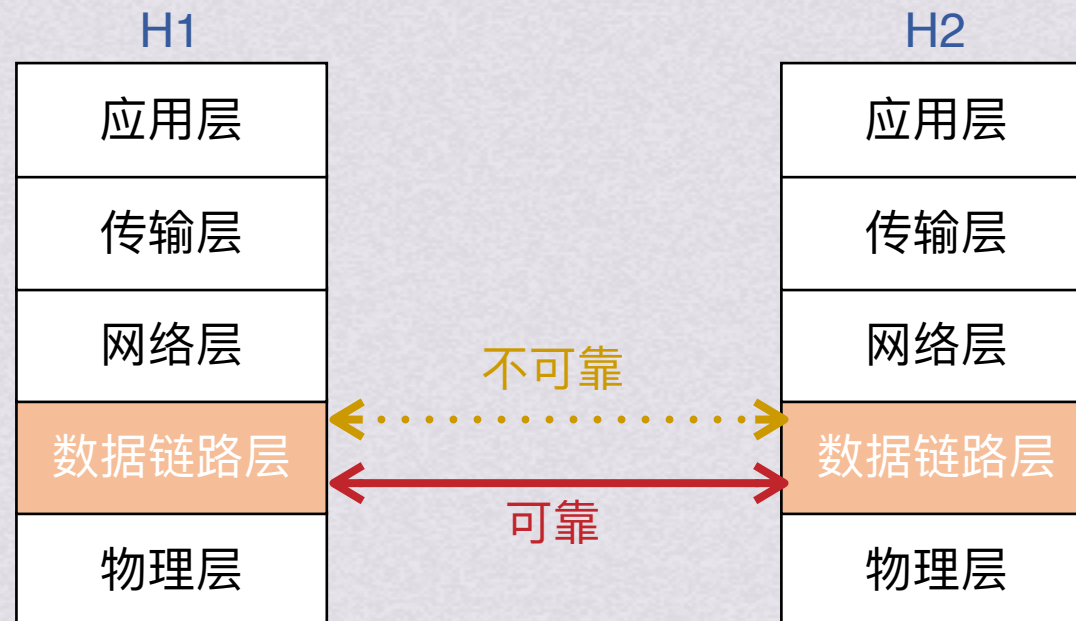
帧3

除了可能发生比特差错，
帧在传输过程中可能失序、重复、丢失等差错

CRC校验可以实现没有比特差错的传输，那么再解决上面几种差错，就能做到提供可靠传输
所以我們还需要帧编号、确认和重传机制



可靠传输



网络并不强制数据链路层提供可靠传输，如果链路质量好，数据链路层就不使用可靠传输机制相应的，如果数据链路层传输数据时出了差错需要纠正，那么将由上层协议（如传输层的TCP）来完成

如果链路质量差，那还是需要数据链路层使用可靠传输机制的

本节主要介绍数据链路层的可靠传输机制



可靠传输

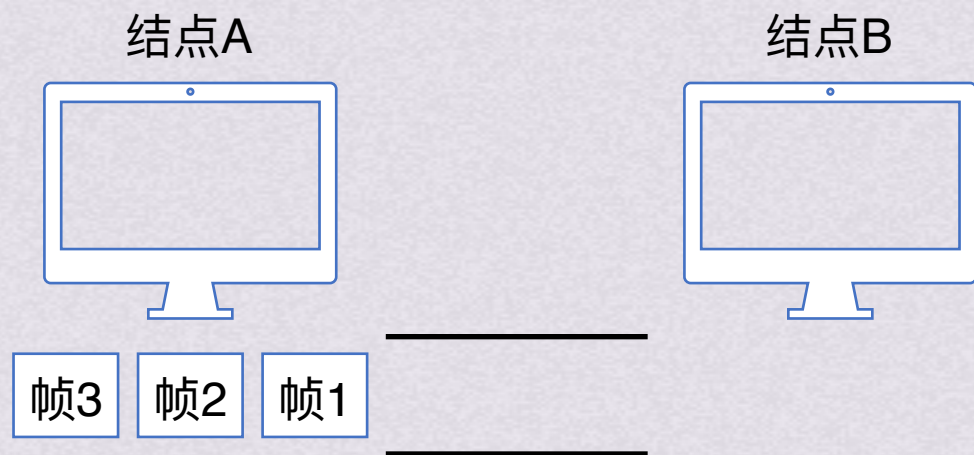


停止等待协议



停止等待协议

无差错情况



结点A



结点B



结点A待发送数据

帧1 帧2 帧3

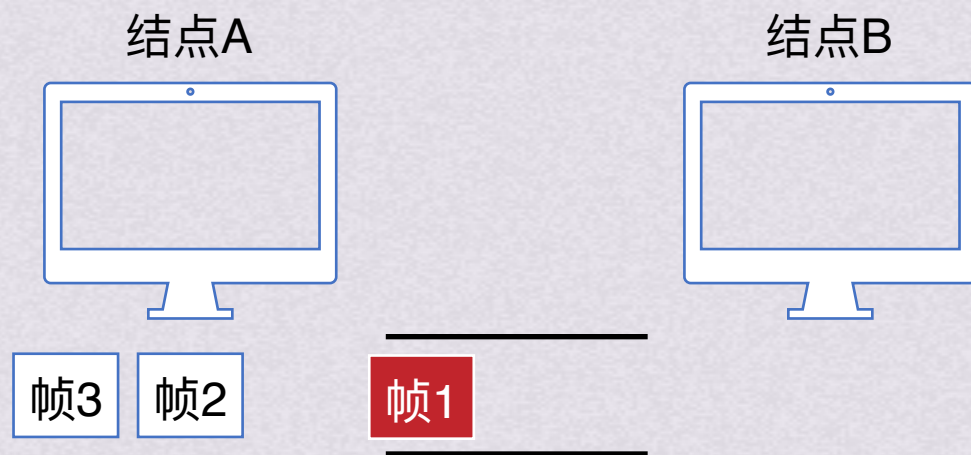
结点B收到数据





停止等待协议

无差错情况



结点A



结点B



将帧1传输到链路上

帧1在链路上传播

结点A待发送数据

帧1

帧2

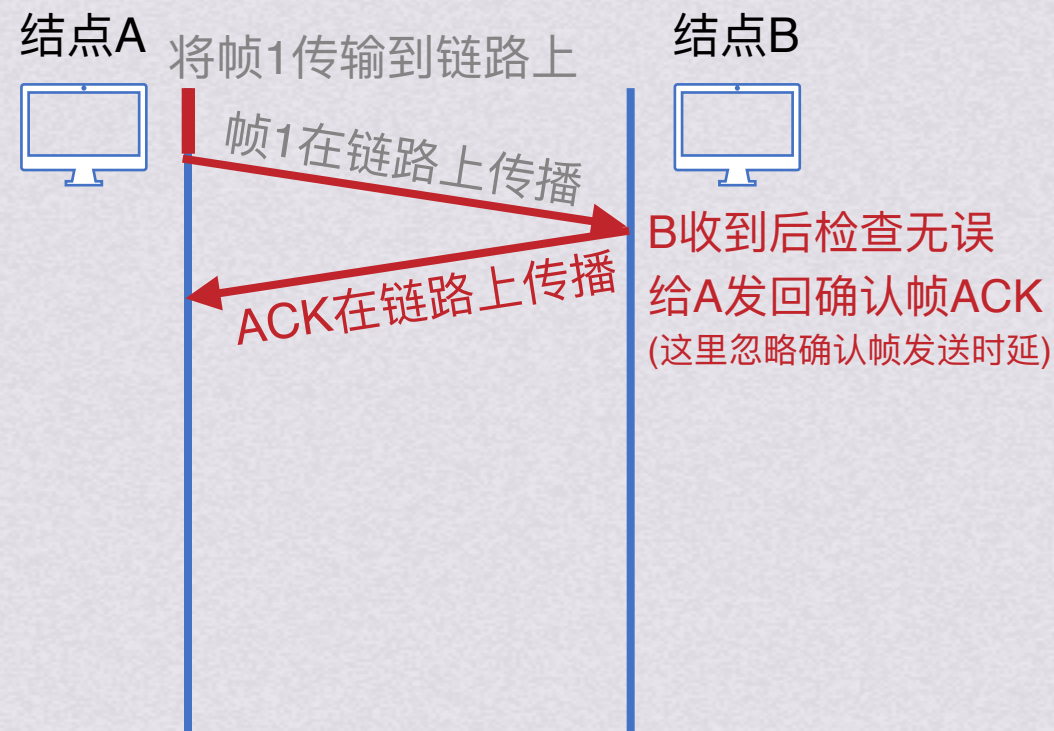
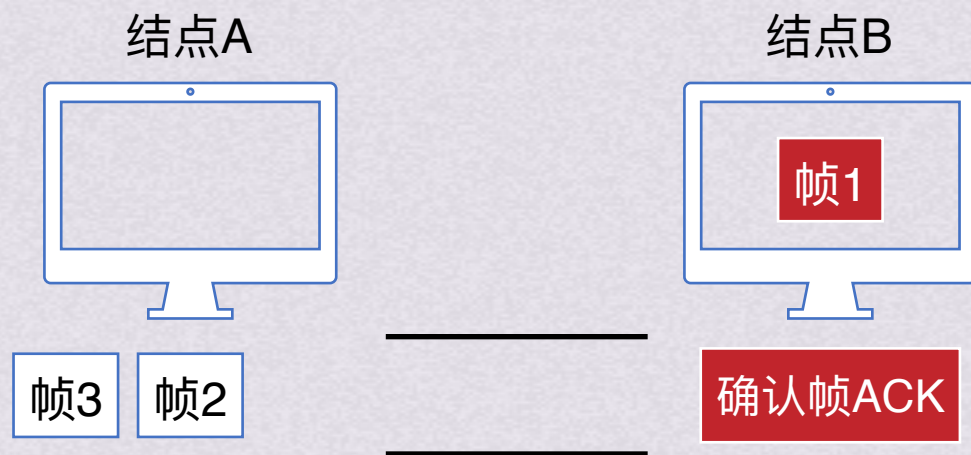
帧3

结点B收到数据



停止等待协议

无差错情况



结点A待发送数据

帧1 帧2 帧3

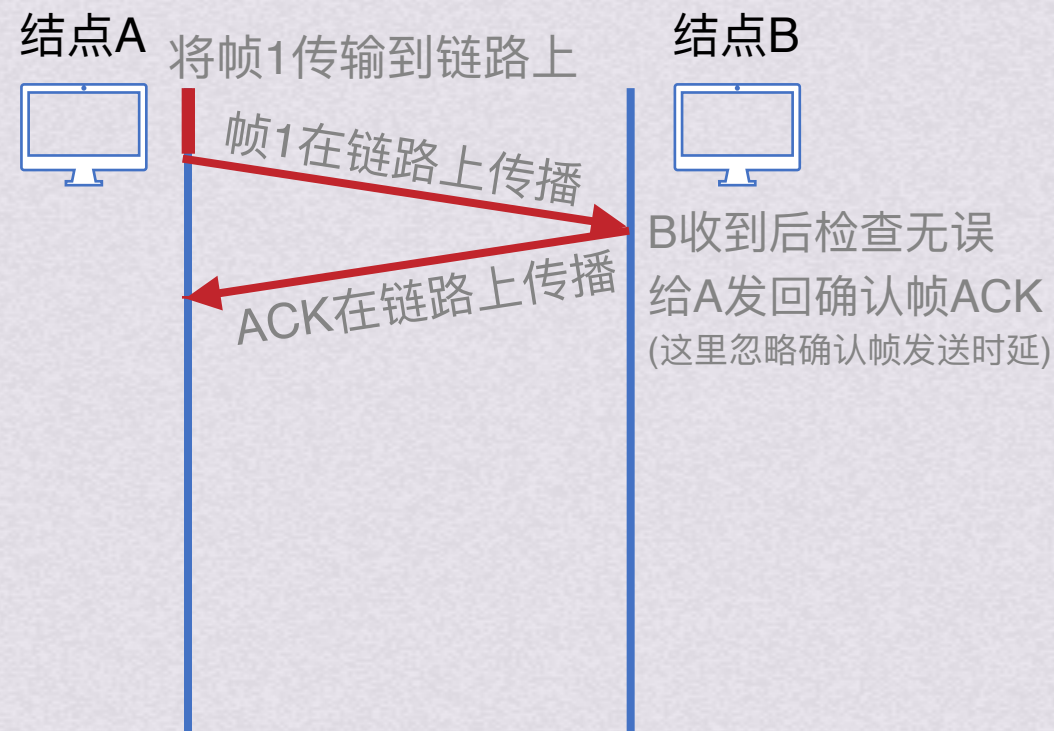
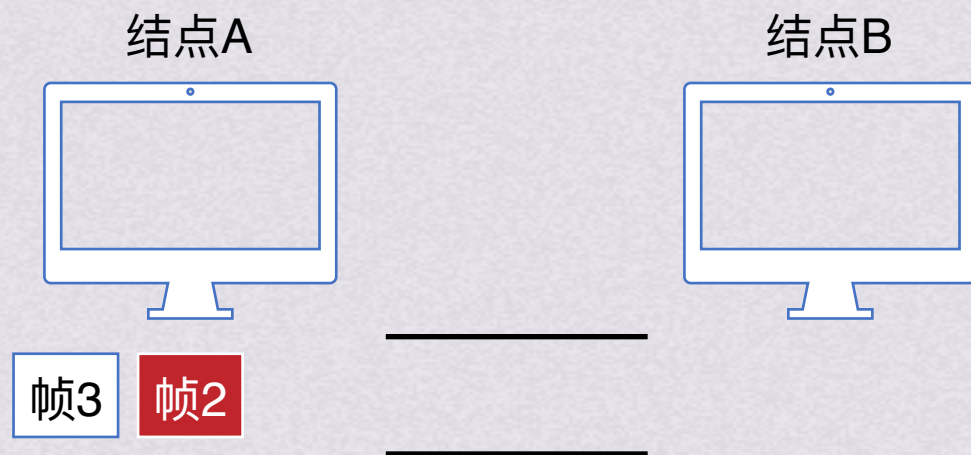
结点B收到数据





停止等待协议

无差错情况



结点A待发送数据



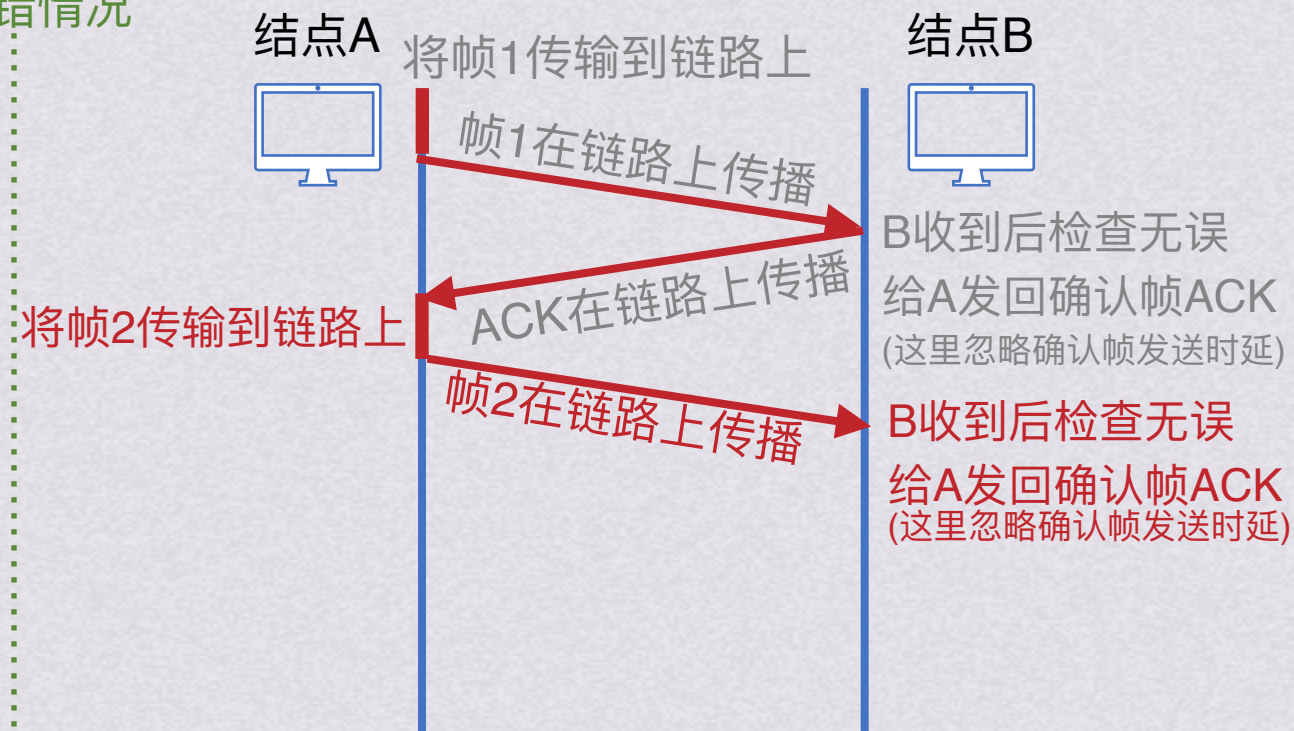
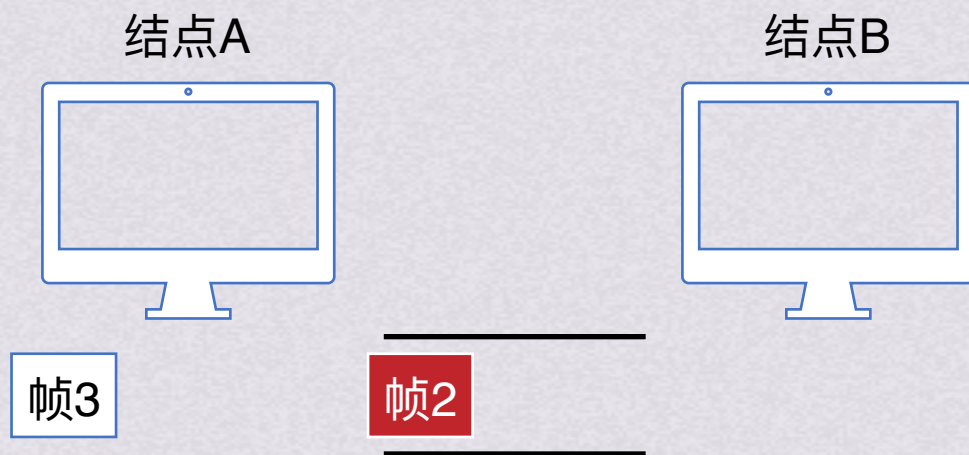
结点B收到数据





停止等待协议

无差错情况



结点A待发送数据



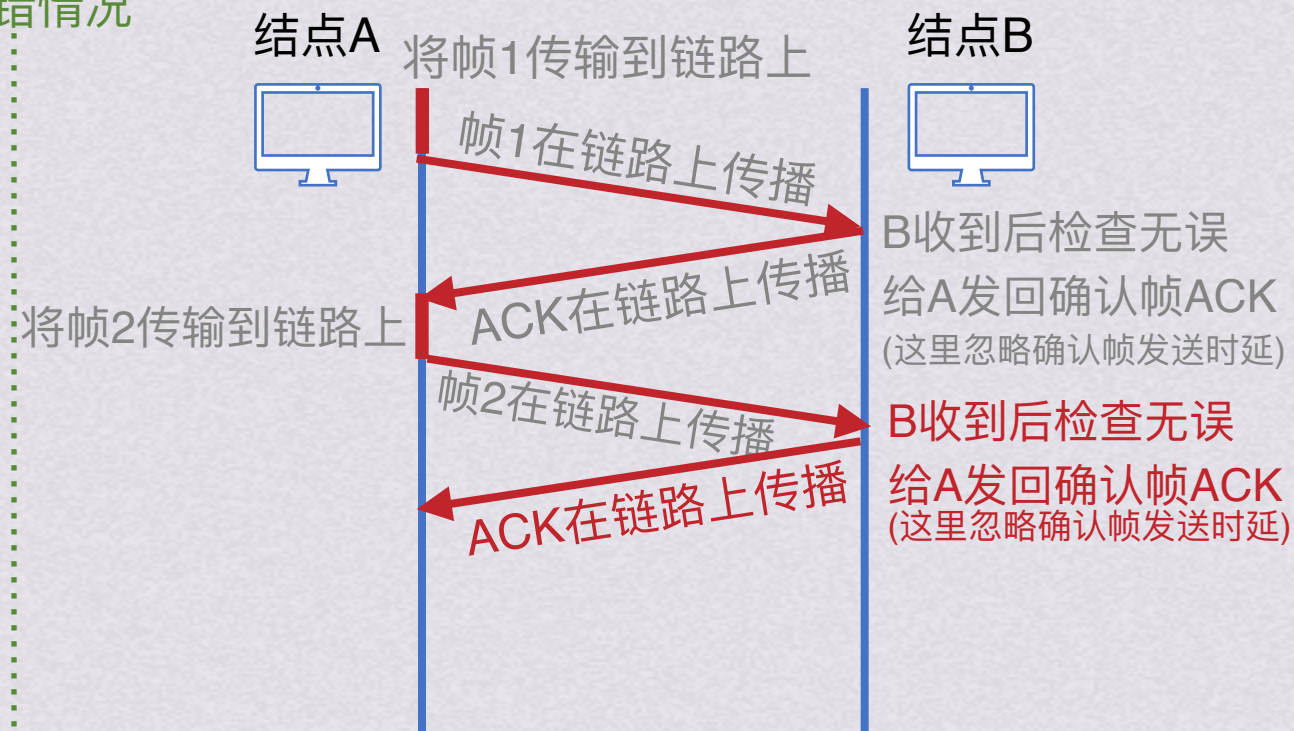
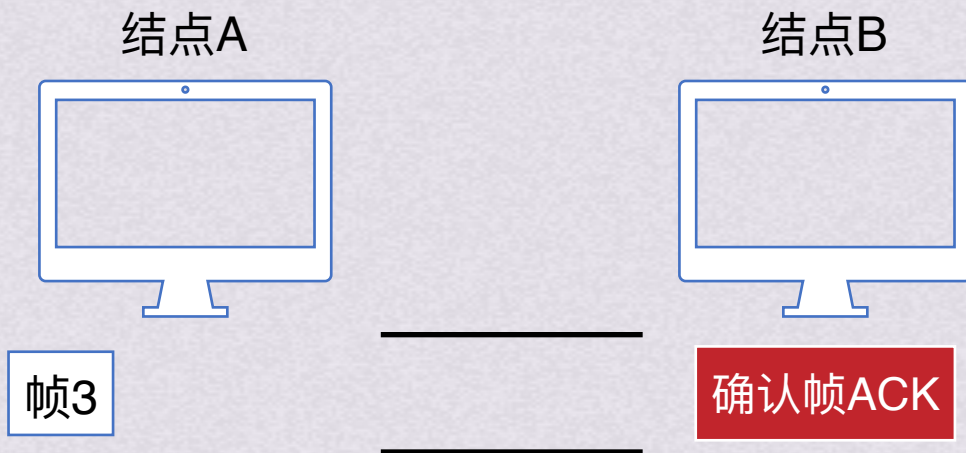
结点B收到数据





停止等待协议

无差错情况



结点A待发送数据



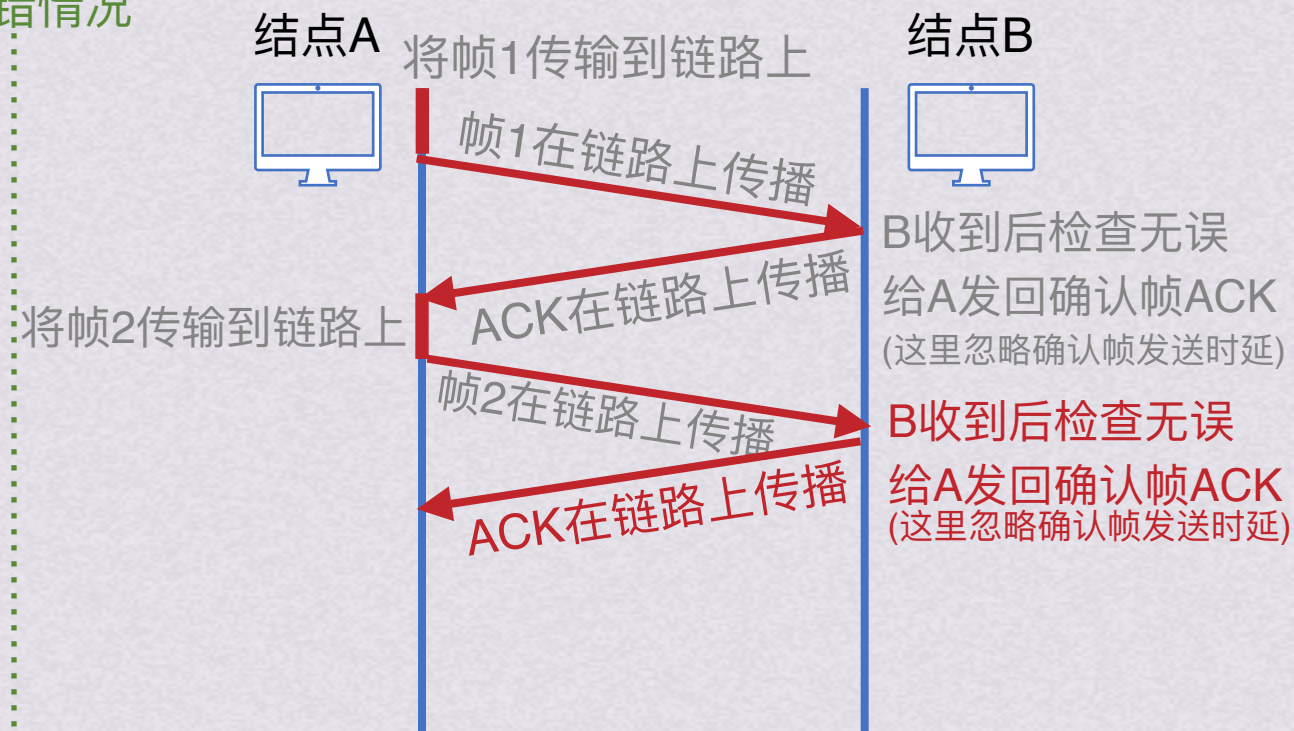
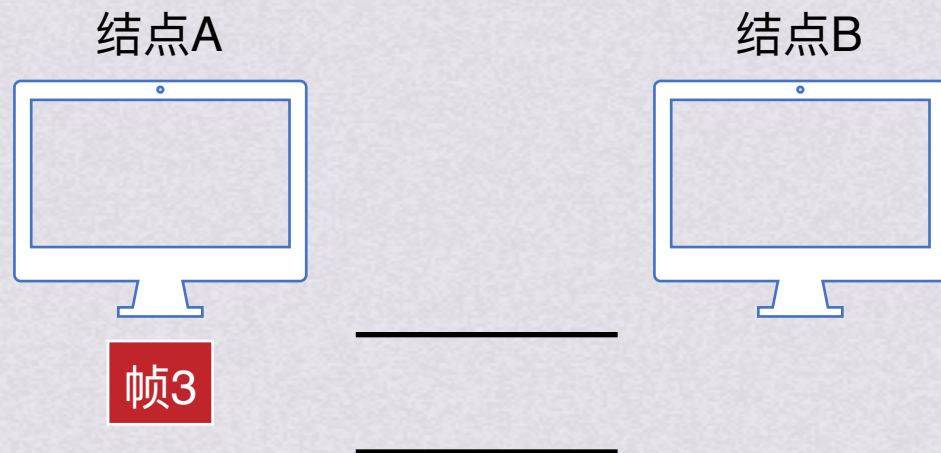
结点B收到数据





停止等待协议

无差错情况



结点A待发送数据



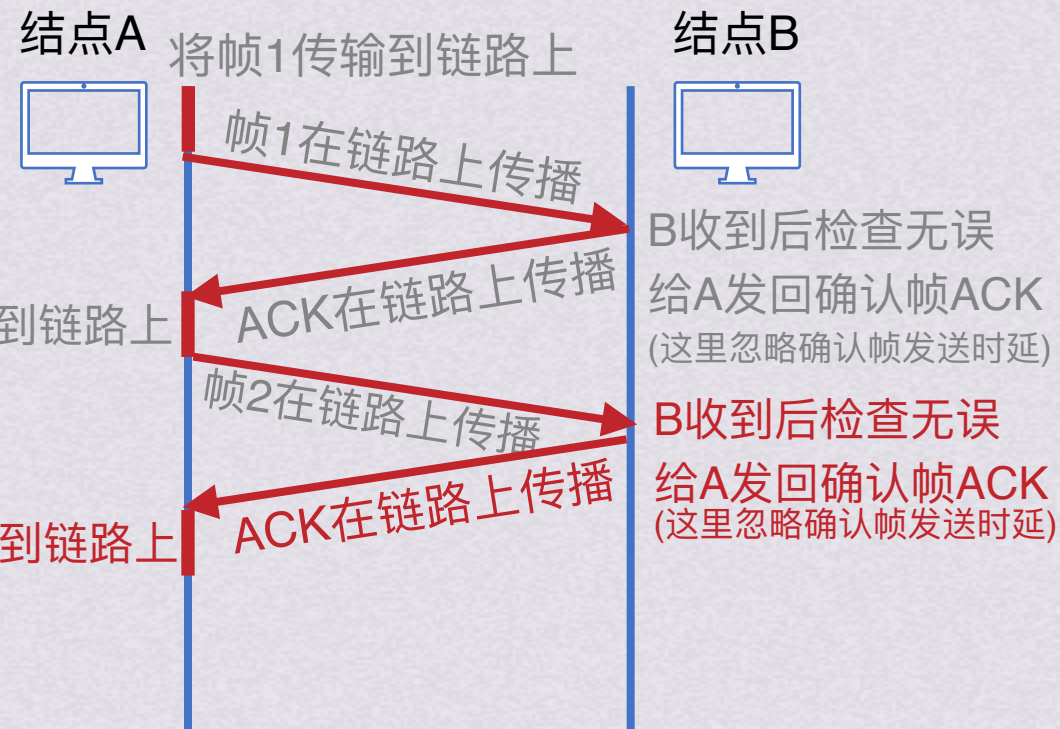
结点B收到数据





停止等待协议

无差错情况



结点A待发送数据



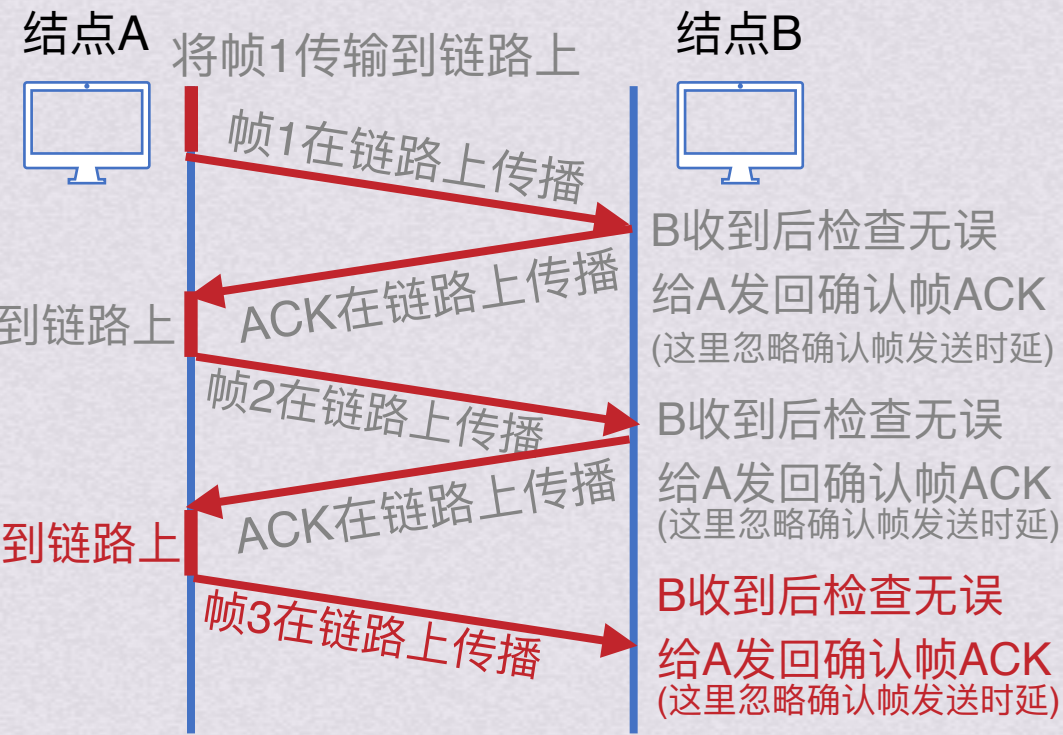
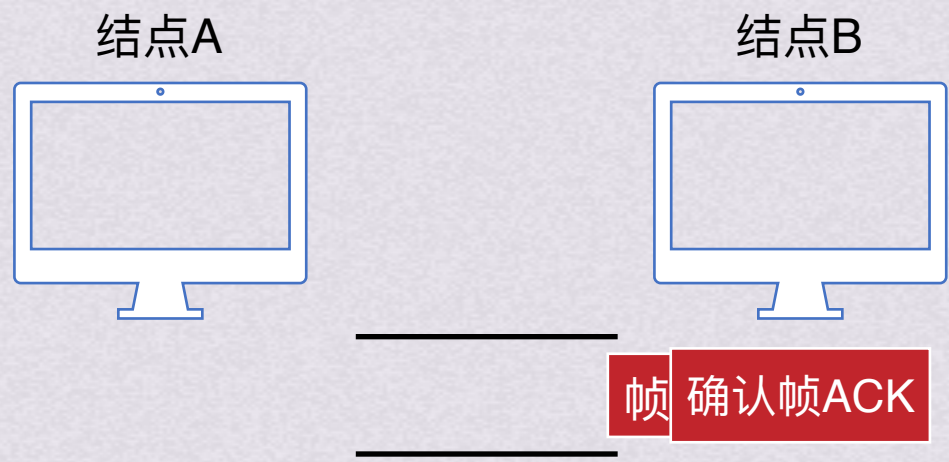
结点B收到数据





停止等待协议

无差错情况



结点A待发送数据

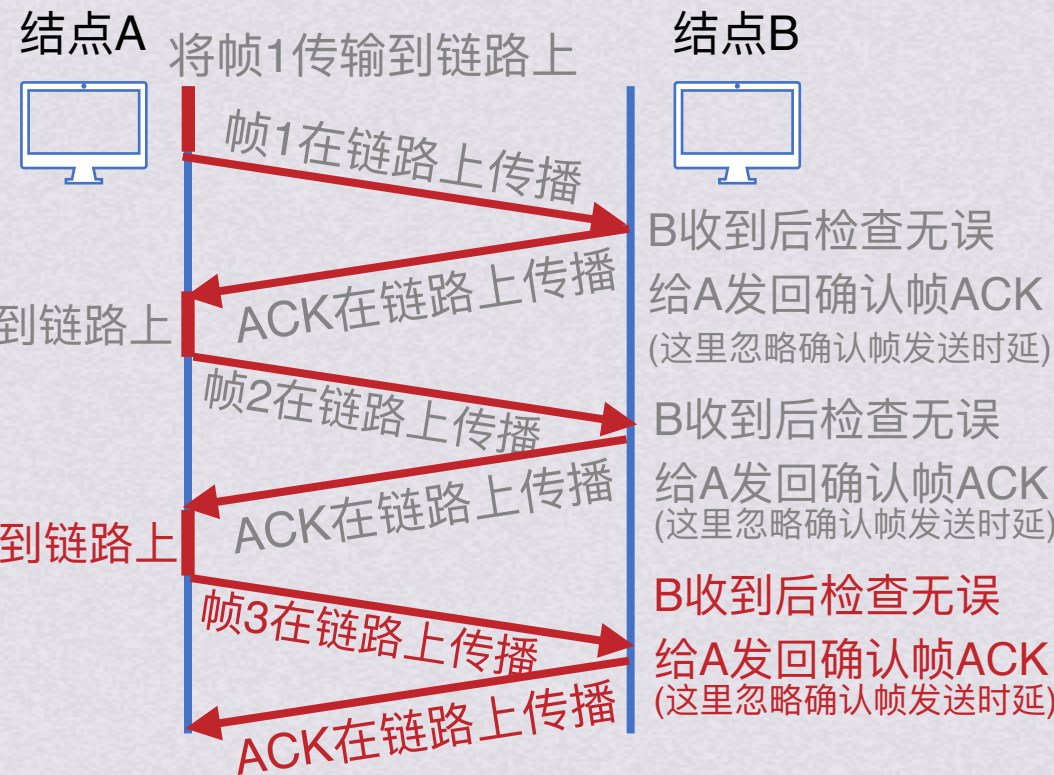
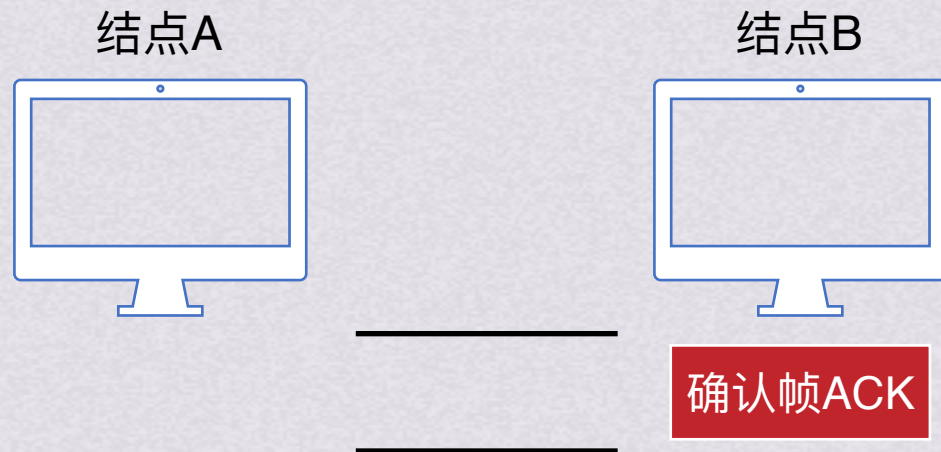


结点B收到数据



停止等待协议

无差错情况



结点A待发送数据

帧1 帧2 帧3

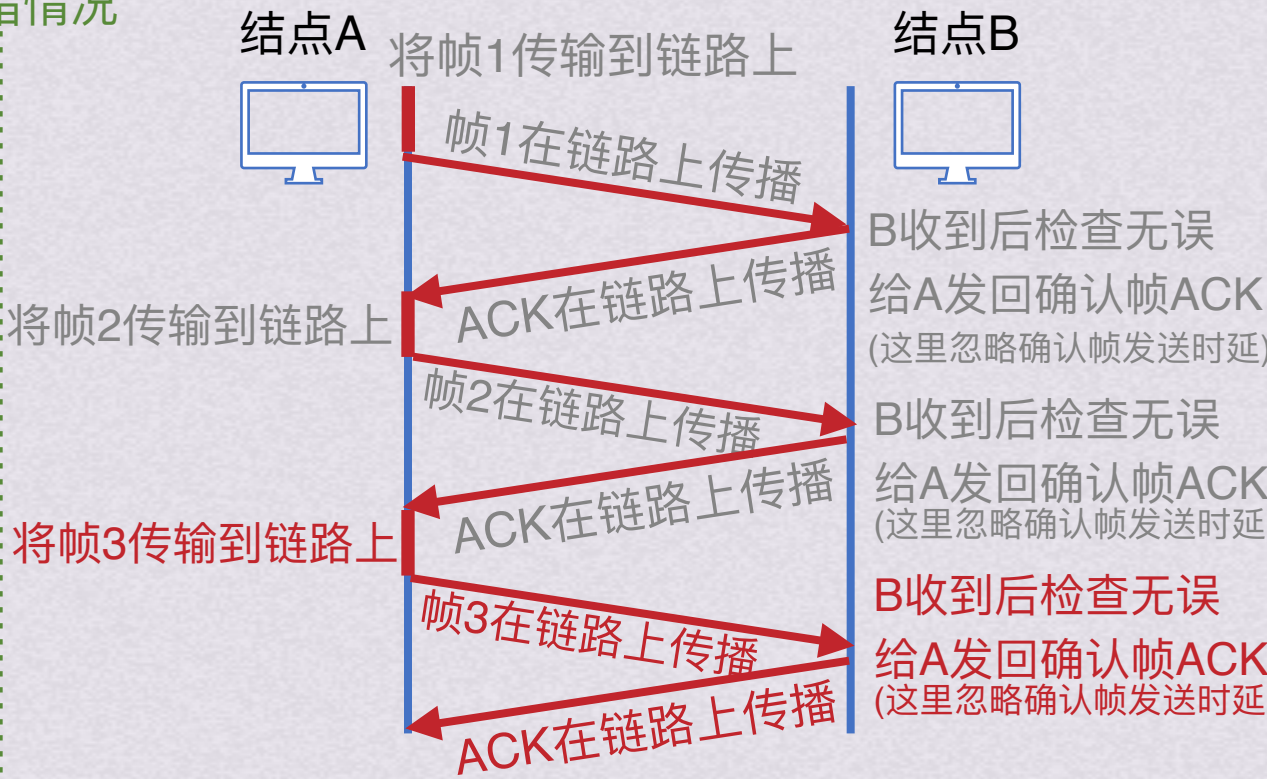
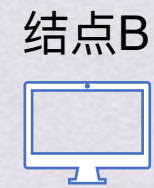
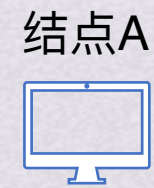
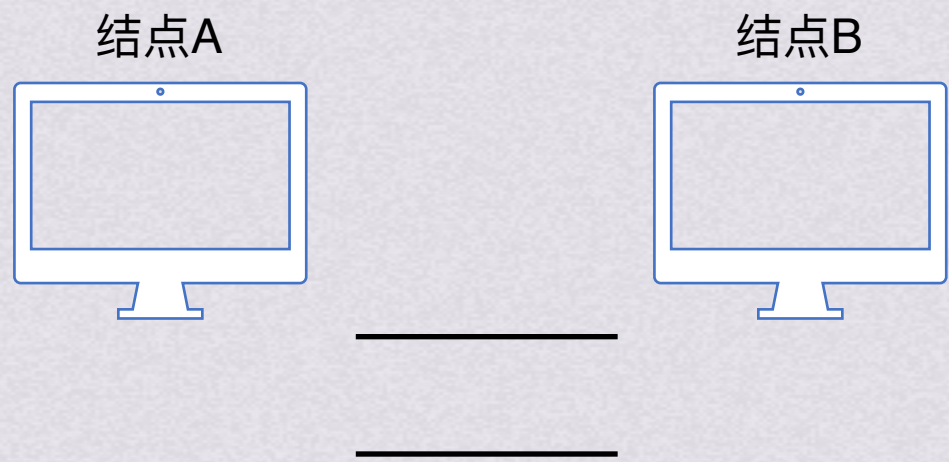
结点B收到数据

帧1 帧2 帧3



停止等待协议

无差错情况



结点A待发送数据

帧1	帧2	帧3
----	----	----

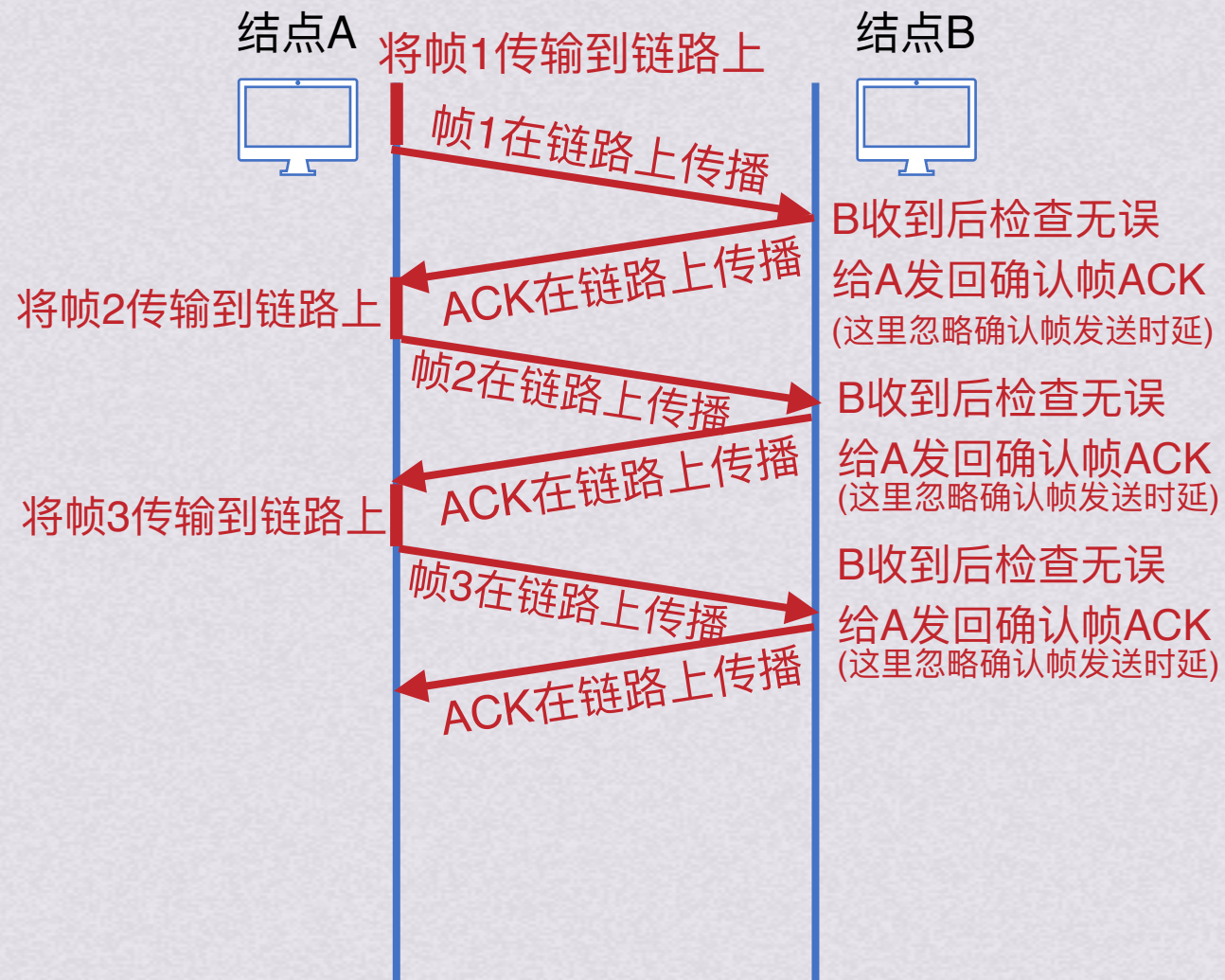
结点B收到数据

帧1	帧2	帧3
----	----	----



停止等待协议

无差错情况

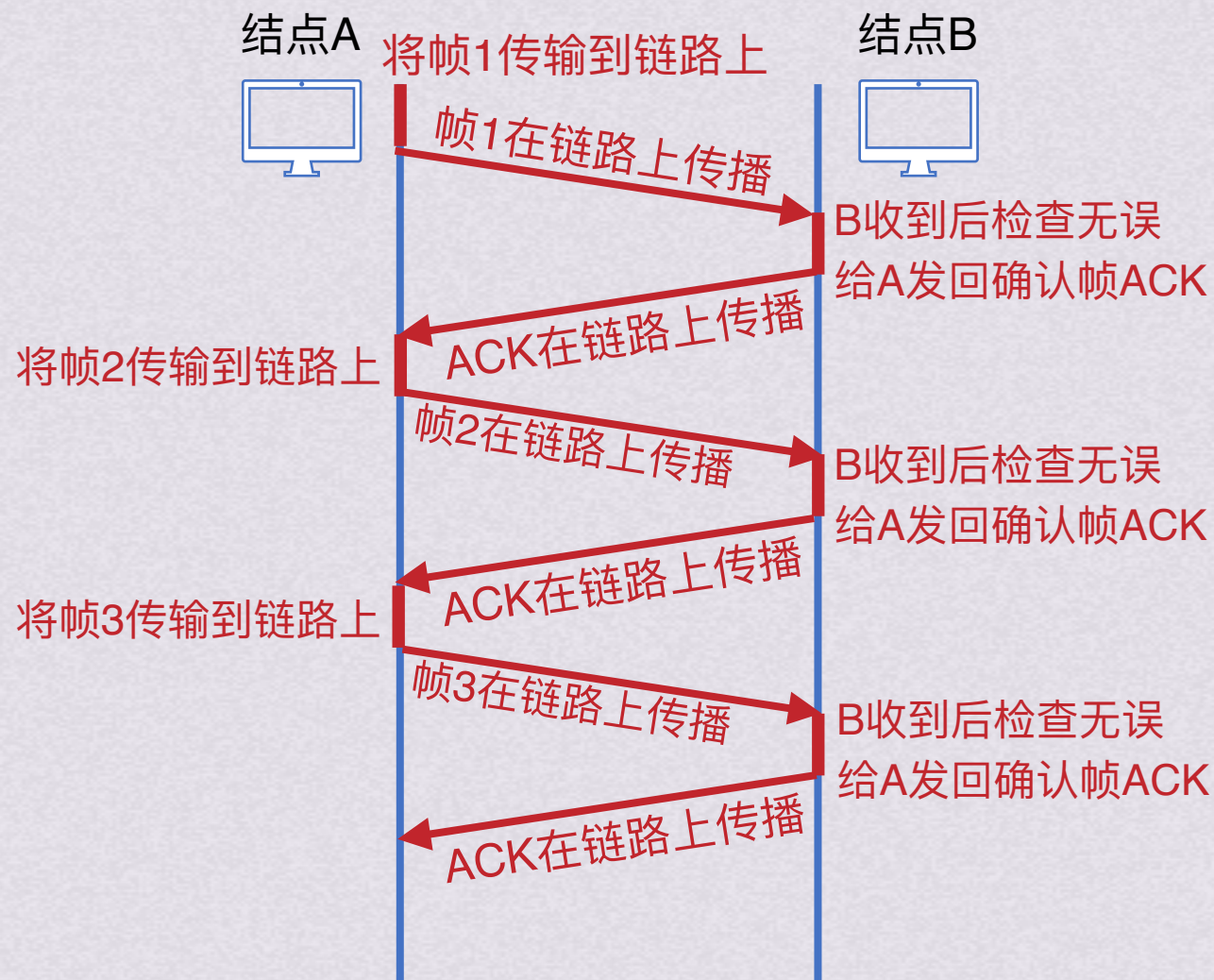




停止等待协议

无差错情况

做题目一般会忽略确认帧的发送时间，或者数据帧和确认帧都考虑在内，这里是同时考虑数据帧和确认帧的情况





停止等待协议

出现差错

超时时器



结点A



结点B



将帧1传输到链路上

帧1在链路上传播



B收到后检查发现出了差错

B会直接丢弃该帧，其他什么都不做



停止等待协议

出现差错

超时计时器



结点A



结点B



将帧1传输到链路上

帧1在链路上传播

超时计时器计时结束



B收到后检查发现出了差错
B会直接丢弃该帧，其他什么都不做

将帧1传输到链路上

帧1在链路上传播

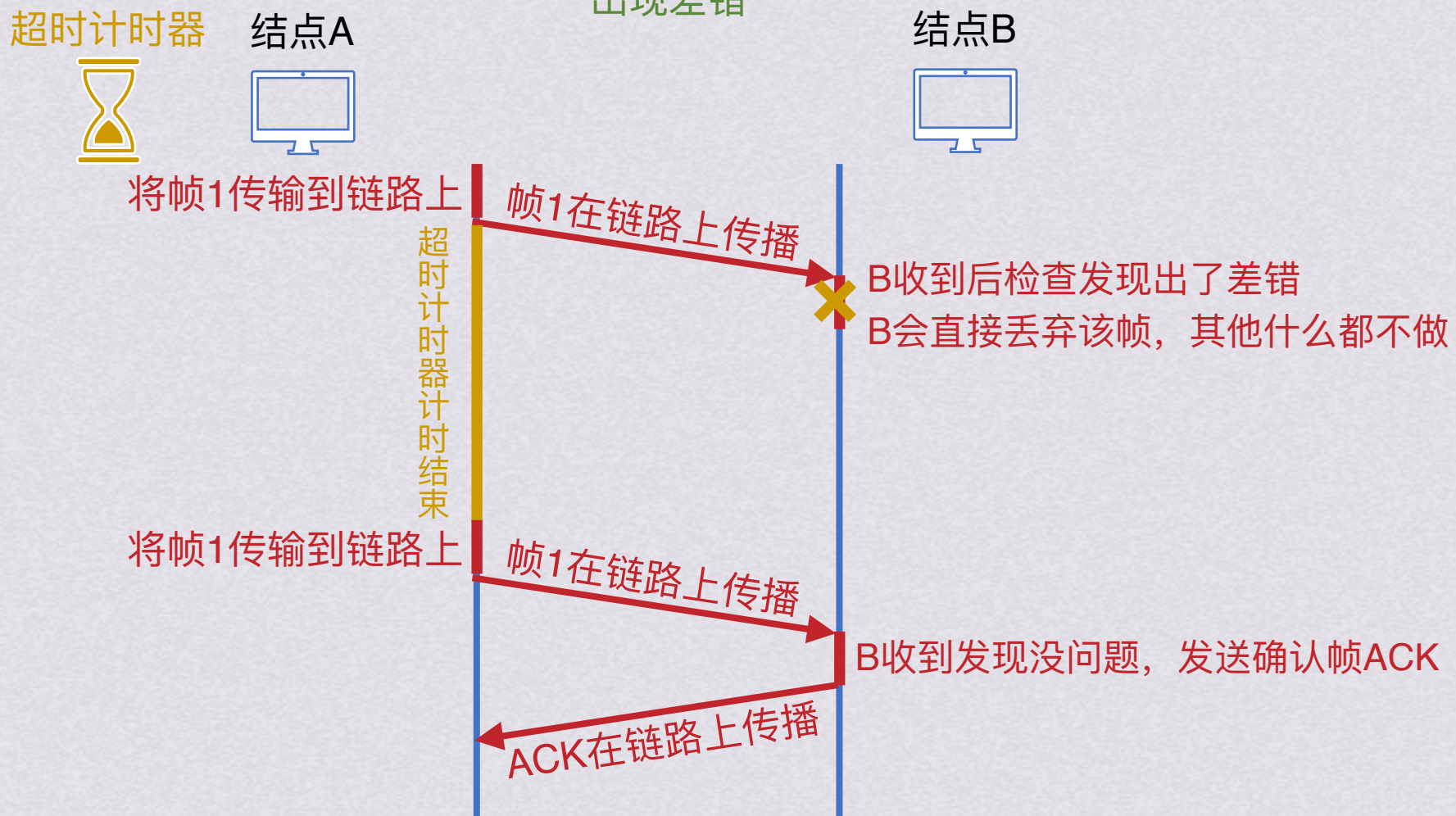
B收到发现没问题，发送确认帧ACK

ACK在链路上传播



停止等待协议

出现差错



注意：可靠传输协议中，也可以在有差错的时候发送“否认报文NAK”给对方，这样能让发送方及时知道除了差错。但是这样处理会让协议复杂化，目前实用的可靠传输协议都不使用这种否认报文了



停止等待协议

出现差错

超时计时器



结点A



结点B



为了实现超时重传，需要注意三点：

- 1.发送方发送完一个帧后，需要暂时保留已发送帧的副本供超时重传时使用，只有收到确认后才能清除副本
- 2.帧和确认帧都需要编号，这样才能明确是哪一个发出去的帧收到了确认。编号是循环使用的，比如使用2bit来编号，范围就只能是0~3，再增加一个号又会回到0
- 3.超时计时器设置的重传时间应比帧传输的RTT更长一些

将帧1传输到链路上

帧1在链路上传播

超时计时器计时结束

ACK在链路上传播

B收到后检查发现出了差错
B会直接丢弃该帧，其他什么都不做

将帧1传输到链路上

帧1在链路上传播

ACK在链路上传播

B收到发现没问题，
发送确认帧ACK

这时候才能删除帧1副本

注意：可靠传输协议中，也可以在有差错的时候发送“否认报文NAK”给对方，这样能让发送方及时知道除了差错。但是这样处理会让协议复杂化，目前实用的可靠传输协议都不使用这种否认报文了



停止等待协议

出现差错

超时计时器



结点A



结点B



为了实现超时重传，需要注意三点：

- 1.发送方发送完一个帧后，需要暂时保留已发送帧的副本供超时重传时使用，只有收到确认后才能清除副本
- 2.帧和确认帧都需要编号，这样才能明确是哪一个发出去的帧收到了确认。编号是循环使用的，比如使用2bit来编号，范围就只能是0~3，再增加一个号又会回到0
- 3.超时计时器设置的重传时间应比帧传输的RTT更长一些

将帧1传输到链路上

帧1在链路上传播

超时计时器计时结束

ACK在链路上传播

B收到后检查发现出了差错
B会直接丢弃该帧，其他什么都不做

将帧1传输到链路上

帧1在链路上传播

ACK在链路上传播

B收到发现没问题，
发送确认帧ACK

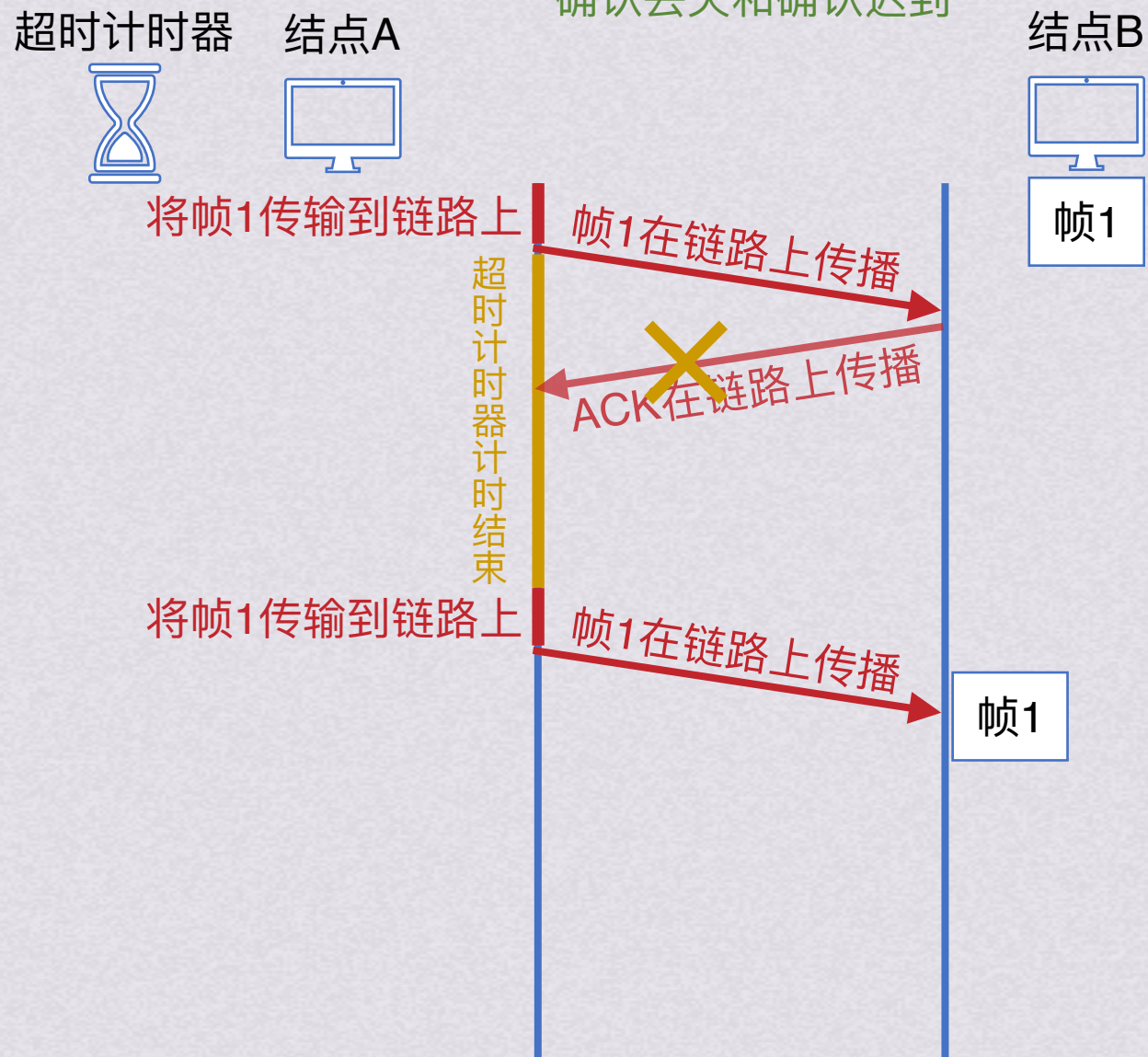
这时候才能删除帧1副本

注意：可靠传输协议中，也可以在有差错的时候发送“否认报文NAK”给对方，这样能让发送方及时知道除了差错。但是这样处理会让协议复杂化，目前实用的可靠传输协议都不使用这种否认报文了



停止等待协议

确认丢失和确认迟到





停止等待协议

确认丢失和确认迟到





停止等待协议

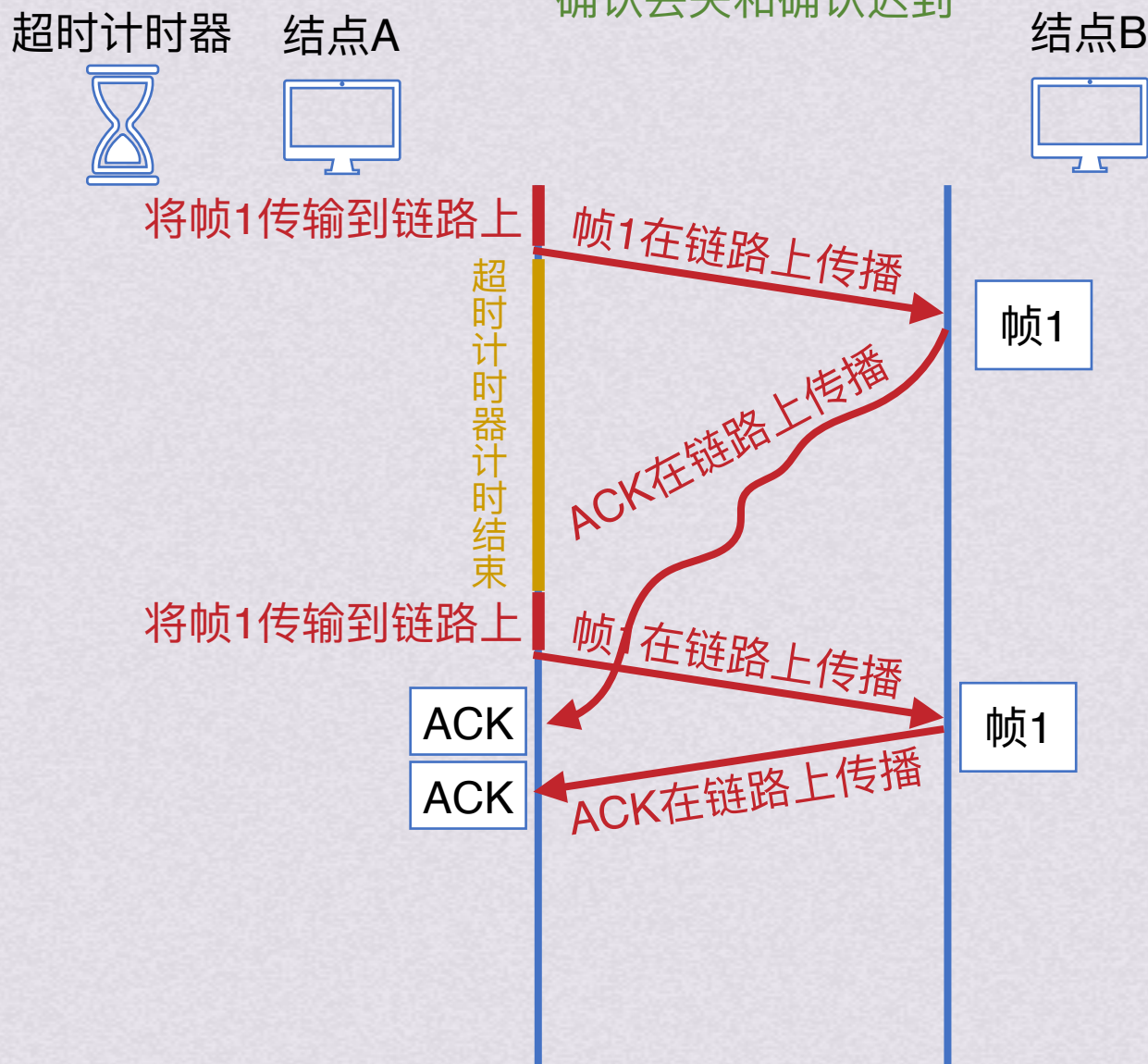
确认丢失和确认迟到





停止等待协议

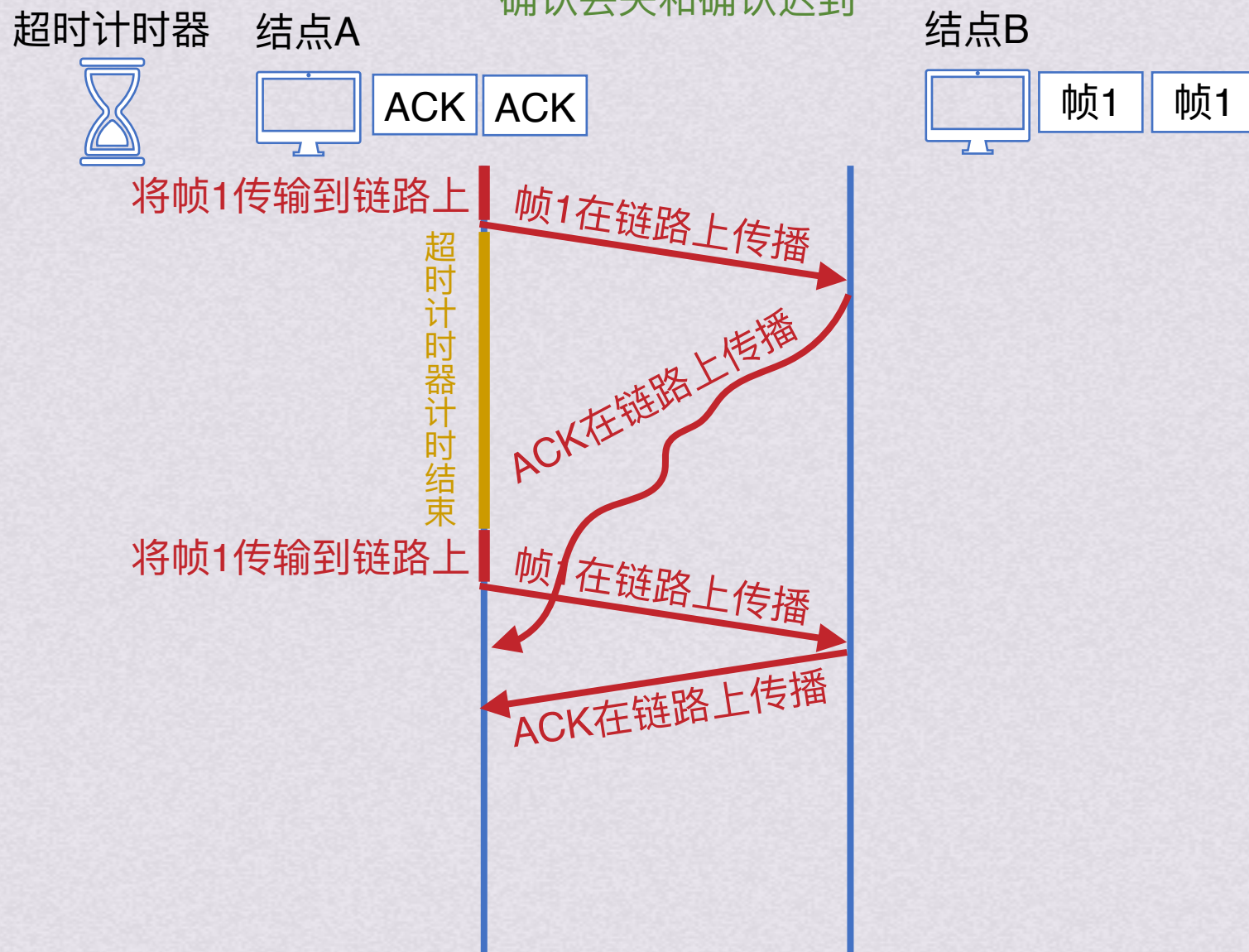
确认丢失和确认迟到





停止等待协议

确认丢失和确认迟到





停止等待协议



连续ARQ协议



连续ARQ协议

采用确认和超时重传两种机制的可靠传输协议称为自动重传请求(ARQ)

ARQ协议有三种，停止等待协议、后退N帧协议和选择重传协议

为了搞懂剩下的两种连续ARQ协议，我们需要先引入滑动窗口的概念



连续ARQ协议

滑动窗口机制





连续ARQ协议

滑动窗口机制

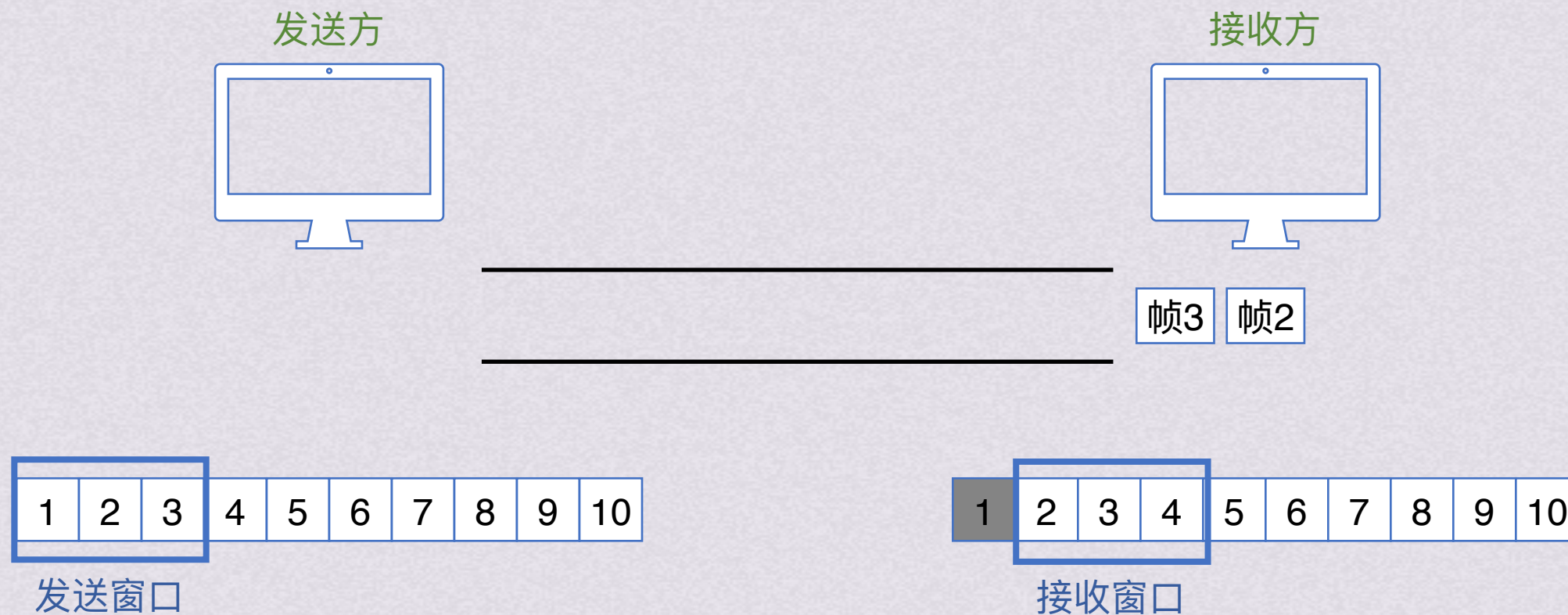


接收方每收到一个编号属于接收窗口的数据帧时，就将该帧收下，然后将接收窗口滑动一个位置，并发挥确认。这样，就会有新的编号落入接收窗口，从而继续接收新的帧。如果收到编号不在窗口内的帧，直接丢弃



连续ARQ协议

滑动窗口机制

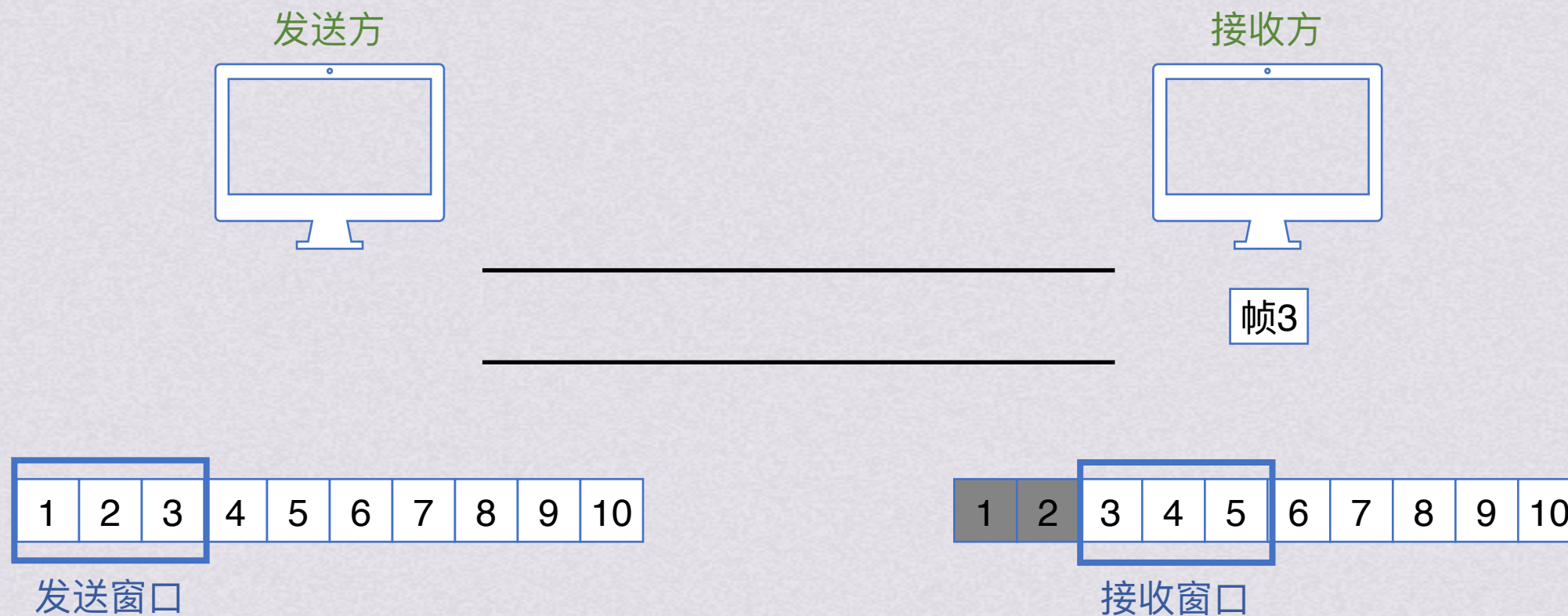


接收方每收到一个编号属于接收窗口的数据帧时，就将该帧收下，然后将接收窗口滑动一个位置，并发挥确认。这样，就会有新的编号落入接收窗口，从而继续接收新的帧。如果收到编号不在窗口内的帧，直接丢弃



连续ARQ协议

滑动窗口机制

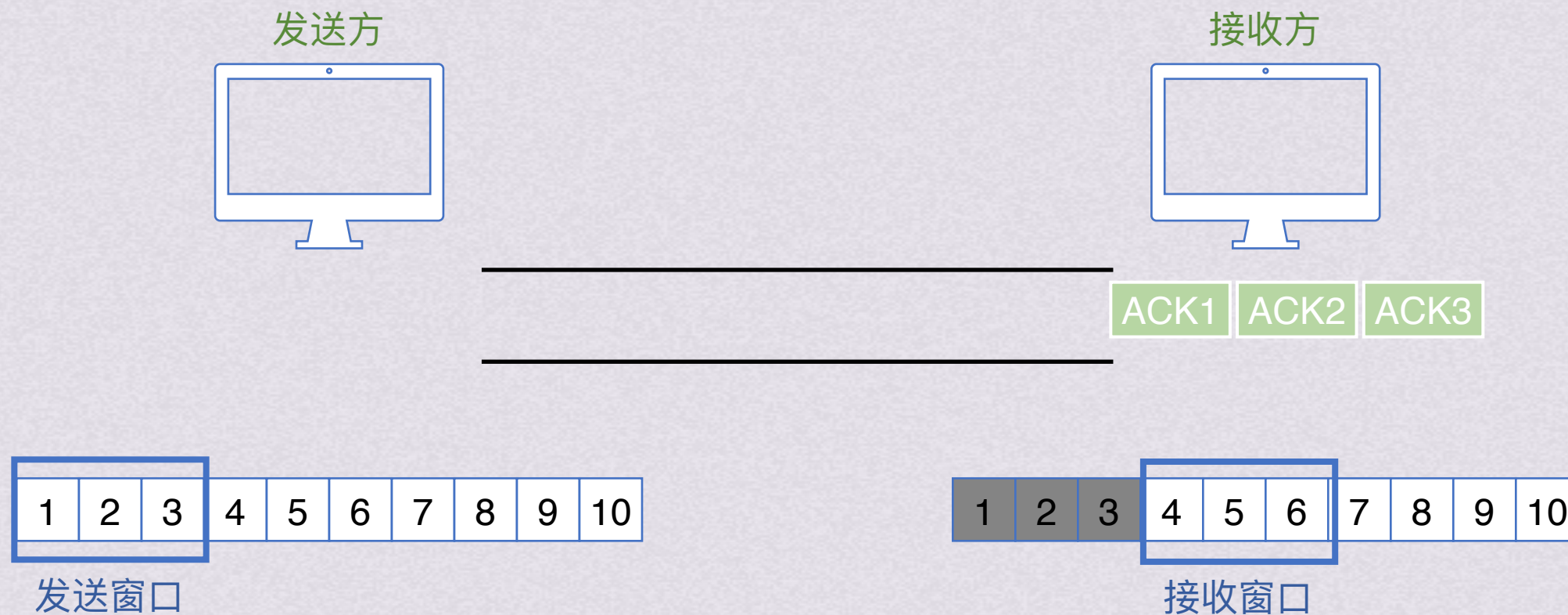


接收方每收到一个编号属于接收窗口的数据帧时，就将该帧收下，然后将接收窗口滑动一个位置，并发挥确认。这样，就会有新的编号落入接收窗口，从而继续接收新的帧。如果收到编号不在窗口内的帧，直接丢弃



连续ARQ协议

滑动窗口机制

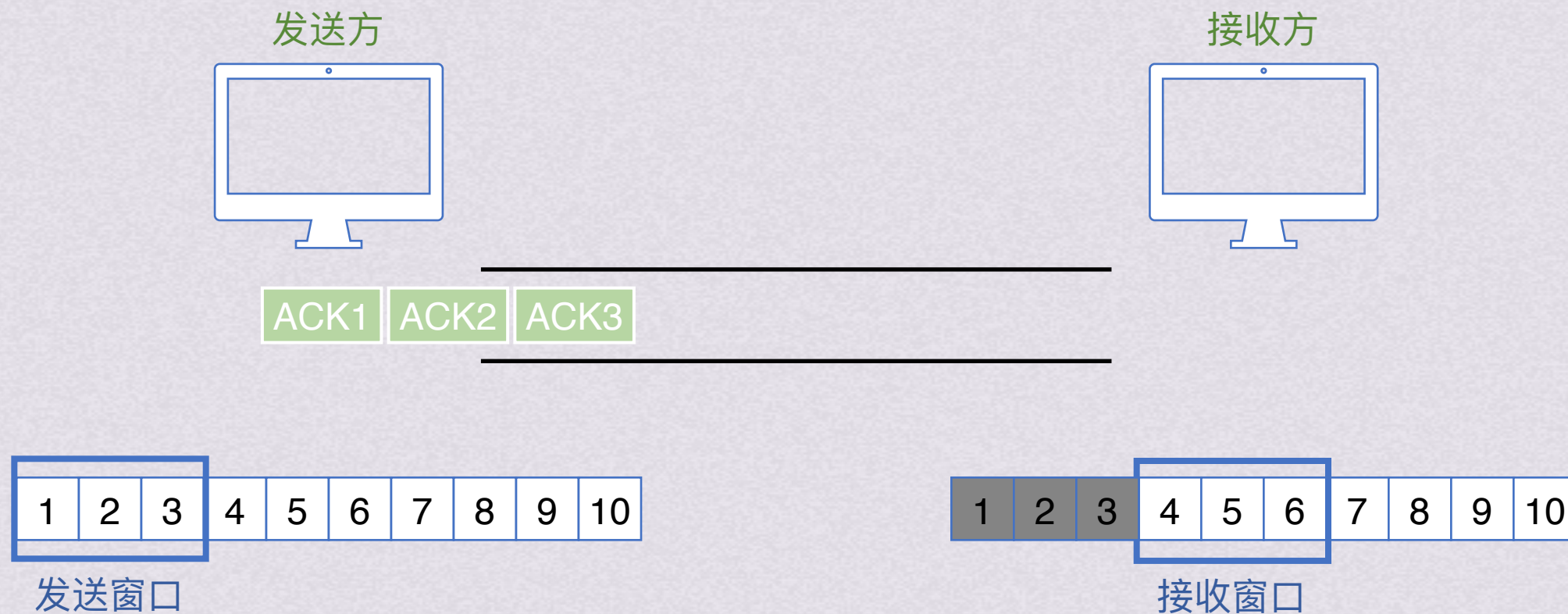


接收方每收到一个编号属于接收窗口的数据帧时，就将该帧收下，然后将接收窗口滑动一个位置，并发挥确认。这样，就会有新的编号落入接收窗口，从而继续接收新的帧。如果收到编号不在窗口内的帧，直接丢弃



连续ARQ协议

滑动窗口机制



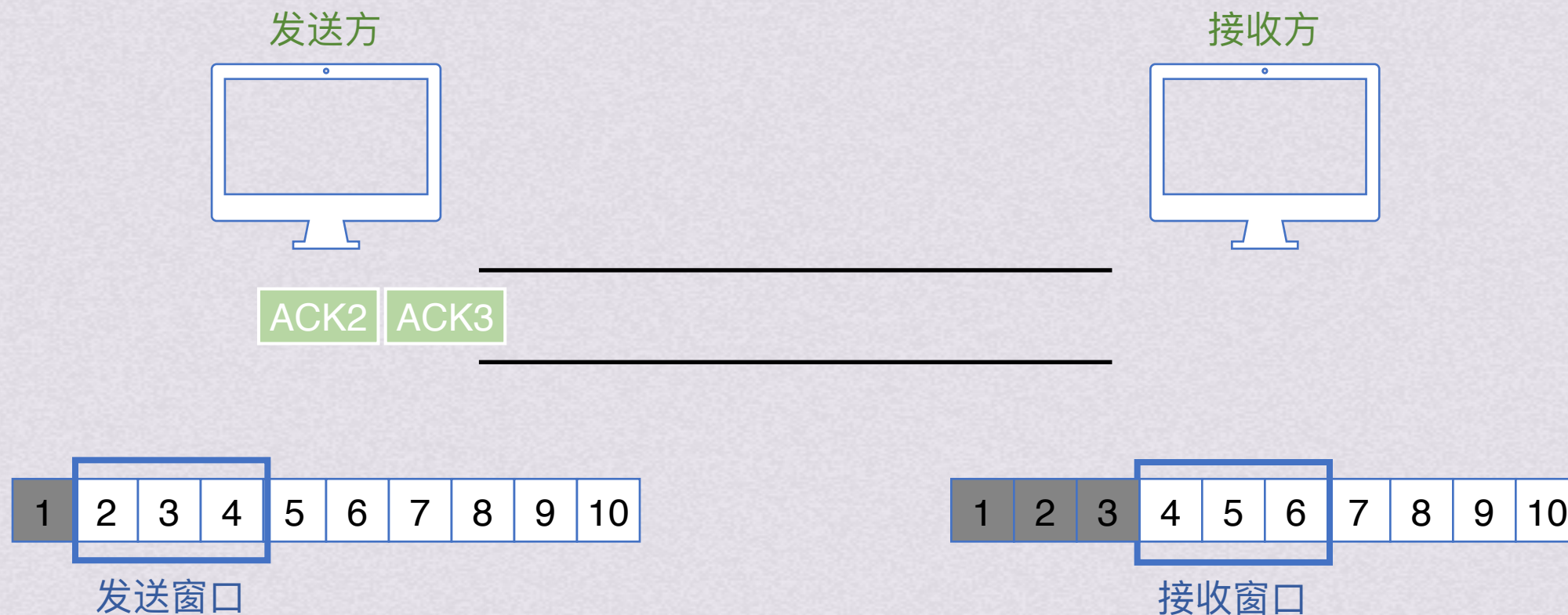
发送方每收到一个按序确认的确认帧，就将发送窗口滑动一个位置。这样，就会有一个新的编号落入发送窗口，从而继续发送。当窗口内的帧都已发送但都未收到确认帧时，窗口不能移动，发送方就停止发送

接收方每收到一个编号属于接收窗口的数据帧时，就将该帧收下，然后将接收窗口滑动一个位置，并发挥确认。这样，就会有新的编号落入接收窗口，从而继续接收新的帧。如果收到编号不在窗口内的帧，直接丢弃



连续ARQ协议

滑动窗口机制



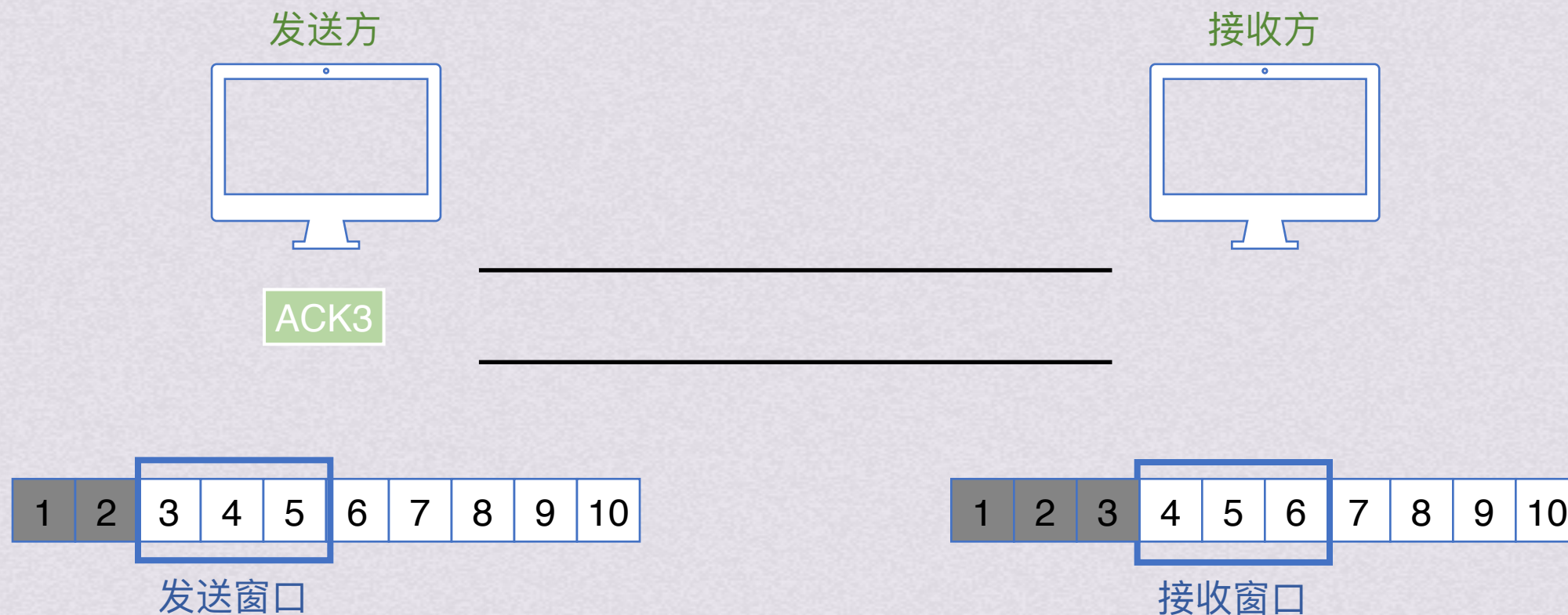
发送方每收到一个按序确认的确认帧，就将发送窗口滑动一个位置。这样，就会有一个新的编号落入发送窗口，从而继续发送。当窗口内的帧都已发送但都未收到确认帧时，窗口不能移动，发送方就停止发送

接收方每收到一个编号属于接收窗口的数据帧时，就将该帧收下，然后将接收窗口滑动一个位置，并发挥确认。这样，就会有一个新的编号落入接收窗口，从而继续接收新的帧。如果收到编号不在窗口内的帧，直接丢弃



连续ARQ协议

滑动窗口机制



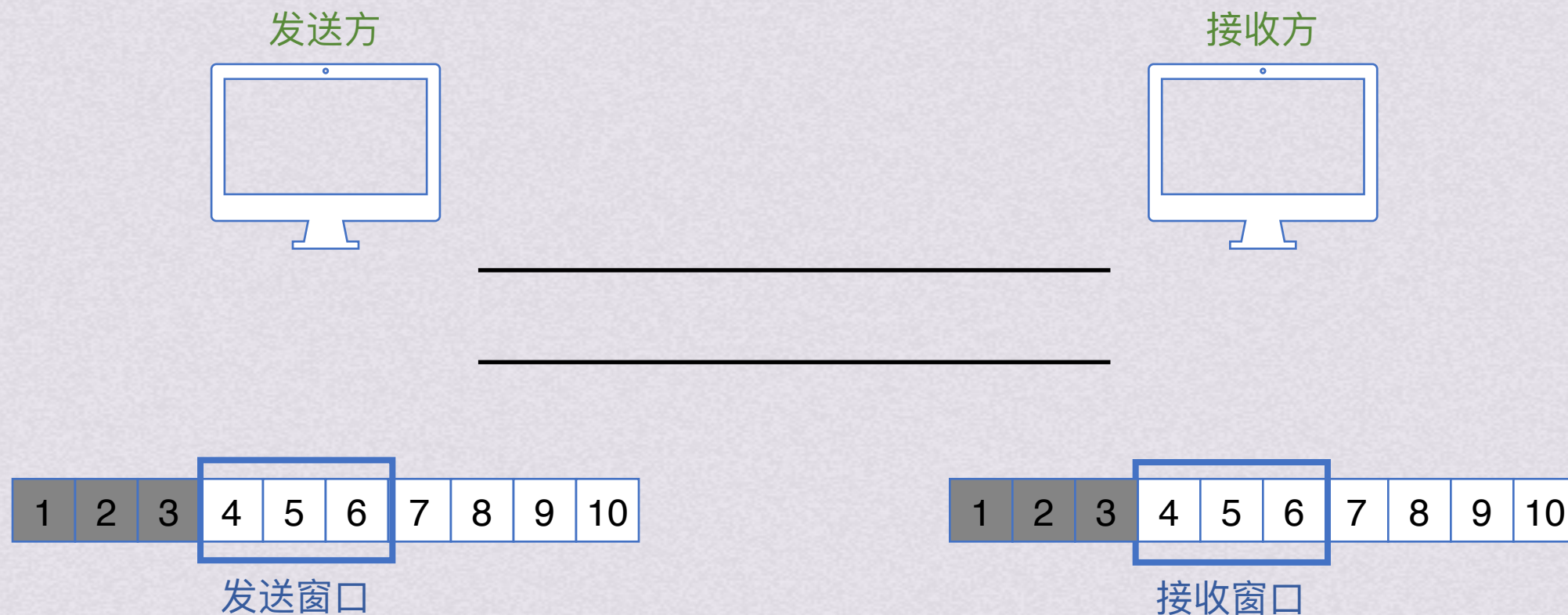
发送方每收到一个按序确认的确认帧，就将发送窗口滑动一个位置。这样，就会有新的编号落入发送窗口，从而继续发送。当窗口内的帧都已发送但都未收到确认帧时，窗口不能移动，发送方就停止发送

接收方每收到一个编号属于接收窗口的数据帧时，就将该帧收下，然后将接收窗口滑动一个位置，并发挥确认。这样，就会有新的编号落入接收窗口，从而继续接收新的帧。如果收到编号不在窗口内的帧，直接丢弃



连续ARQ协议

滑动窗口机制



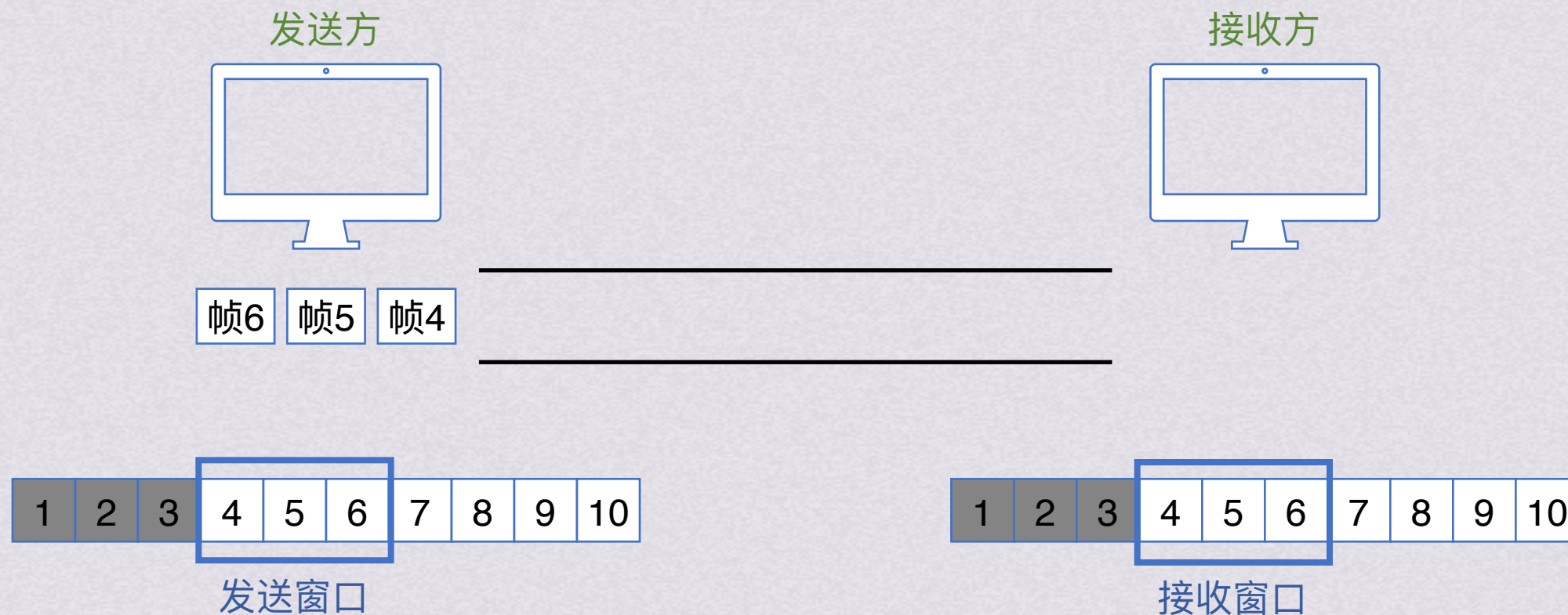
发送方每收到一个按序确认的确认帧，就将发送窗口滑动一个位置。这样，就会有一个新的编号落入发送窗口，从而继续发送。当窗口内的帧都已发送但都未收到确认帧时，窗口不能移动，发送方就停止发送

接收方每收到一个编号属于接收窗口的数据帧时，就将该帧收下，然后将接收窗口滑动一个位置，并发挥确认。这样，就会有新的编号落入接收窗口，从而继续接收新的帧。如果收到编号不在窗口内的帧，直接丢弃



连续ARQ协议

滑动窗口机制



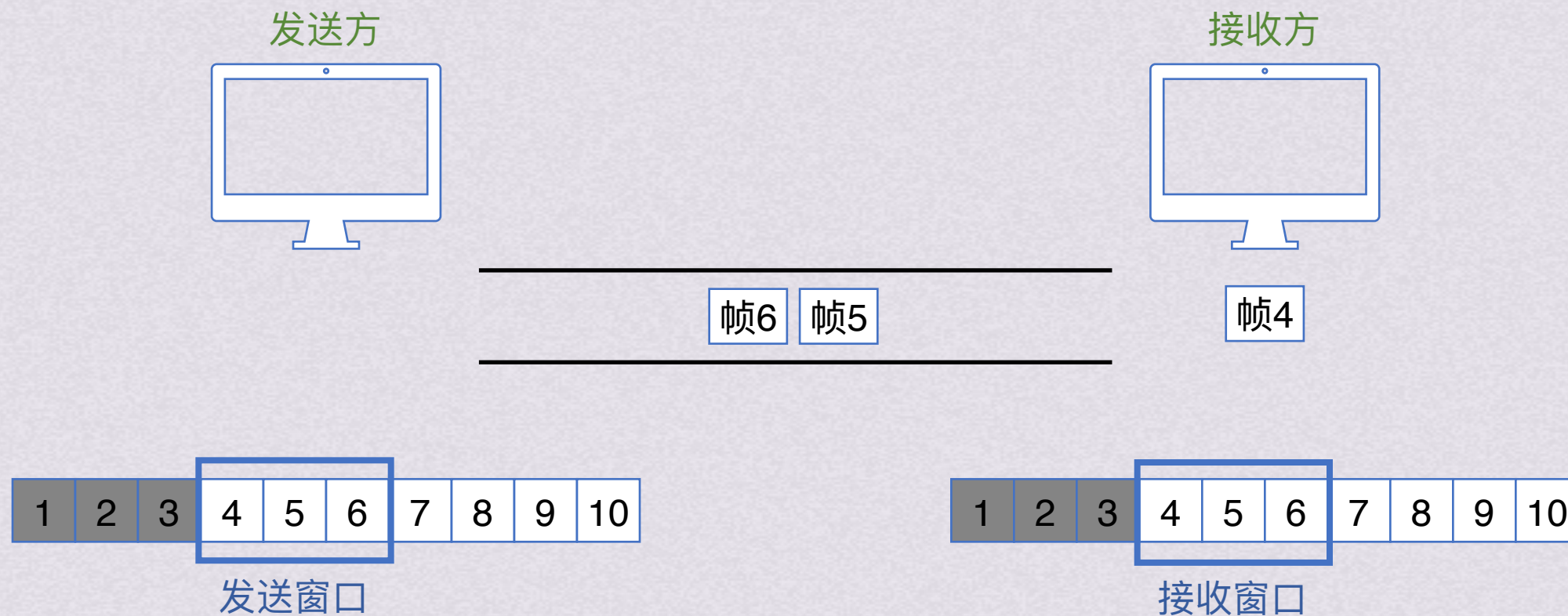
发送方每收到一个按序确认的确认帧，就将发送窗口滑动一个位置。这样，就会有新的编号落入发送窗口，从而继续发送。当窗口内的帧都已发送但都未收到确认帧时，窗口不能移动，发送方就停止发送

接收方每收到一个编号属于接收窗口的数据帧时，就将该帧收下，然后将接收窗口滑动一个位置，并发挥确认。这样，就会有新的编号落入接收窗口，从而继续接收新的帧。如果收到编号不在窗口内的帧，直接丢弃



连续ARQ协议

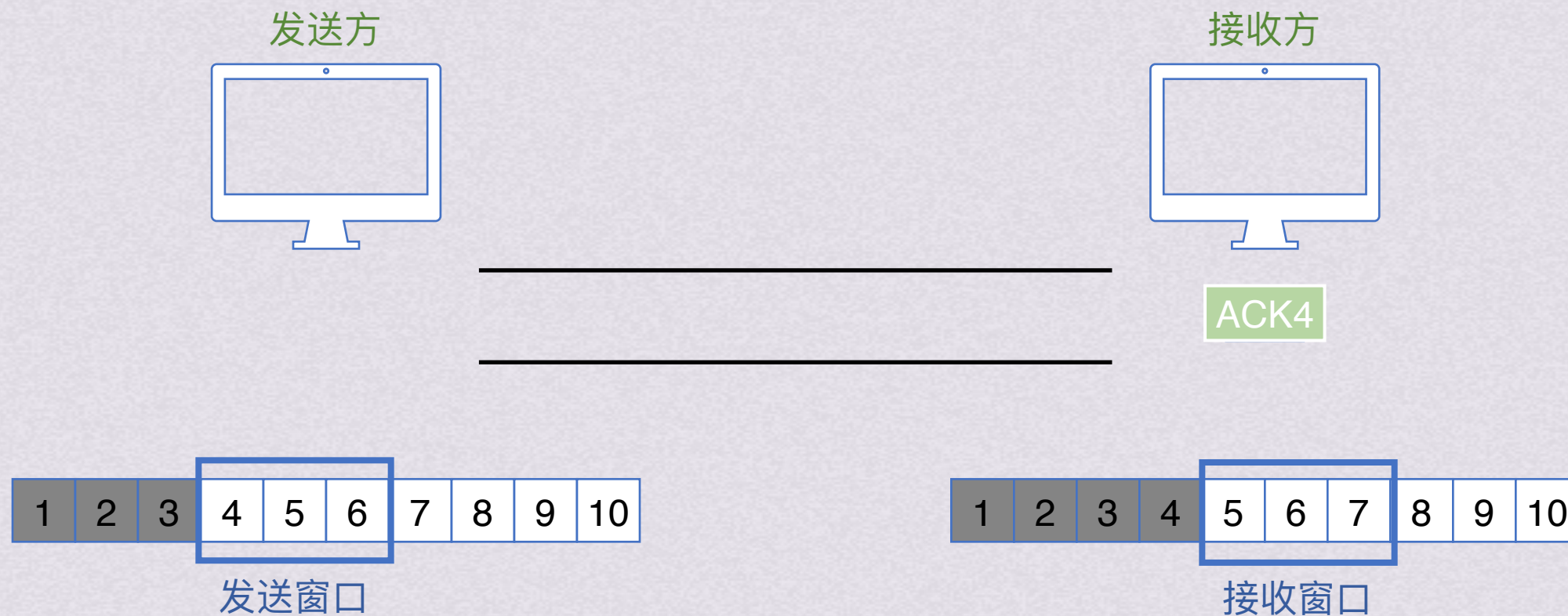
滑动窗口机制





连续ARQ协议

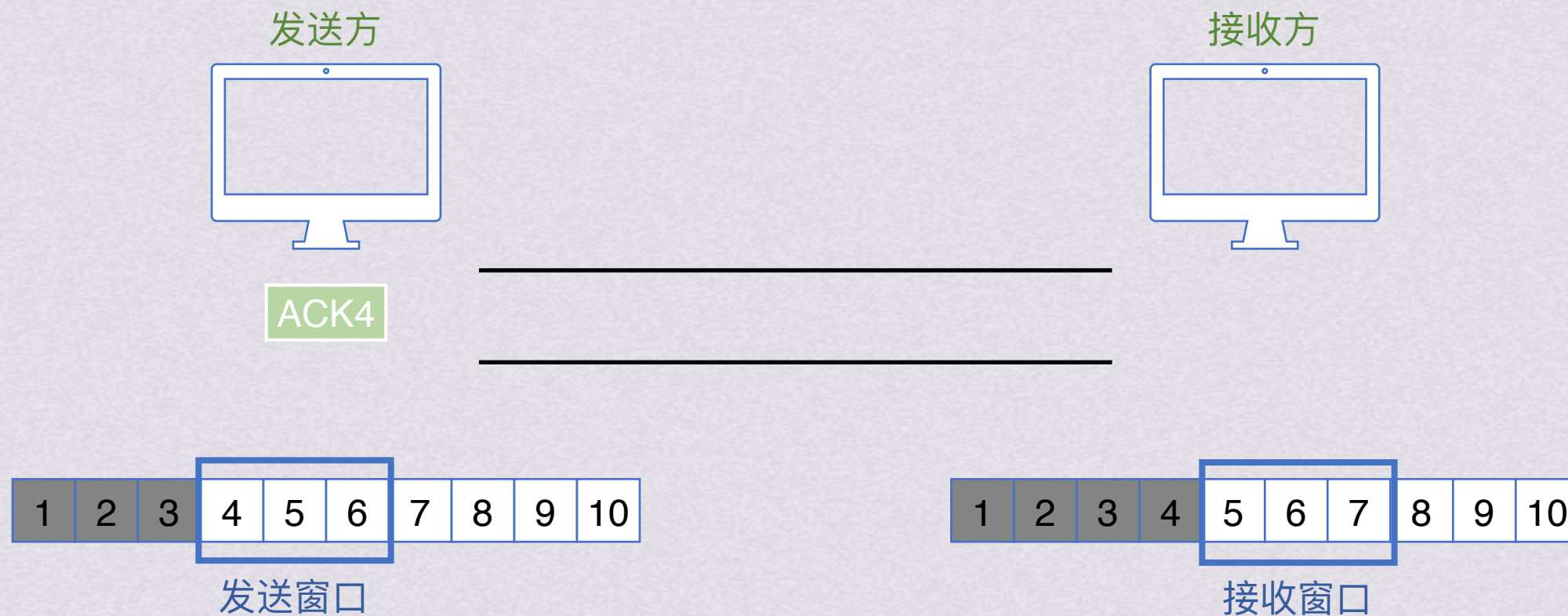
滑动窗口机制

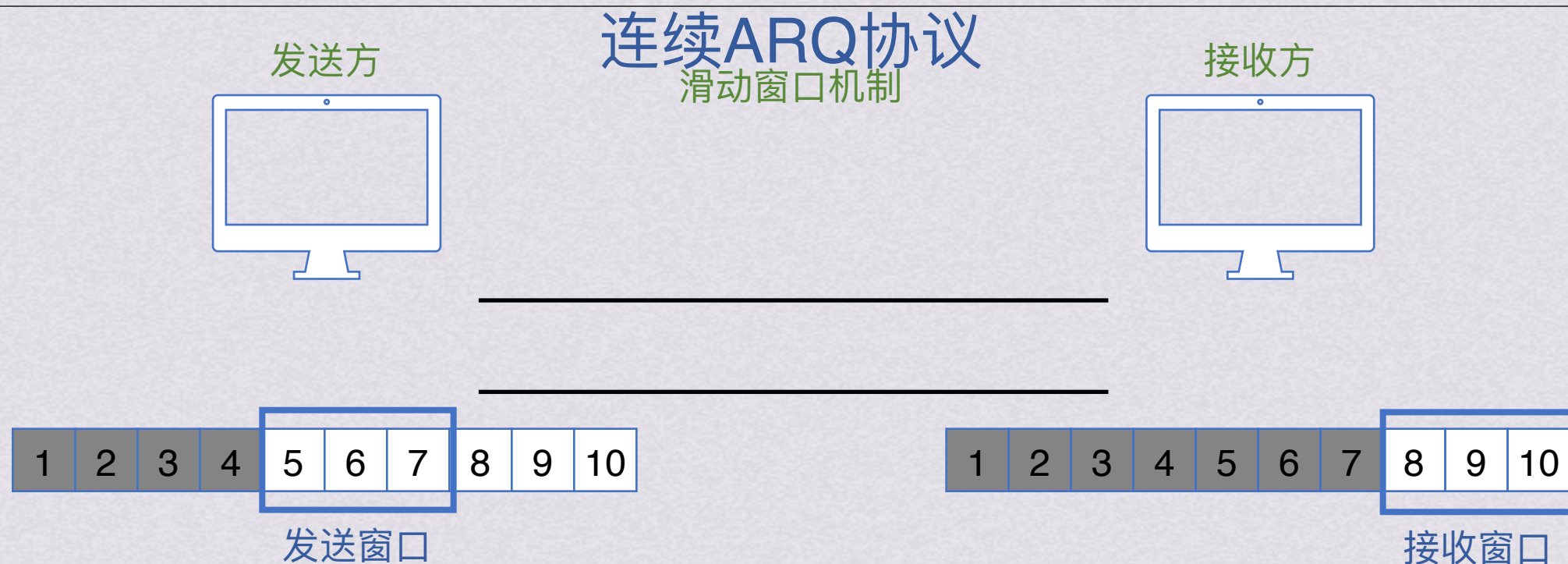




连续ARQ协议

滑动窗口机制



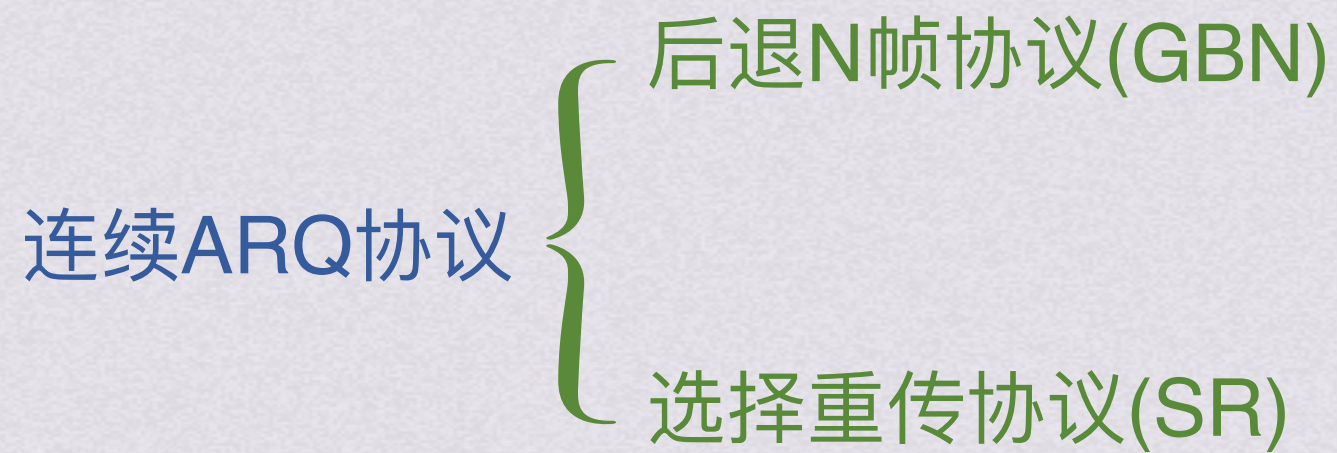


滑动窗口机制的特点：发送方只有收到一帧的确认，才能将窗口滑过该帧
接收方只有收到一帧并发送确认，才能将窗口滑过该帧

当接收窗口大小为1时，可以保证按序接收。

数据链路层的滑动窗口大小不变，而传输层的滑动窗口大小可变

对于连续ARQ协议来说，若采用n bit对帧编号，必须满足 $W_t + W_r \leq 2^n$





连续ARQ协议

后退N帧协议(GBN)

GBN允许发送方将发送窗口中的多个数据帧连续发送出去



发送方只能发送 发送窗口内的帧

接收方只接收 接收窗口内的帧

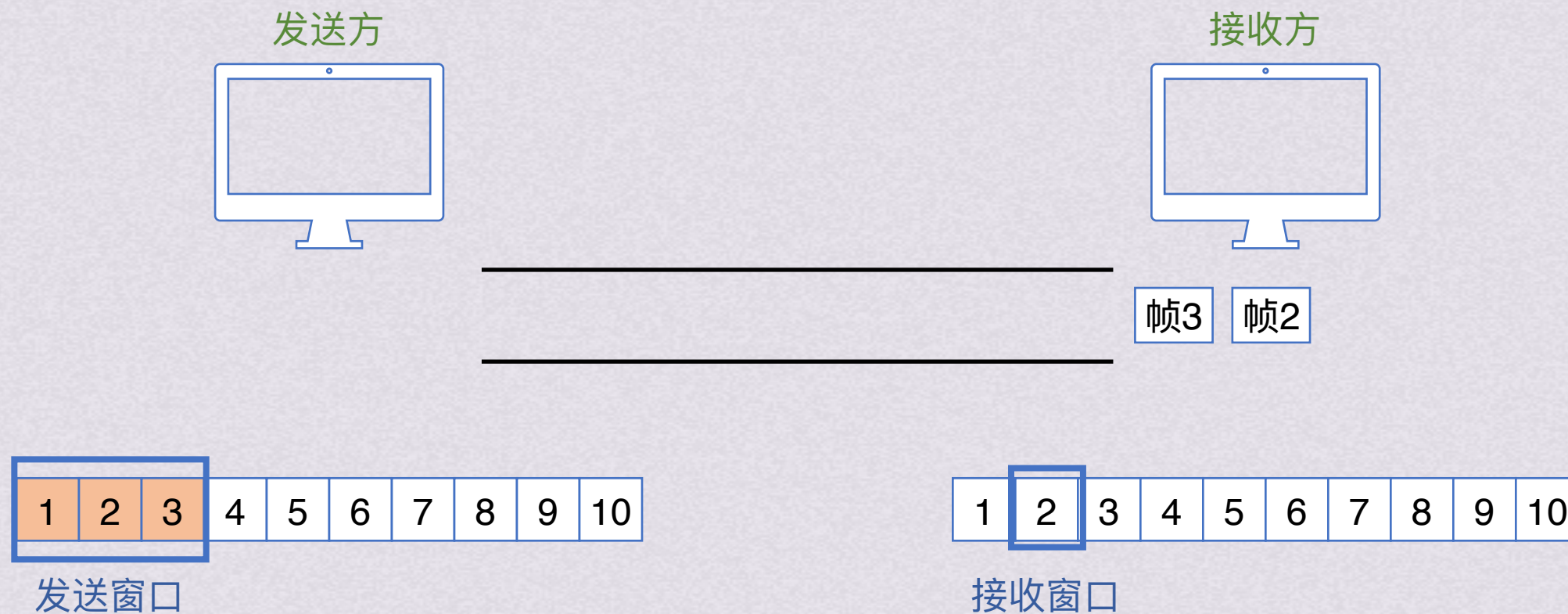


后退N帧协议(GBN)



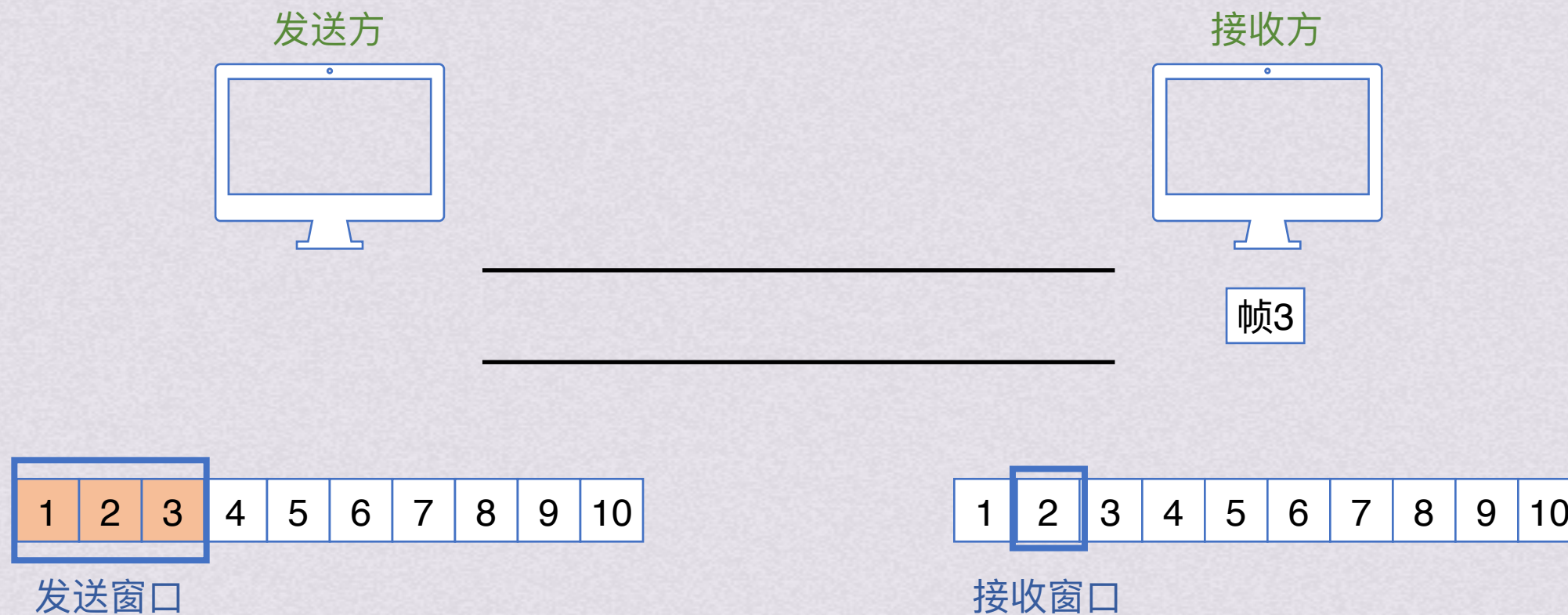


后退N帧协议(GBN)





后退N帧协议(GBN)





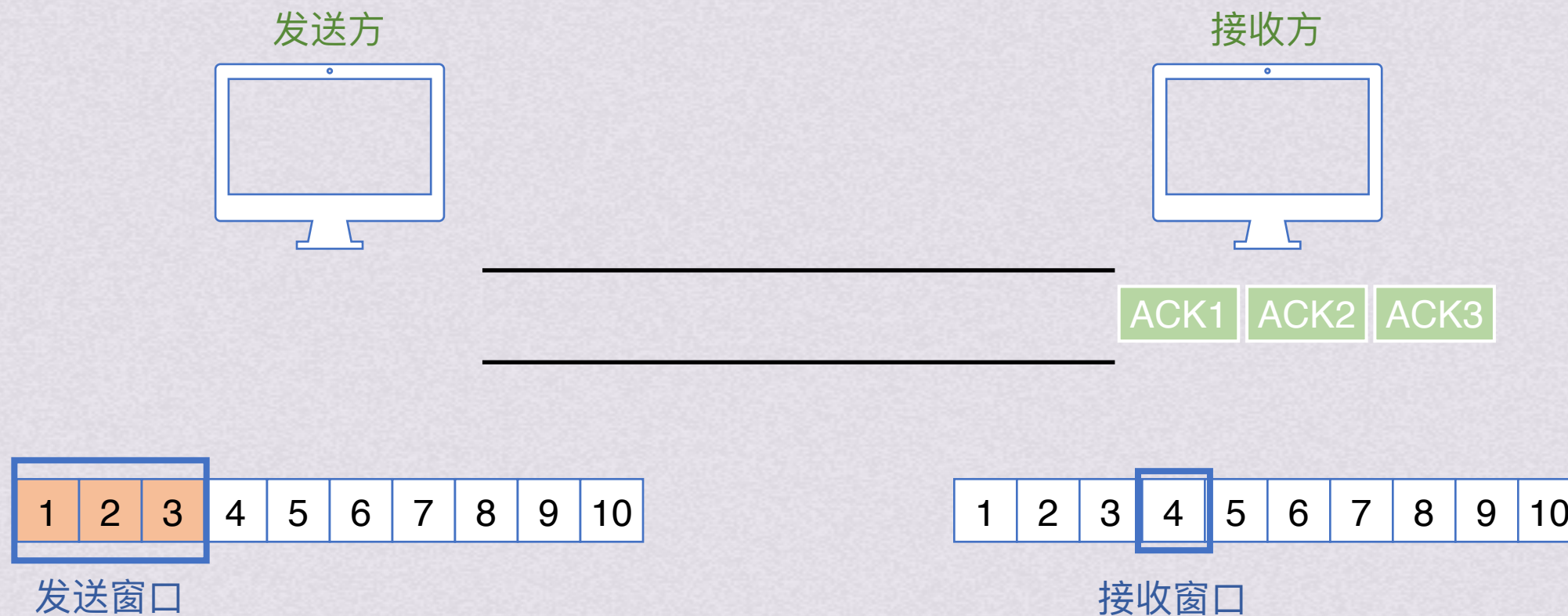
连续ARQ协议

后退N帧协议(GBN)





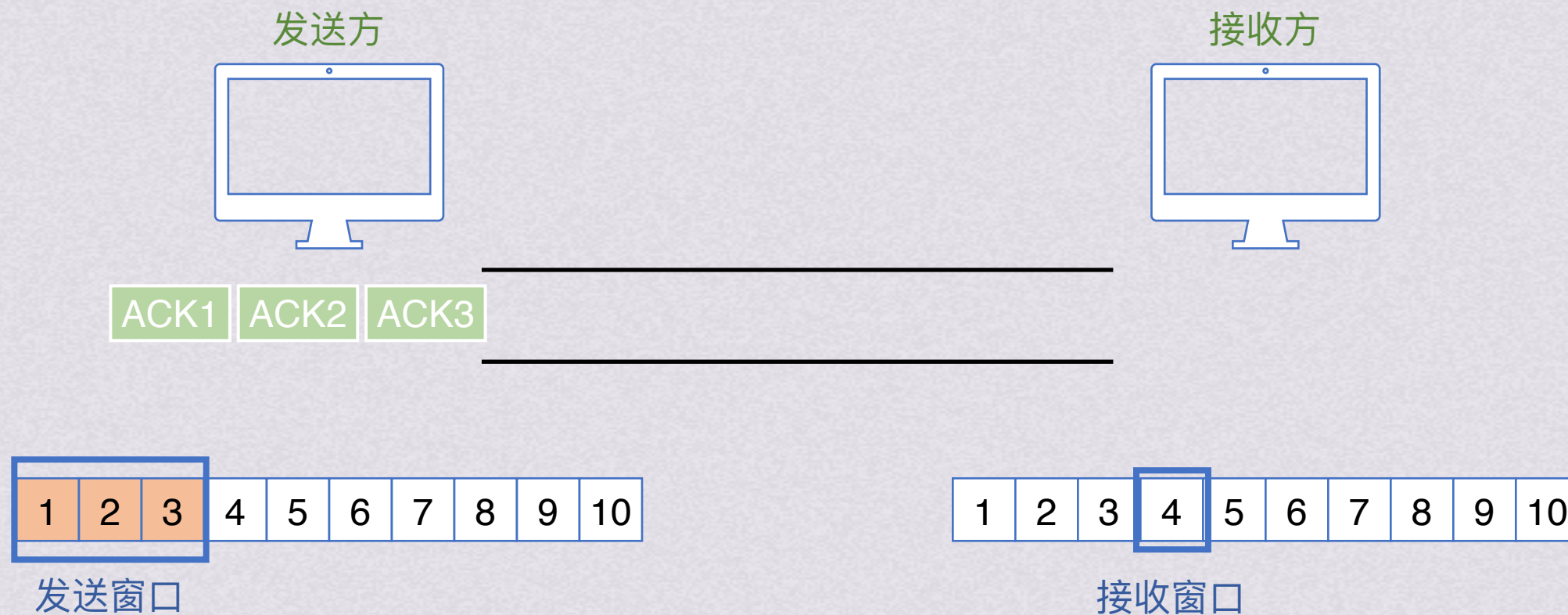
后退N帧协议(GBN)





连续ARQ协议

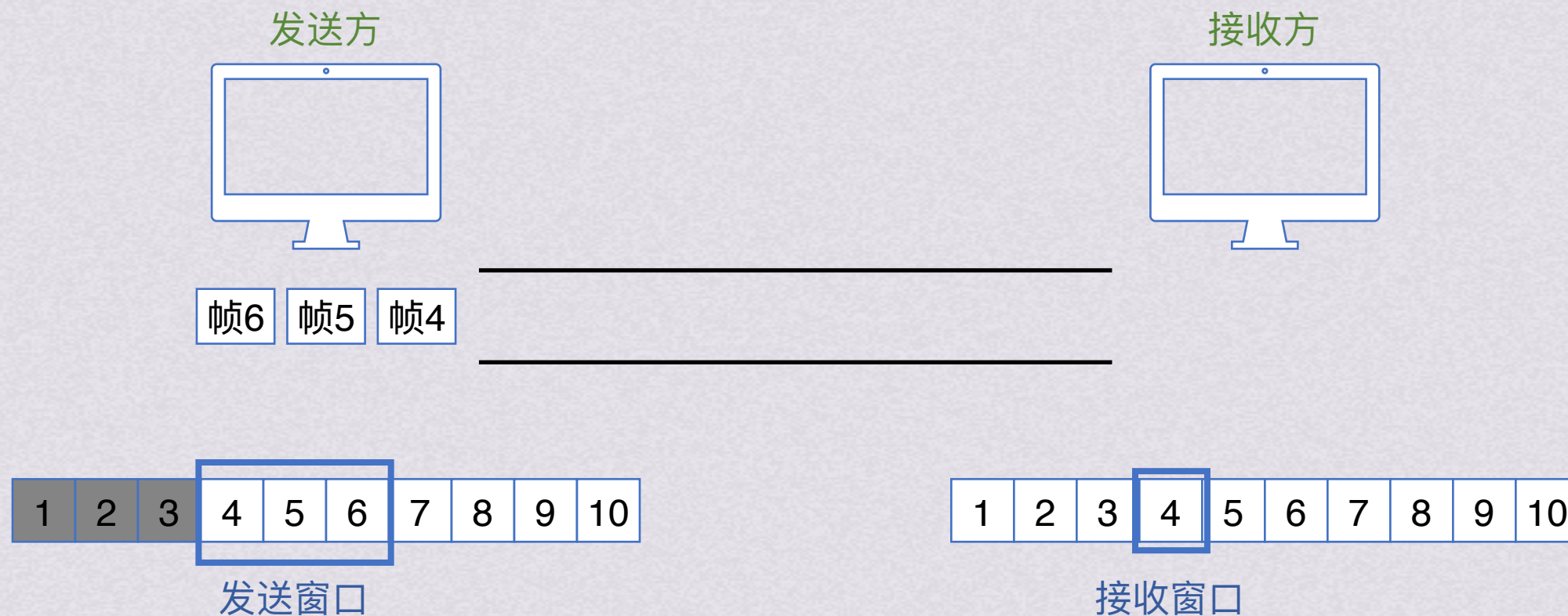
后退N帧协议(GBN)





连续ARQ协议

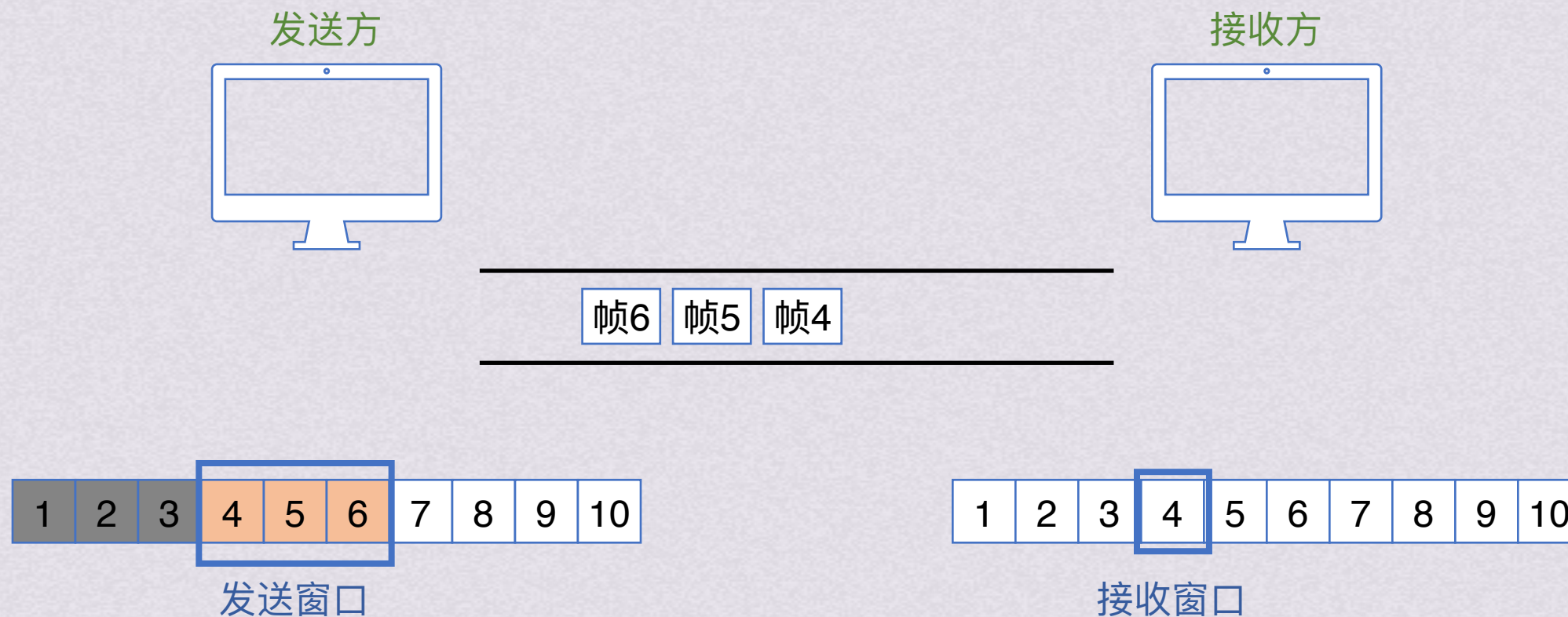
后退N帧协议(GBN)



收到ACK后可以将帧1,2,3从发送缓存中删除



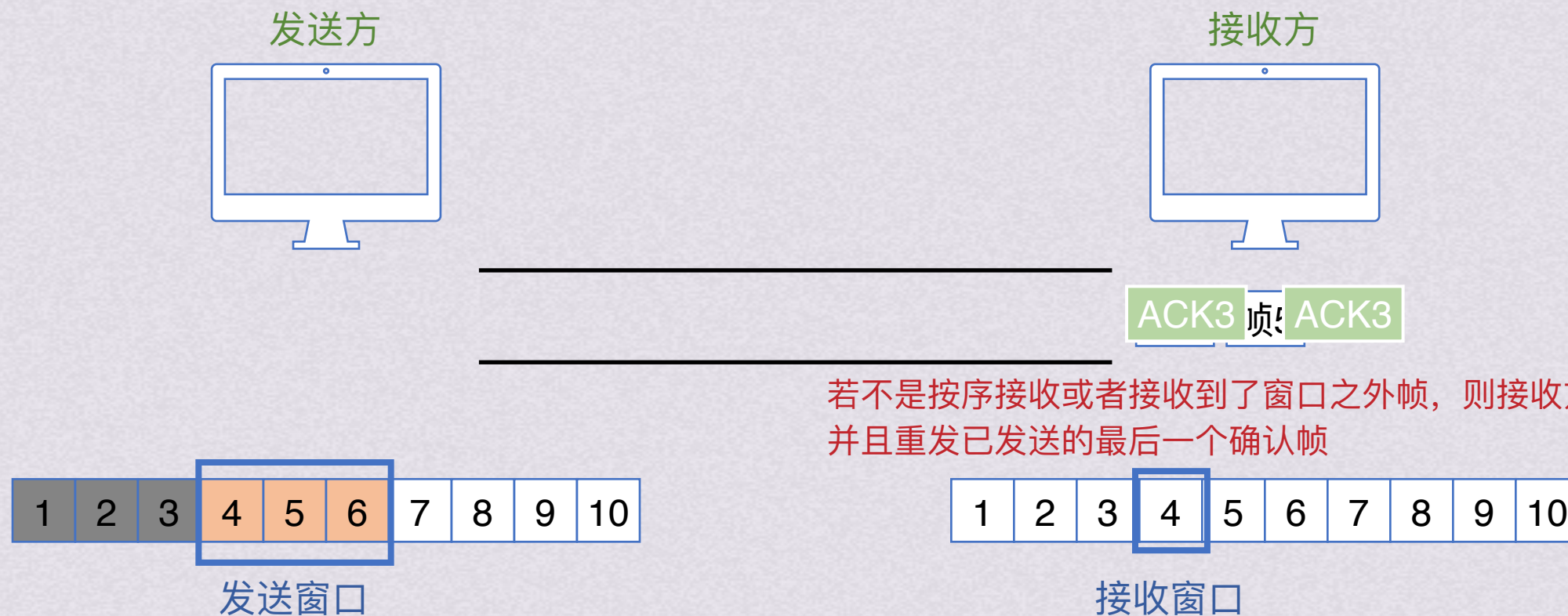
后退N帧协议(GBN)



收到ACK后可以将帧1,2,3从发送缓存中删除



后退N帧协议(GBN)



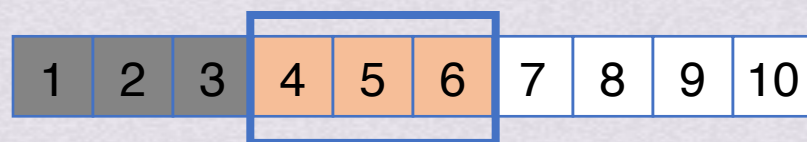
收到ACK后可以将帧1,2,3从发送缓存中删除



后退N帧协议(GBN)



发送方发现没有收到ACK4，超时后重传发送窗口内的3帧



发送窗口

收到ACK后可以将帧1,2,3从发送缓存中删除

若不是按序接收或者接收到了窗口之外帧，则接收方丢弃这些帧并且重发已发送的最后一个确认帧

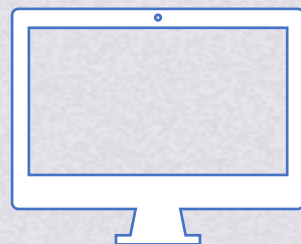


接收窗口

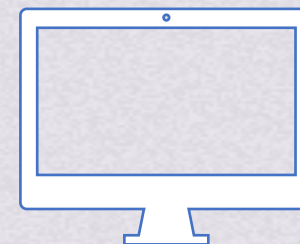


后退N帧协议(GBN)

发送方

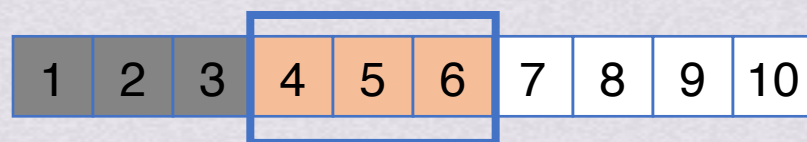


接收方



帧6 帧5 帧4

发送方发现没有收到ACK4，超时后重传发送窗口内的3帧



发送窗口

若不是按序接收或者接收到了窗口之外帧，则接收方丢弃这些帧并且重发已发送的最后一个确认帧



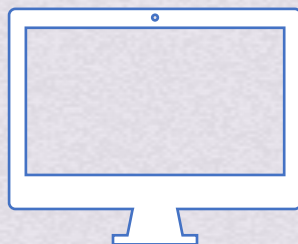
接收窗口

收到ACK后可以将帧1,2,3从发送缓存中删除

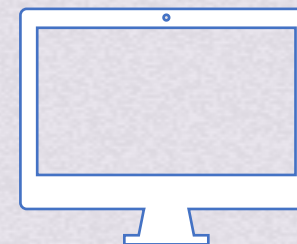


后退N帧协议(GBN)

发送方

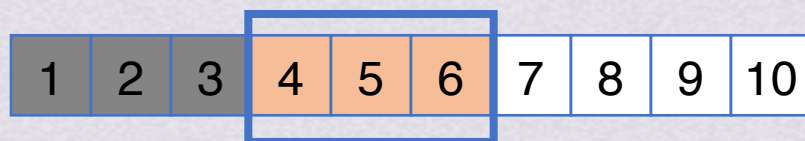


接收方



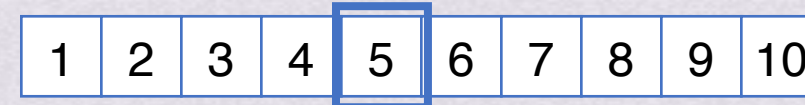
帧6 帧5

发送方发现没有收到ACK4，超时后重传发送窗口内的3帧



发送窗口

若不是按序接收或者接收到了窗口之外帧，则接收方丢弃这些帧并且重发已发送的最后一个确认帧



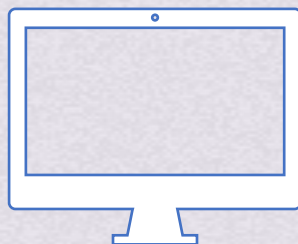
接收窗口

收到ACK后可以将帧1,2,3从发送缓存中删除

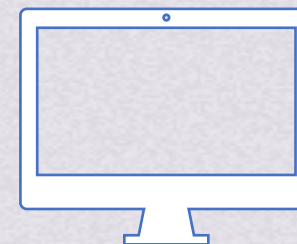


后退N帧协议(GBN)

发送方

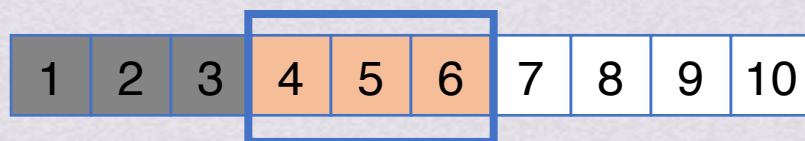


接收方



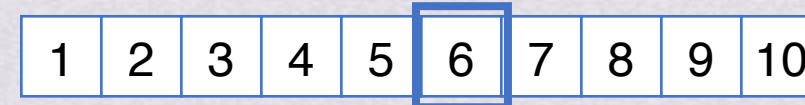
帧6

发送方发现没有收到ACK4，超时后重传发送窗口内的3帧



发送窗口

若不是按序接收或者接收到了窗口之外帧，则接收方丢弃这些帧并且重发已发送的最后一个确认帧



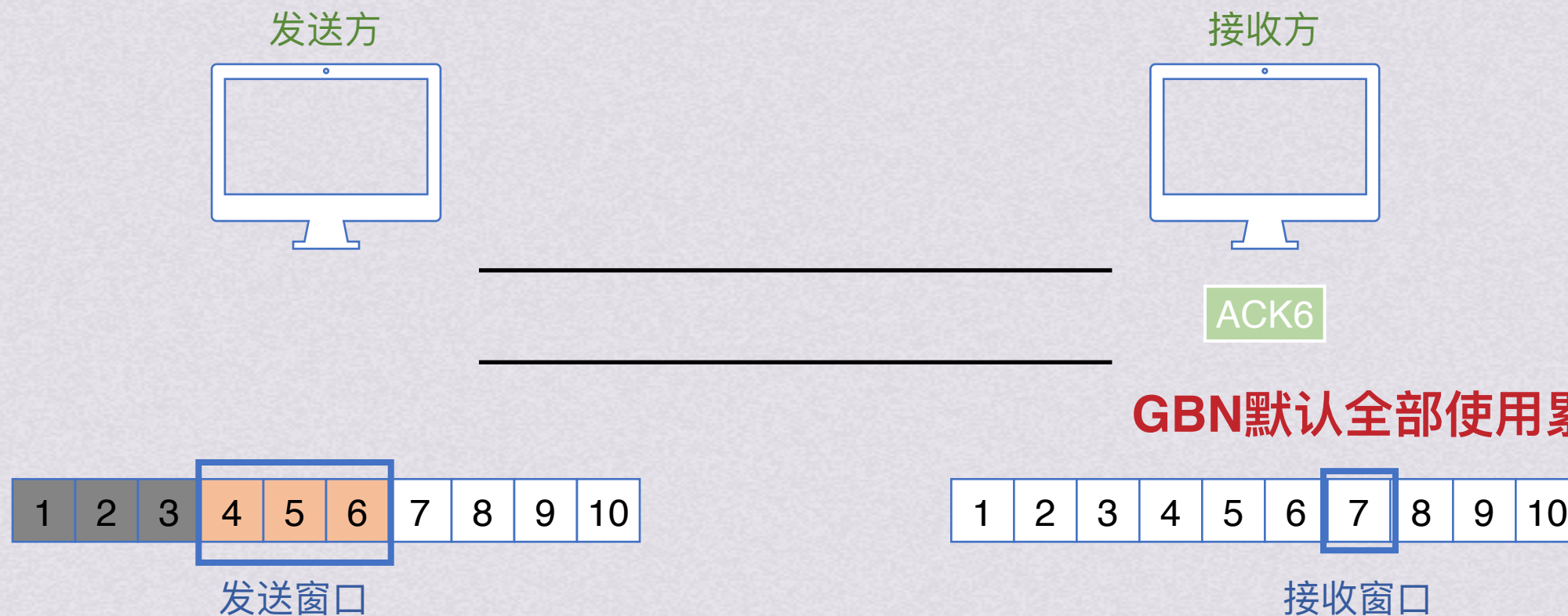
接收窗口

收到ACK后可以将帧1,2,3从发送缓存中删除



连续ARQ协议

后退N帧协议(GBN)



为了降低开销，GBN协议允许接收方进行累积确认，即不需要对收到的每一帧都进行确认，而是可以等到连续收到多个正确的数据帧后，对最后一个帧发回确认

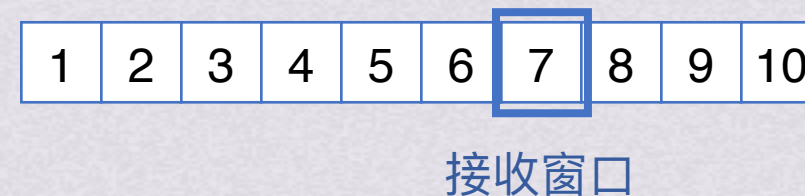
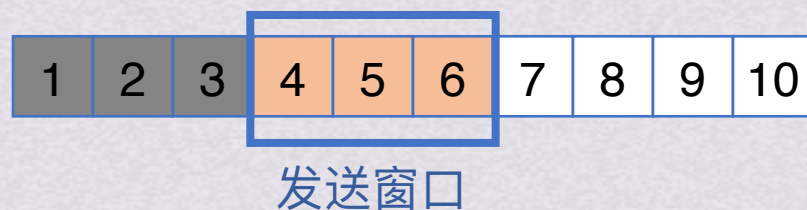


后退N帧协议(GBN)



采用累计确认时，发送了确认帧意味着确认帧及之前的帧全都已经正确接收

GBN默认全部使用累计确认！



为了降低开销，GBN协议允许接收方进行累积确认，即不需要对收到的每一帧都进行确认，而是可以等到连续收到多个正确的数据帧后，对最后一个帧发回确认

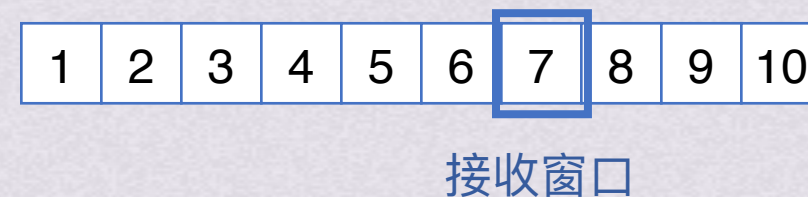
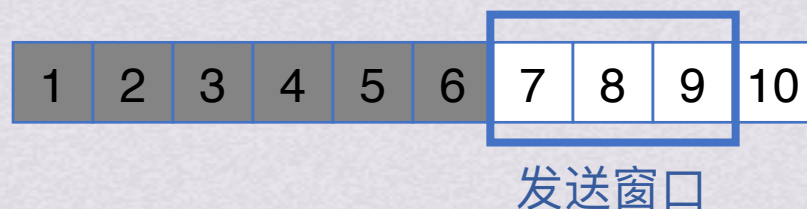


后退N帧协议(GBN)



采用累计确认时，发送了确认帧意味着确认帧及之前的帧全都已经正确接收

GBN默认全部使用累计确认！



为了降低开销，GBN协议允许接收方进行累积确认，即不需要对收到的每一帧都进行确认，而是可以等到连续收到多个正确的数据帧后，对最后一个帧发回确认



后退N帧协议(GBN)



因为需要对帧进行编号，所以我们需要决定用几个比特给帧编号
假设用3个比特，帧编号就是0~7之间循环



后退N帧协议(GBN)



因为需要对帧进行编号，所以我们需要决定用几个比特给帧编号
假设用3个比特，帧编号就是0~7之间循环



后退N帧协议(GBN)



因为需要对帧进行编号，所以我们需要决定用几个比特给帧编号
假设用3个比特，帧编号就是0~7之间循环

在后退n帧协议中，发送窗口 W_t 的范围是 $1 < W_t \leq 2^3 - 1$;接收窗口是1
($2^t - 1$, t是给帧编号的比特个数)



后退N帧协议(GBN)



在后退n帧协议中, 发送窗口 W_t 的范围是 $1 < W_t \leq 2^t - 1$; 接收窗口是1
($2^t - 1$, t 是给帧编号的比特个数)

那如果就是要用 $W_t=8$ 会怎样呢



后退N帧协议(GBN)



在后退n帧协议中，发送窗口 W_t 的范围是 $1 < W_t \leq 2^3 - 1$;接收窗口是1
($2^t - 1$, t 是给帧编号的比特个数)

那如果就是要用 $W_t=8$ 会怎样呢



后退N帧协议(GBN)



在后退n帧协议中，发送窗口 W_t 的范围是 $1 < W_t \leq 2^3 - 1$;接收窗口是1
($2^t - 1$, t 是给帧编号的比特个数)

那如果就是要用 $W_t=8$ 会怎样呢



后退N帧协议(GBN)

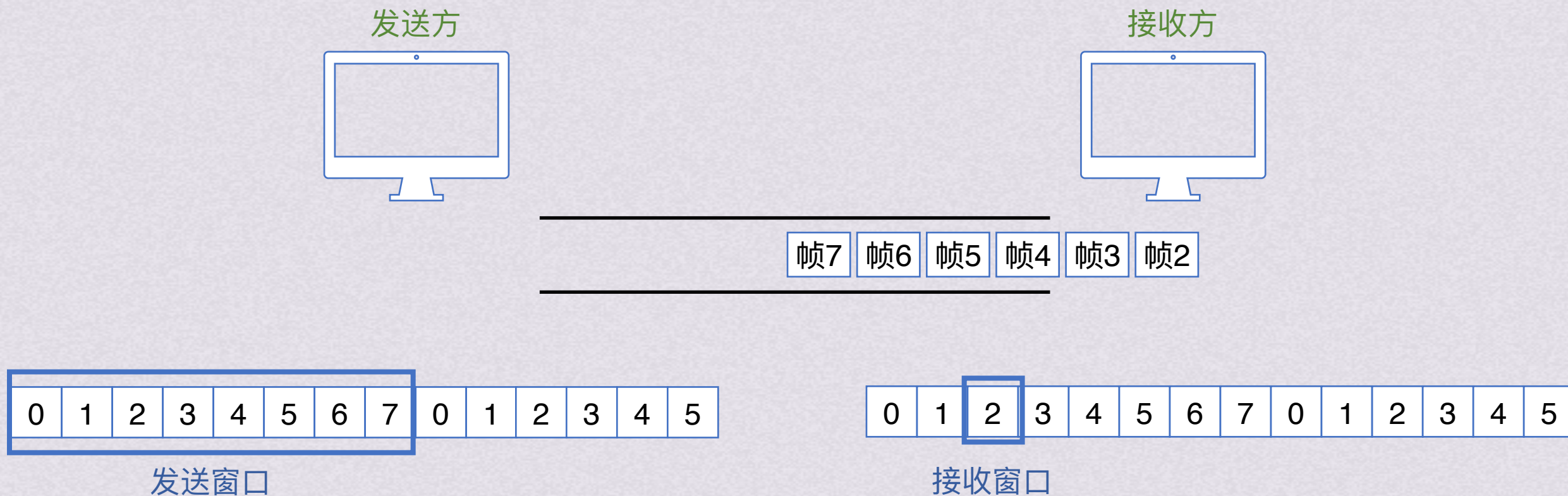


在后退n帧协议中，发送窗口 W_t 的范围是 $1 < W_t \leq 2^3 - 1$;接收窗口是1
($2^t - 1$, t 是给帧编号的比特个数)

那如果就是要用 $W_t=8$ 会怎样呢



后退N帧协议(GBN)

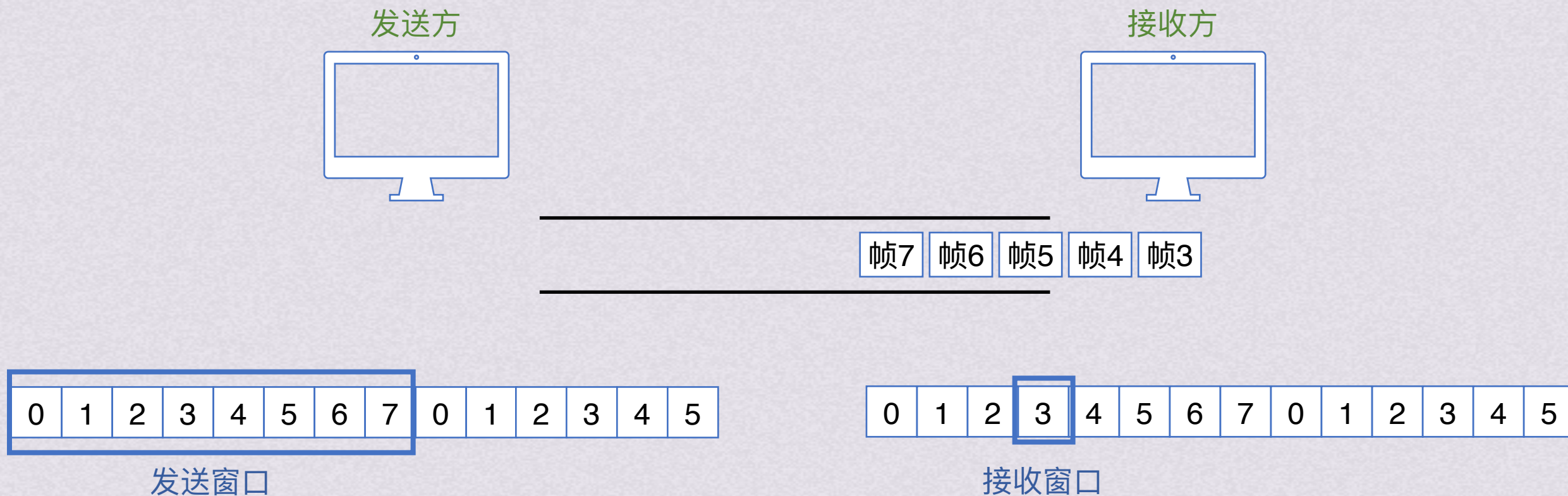


在后退n帧协议中，发送窗口 W_t 的范围是 $1 < W_t \leq 2^3 - 1$;接收窗口是1
($2^t - 1$, t 是给帧编号的比特个数)

那如果就是要用 $W_t=8$ 会怎样呢



后退N帧协议(GBN)

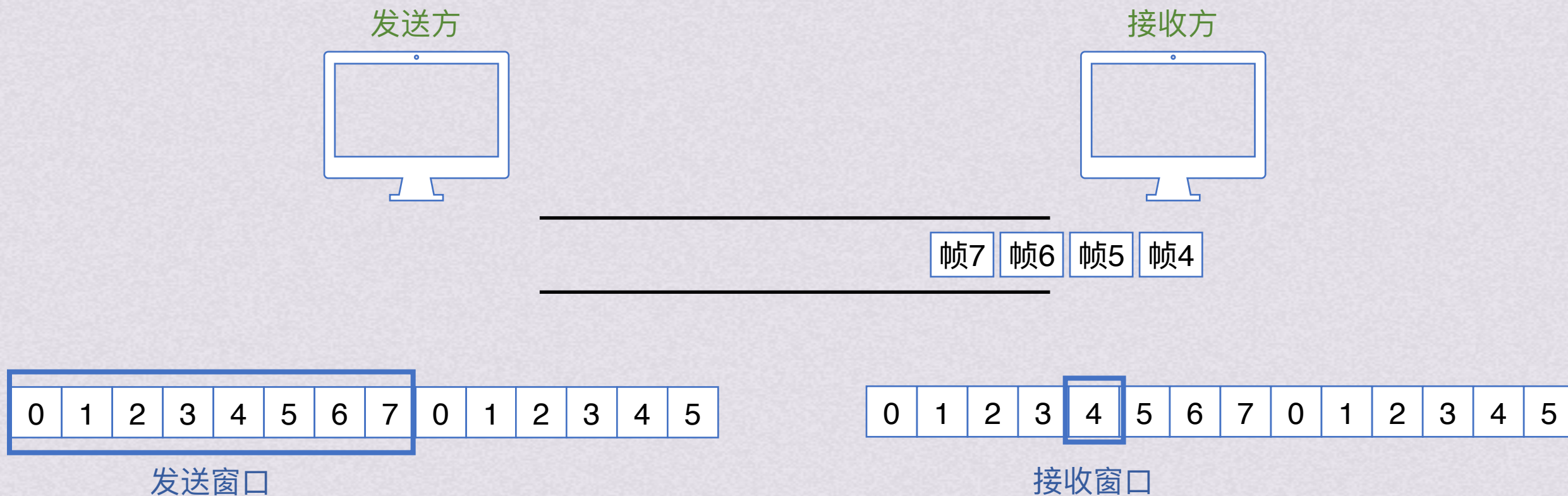


在后退n帧协议中，发送窗口 W_t 的范围是 $1 < W_t \leq 2^3 - 1$ ；接收窗口是1
($2^t - 1$, t 是给帧编号的比特个数)

那如果就是要用 $W_t=8$ 会怎样呢



后退N帧协议(GBN)

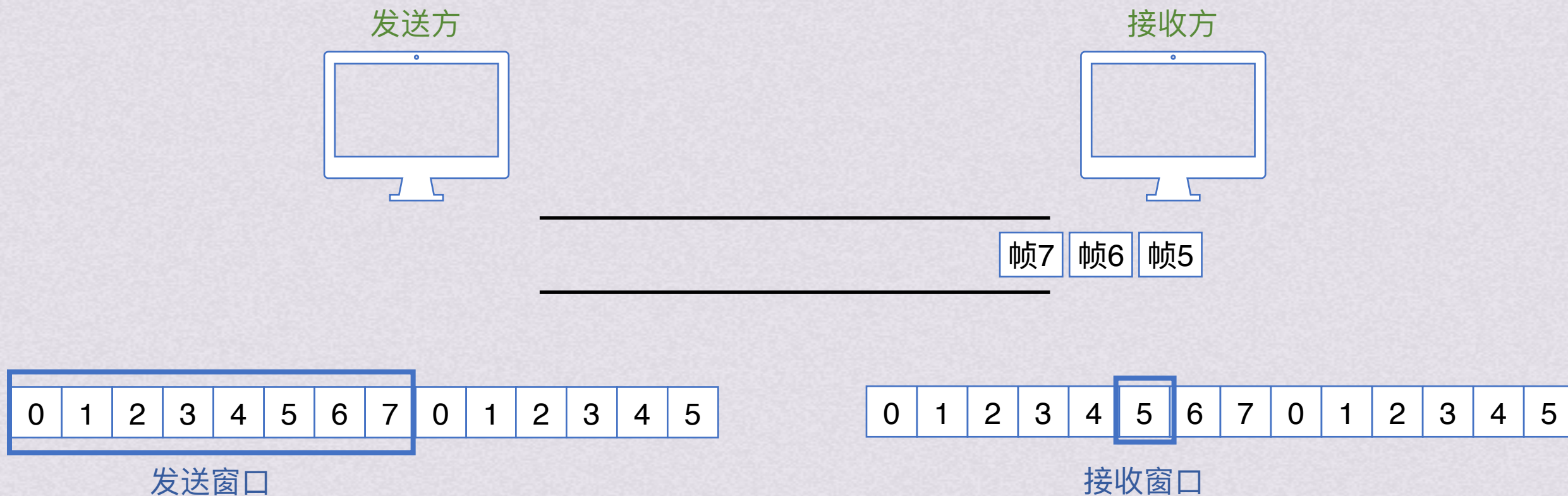


在后退n帧协议中，发送窗口 W_t 的范围是 $1 < W_t \leq 2^t - 1$;接收窗口是1
($2^t - 1$, t 是给帧编号的比特个数)

那如果就是要用 $W_t=8$ 会怎样呢



后退N帧协议(GBN)

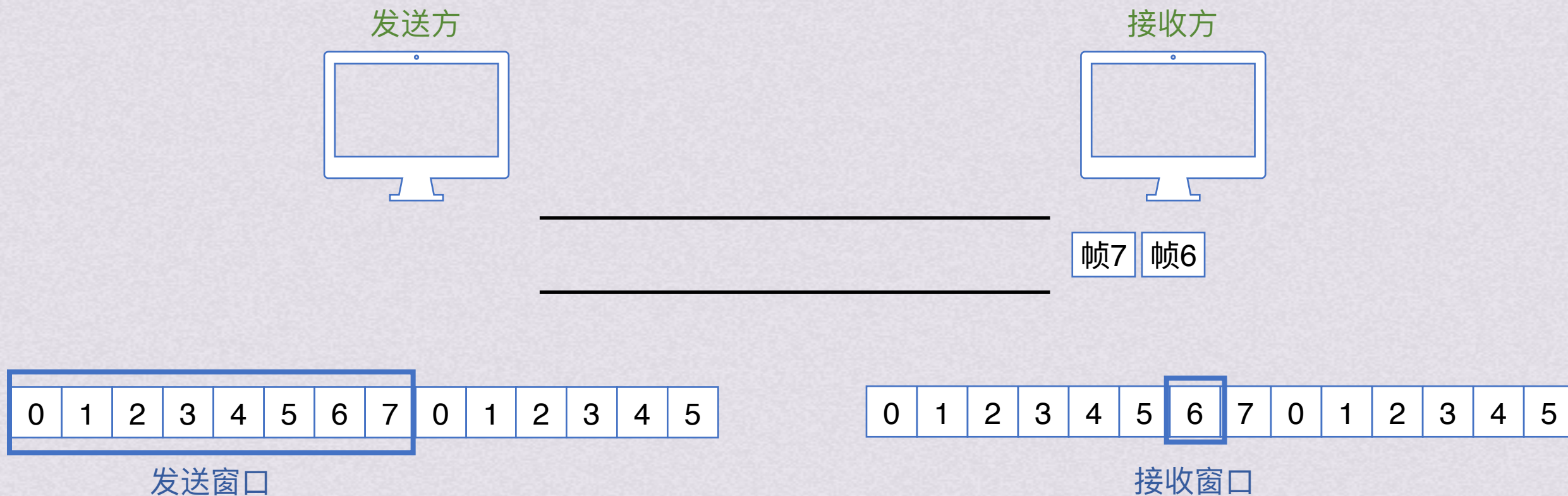


在后退n帧协议中，发送窗口 W_t 的范围是 $1 < W_t \leq 2^t - 1$ ；接收窗口是1
($2^t - 1$, t 是给帧编号的比特个数)

那如果就是要用 $W_t=8$ 会怎样呢



后退N帧协议(GBN)

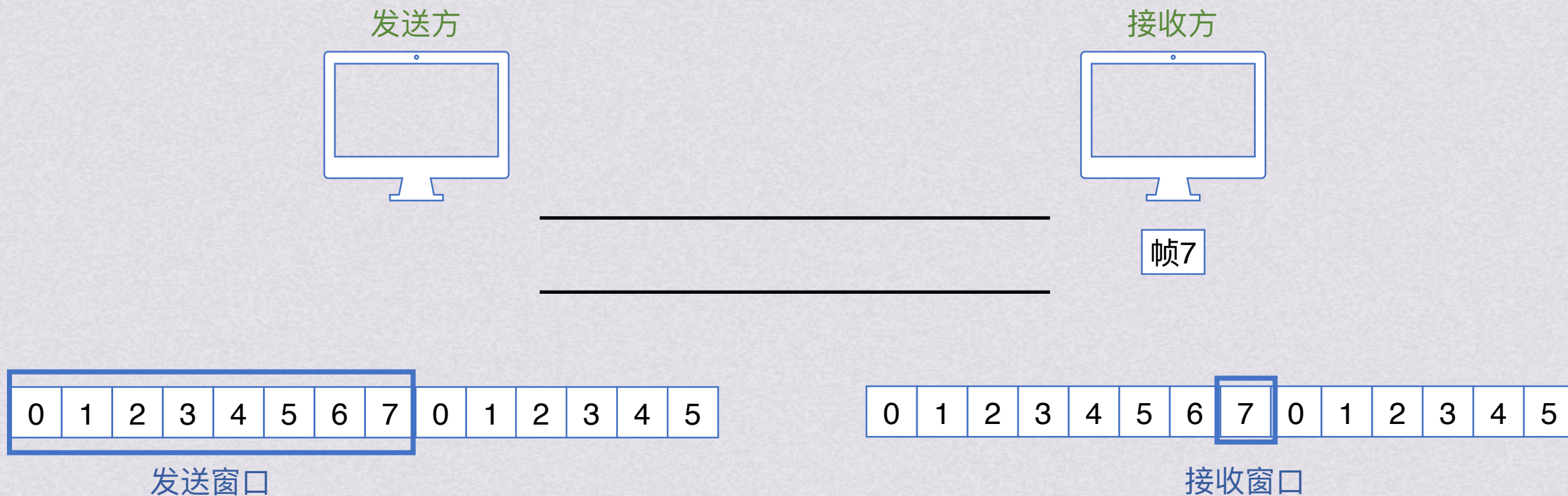


在后退n帧协议中，发送窗口 W_t 的范围是 $1 < W_t \leq 2^3 - 1$;接收窗口是1
($2^t - 1$, t 是给帧编号的比特个数)

那如果就是要用 $W_t=8$ 会怎样呢



后退N帧协议(GBN)

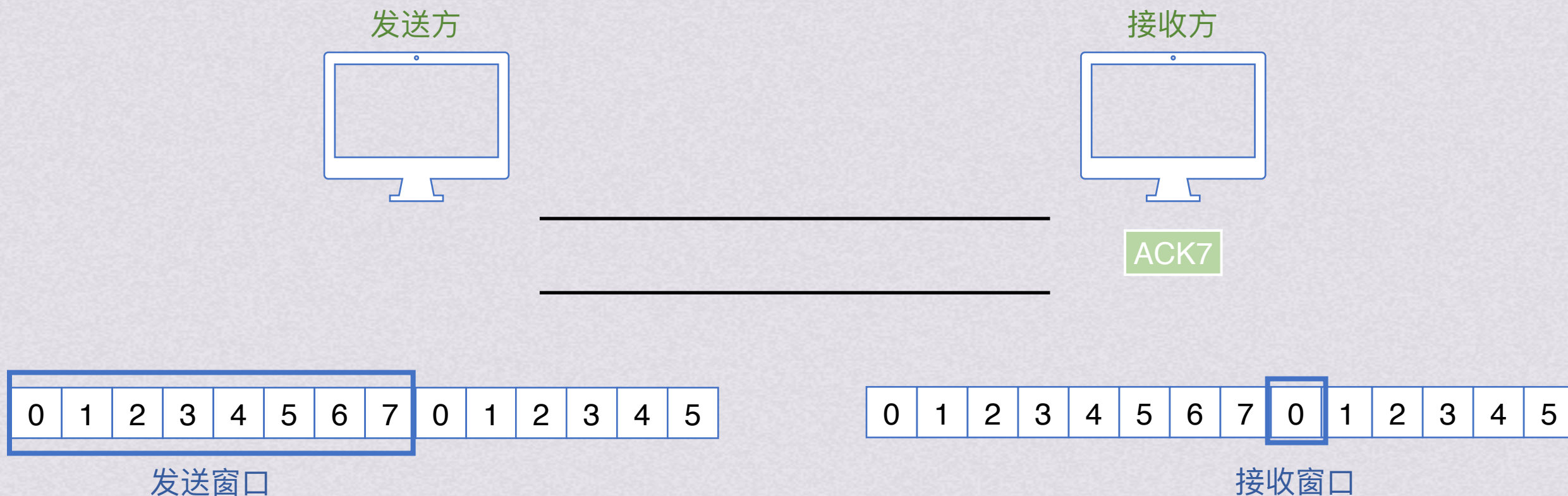


在后退n帧协议中，发送窗口 W_t 的范围是 $1 < W_t \leq 2^3 - 1$;接收窗口是1
($2^t - 1$, t 是给帧编号的比特个数)

那如果就是要用 $W_t=8$ 会怎样呢



后退N帧协议(GBN)



在后退n帧协议中，发送窗口 W_t 的范围是 $1 < W_t \leq 2^t - 1$;接收窗口是1
($2^t - 1$, t 是给帧编号的比特个数)

那如果就是要用 $W_t=8$ 会怎样呢



后退N帧协议(GBN)



在后退n帧协议中，发送窗口 W_t 的范围是 $1 < W_t \leq 2^t - 1$;接收窗口是1
($2^t - 1$, t 是帧编号的比特个数)

那如果就是要用 $W_t=8$ 会怎样呢



后退N帧协议(GBN)

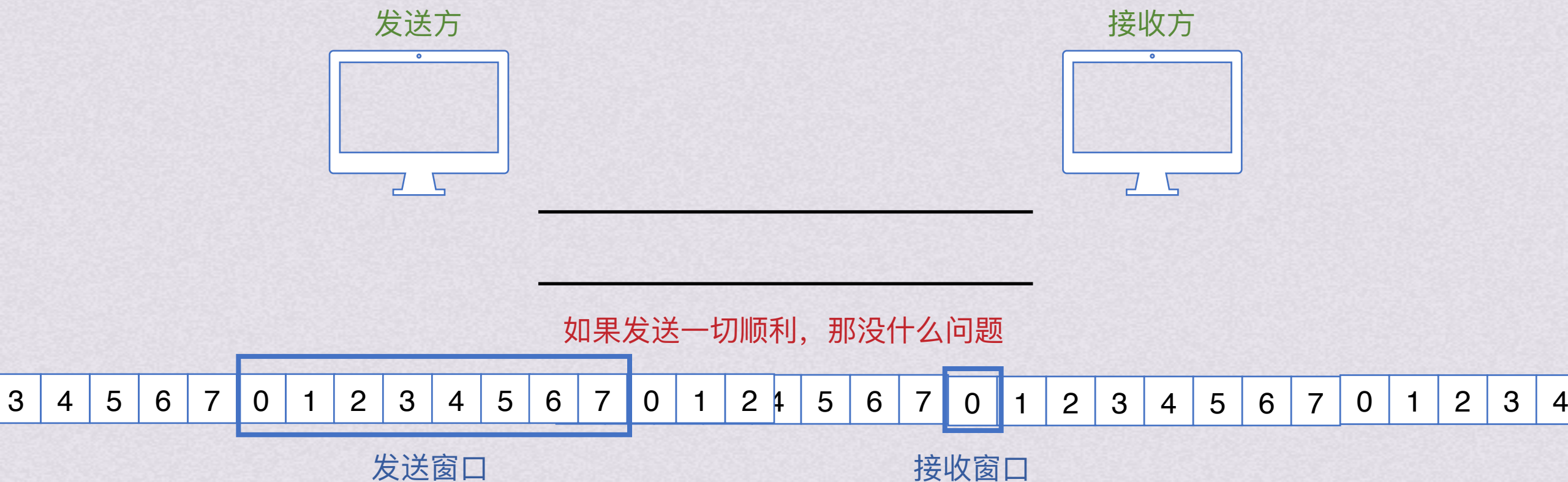


在后退n帧协议中，发送窗口 W_t 的范围是 $1 < W_t \leq 2^t - 1$;接收窗口是1
($2^t - 1$, t 是帧编号的比特个数)

那如果就是要用 $W_t=8$ 会怎样呢



后退N帧协议(GBN)

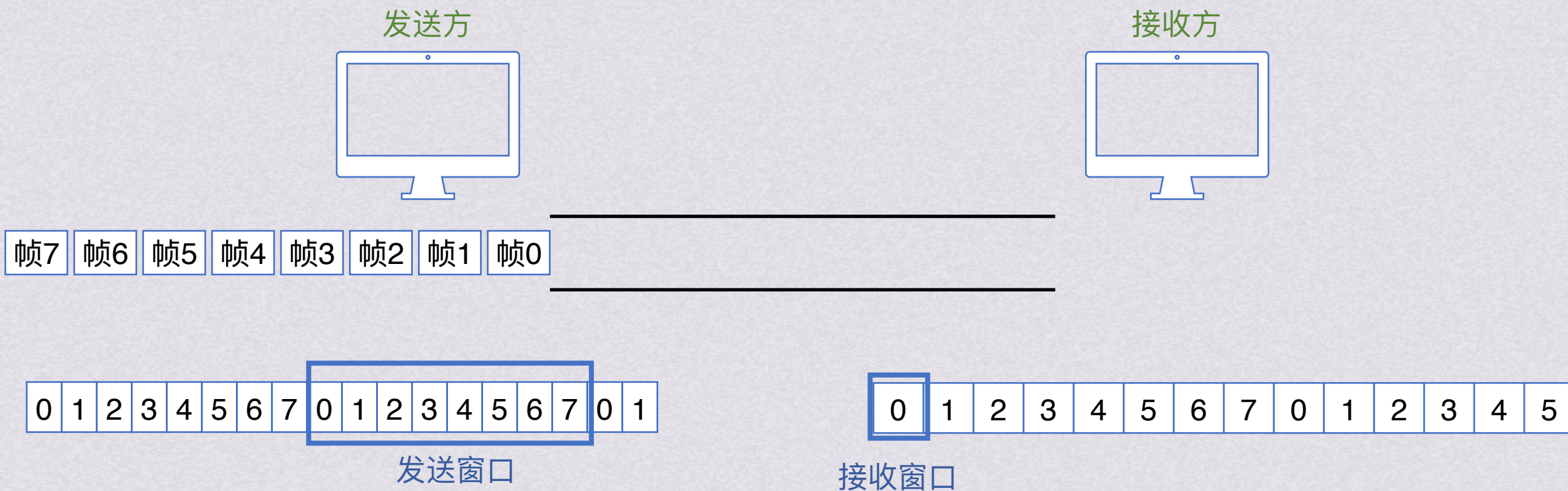


在后退n帧协议中, 发送窗口 W_t 的范围是 $1 < W_t \leq 2^t - 1$; 接收窗口是1
($2^t - 1$, t 是帧编号的比特个数)

那如果就是要用 $W_t=8$ 会怎样呢



后退N帧协议(GBN)



在后退n帧协议中，发送窗口 W_t 的范围是 $1 < W_t \leq 2^3 - 1$;接收窗口是1
($2^t - 1$, t 是给帧编号的比特个数)

那如果就是要用 $W_t=8$ 会怎样呢



后退N帧协议(GBN)



在后退n帧协议中，发送窗口 W_t 的范围是 $1 < W_t \leq 2^3 - 1$;接收窗口是1
($2^t - 1$, t 是给帧编号的比特个数)

那如果就是要用 $W_t=8$ 会怎样呢



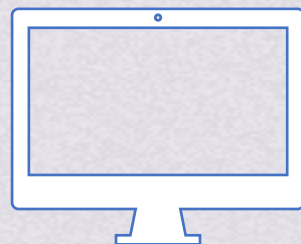
连续ARQ协议

后退N帧协议(GBN)

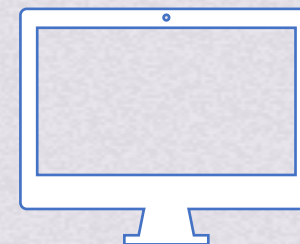
这时候发送方就无法区分ACK7的意思是这一组数据安全收到
请求发送下一组
还是这一组数据的0帧丢失

此时接收方发现没有收到接收窗口的0号帧
于是发送上一帧的确认帧，也就是ACK7

发送方

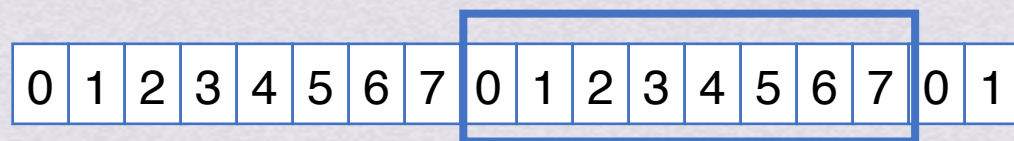


接收方

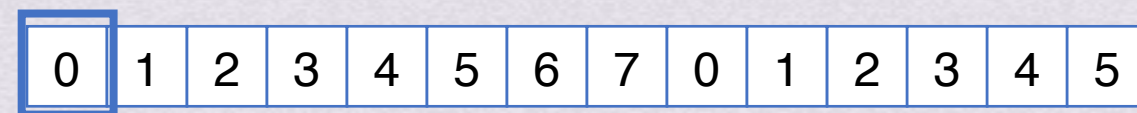


帧7 ACK7 帧5 帧4 帧3 帧2 帧1

如果帧0在传输过程中丢失，其他帧都到达



发送窗口



接收窗口

在后退n帧协议中，发送窗口 W_t 的范围是 $1 < W_t \leq 2^t - 1$;接收窗口是1
($2^t - 1$, t是给帧编号的比特个数)

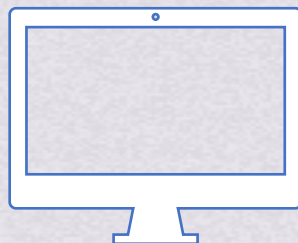
那如果就是要用 $W_t=8$ 会怎样呢



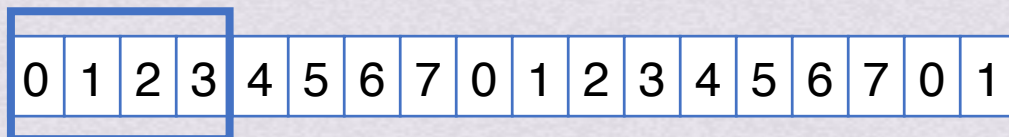
连续ARQ协议

选择重传协议(SR)

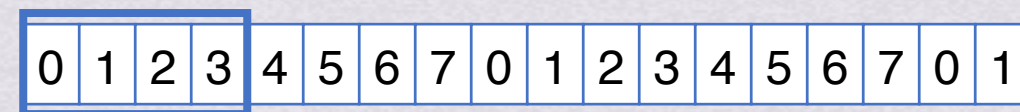
发送方



接收方



发送窗口



接收窗口

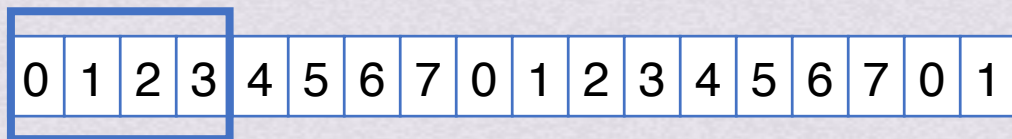
GBN协议中，数据如果不是按序到达的话，失序的部分会被直接丢弃，这样造成了很多通信资源浪费
因此出现了选择重传SR协议，增大了接收窗口，这样就能暂时收下失序的帧，等窗口内缺的帧收齐后一起上交上层

因为要让发送方只重传那些丢失/出错的帧
所以不再使用累计确认，改为逐帧确认

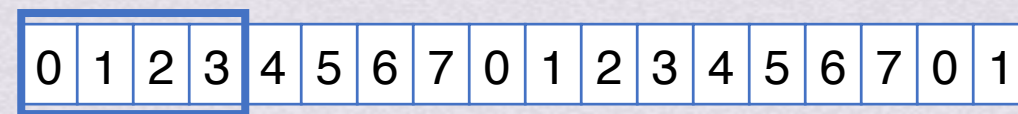


连续ARQ协议

选择重传协议(SR)



发送窗口



接收窗口

SR只重传出现差错的数据帧和计时器超时的数据帧：

当发送方发送的某个数据帧超时仍未收到确认
将自动重传该帧

当接收方检测到某个数据帧出错
它会向发送方发送一个否定帧NAK
要求发送方立刻重传该帧



连续ARQ协议

选择重传协议(SR)



SR只重传出现差错的数据帧和计时器超时的数据帧：

当发送方发送的某个数据帧超时仍未收到确认
将自动重传该帧

当接收方检测到某个数据帧出错
它会向发送方发送一个否定帧NAK
要求发送方立刻重传该帧



连续ARQ协议

选择重传协议(SR)



SR只重传出现差错的数据帧和计时器超时的数据帧：

当发送方发送的某个数据帧超时仍未收到确认
将自动重传该帧

当接收方检测到某个数据帧出错
它会向发送方发送一个否定帧NAK
要求发送方立刻重传该帧



连续ARQ协议

选择重传协议(SR)



SR只重传出现差错的数据帧和计时器超时的数据帧:

当发送方发送的某个数据帧超时仍未收到确认
将自动重传该帧

当接收方检测到某个数据帧出错
它会向发送方发送一个否定帧NAK
要求发送方立刻重传该帧



连续ARQ协议

选择重传协议(SR)



SR只重传出现差错的数据帧和计时器超时的数据帧:

当发送方发送的某个数据帧超时仍未收到确认
将自动重传该帧

当接收方检测到某个数据帧出错
它会向发送方发送一个否定帧NAK
要求发送方立刻重传该帧



连续ARQ协议

选择重传协议(SR)



SR只重传出现差错的数据帧和计时器超时的数据帧:

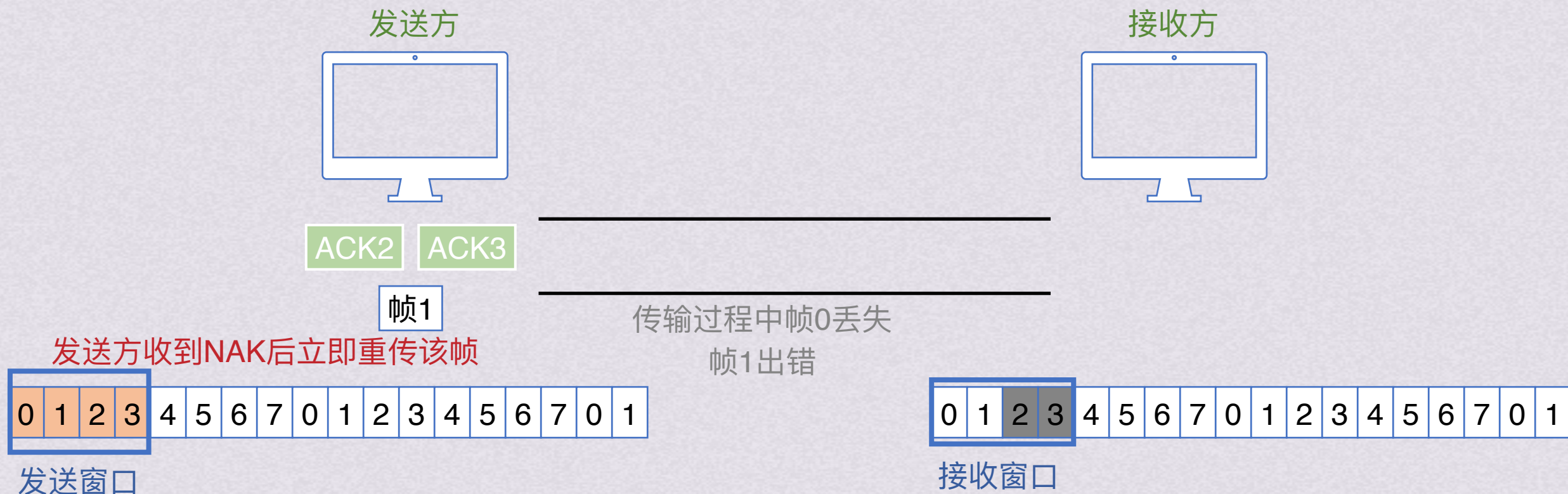
当发送方发送的某个数据帧超时仍未收到确认
将自动重传该帧

当接收方检测到某个数据帧出错
它会向发送方发送一个否定帧NAK
要求发送方立刻重传该帧



连续ARQ协议

选择重传协议(SR)



SR只重传出现差错的数据帧和计时器超时的数据帧:

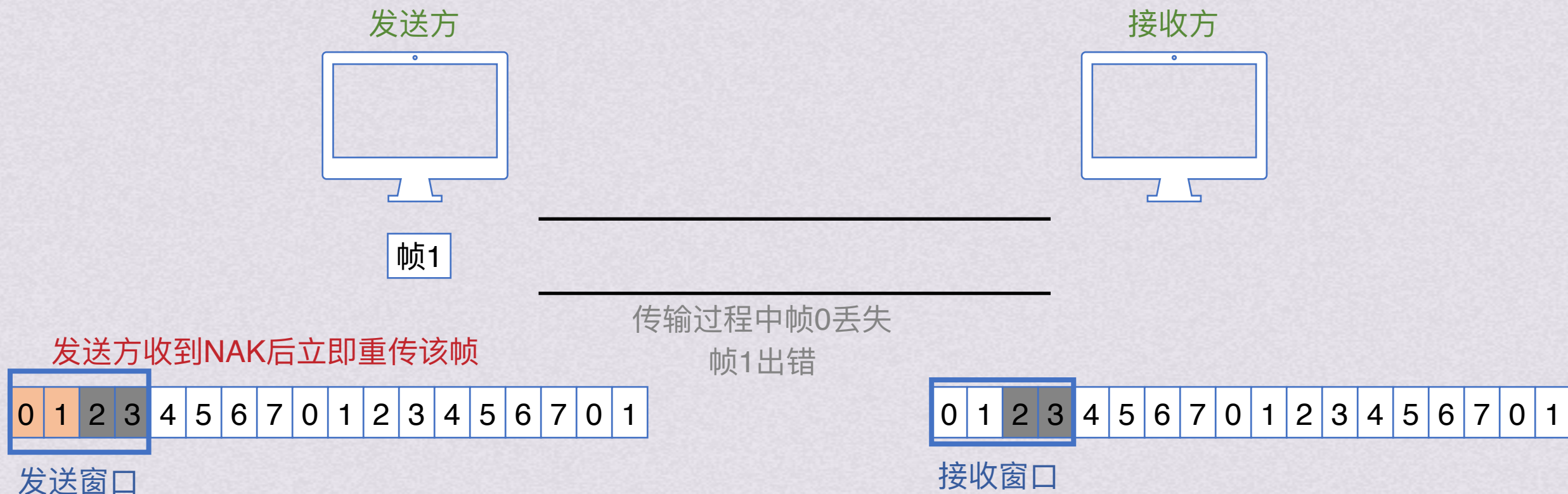
当发送方发送的某个数据帧超时仍未收到确认
将自动重传该帧

当接收方检测到某个数据帧出错
它会向发送方发送一个否定帧NAK
要求发送方立刻重传该帧



连续ARQ协议

选择重传协议(SR)



SR只重传出现差错的数据帧和计时器超时的数据帧：

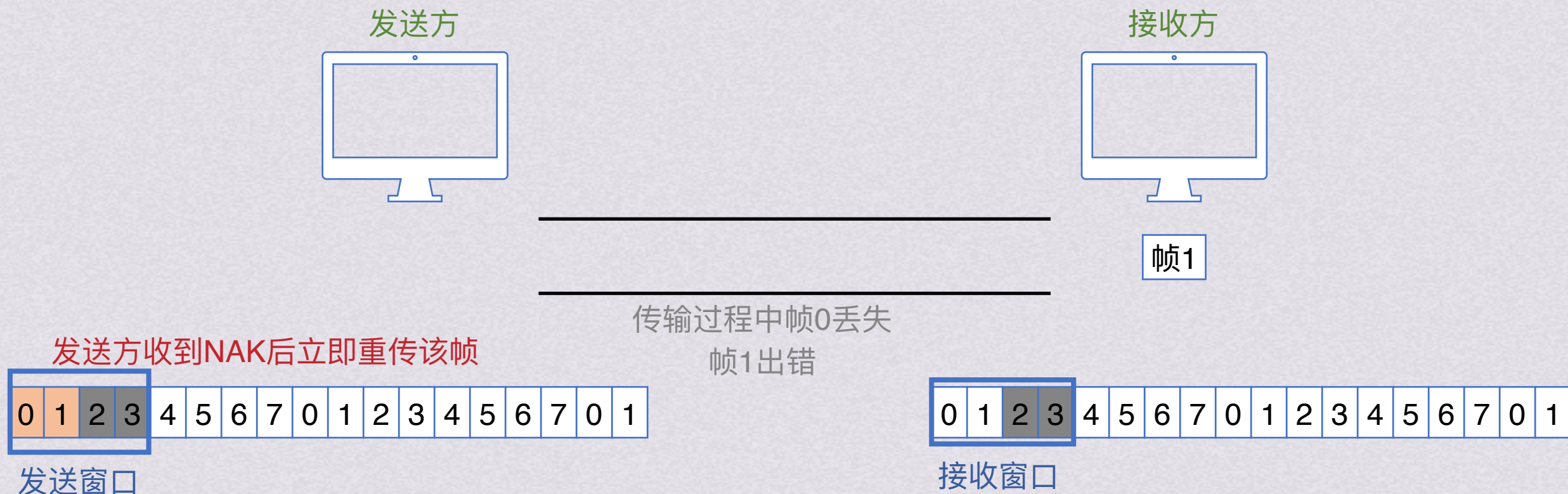
当发送方发送的某个数据帧超时仍未收到确认
将自动重传该帧

当接收方检测到某个数据帧出错
它会向发送方发送一个否定帧NAK
要求发送方立刻重传该帧



连续ARQ协议

选择重传协议(SR)



SR只重传出现差错的数据帧和计时器超时的数据帧：

当发送方发送的某个数据帧超时仍未收到确认
将自动重传该帧

当接收方检测到某个数据帧出错
它会向发送方发送一个否定帧NAK
要求发送方立刻重传该帧



连续ARQ协议

选择重传协议(SR)



SR只重传出现差错的数据帧和计时器超时的数据帧:

当发送方发送的某个数据帧超时仍未收到确认
将自动重传该帧

当接收方检测到某个数据帧出错
它会向发送方发送一个否定帧NAK
要求发送方立刻重传该帧



连续ARQ协议

选择重传协议(SR)



SR只重传出现差错的数据帧和计时器超时的数据帧:

当发送方发送的某个数据帧超时仍未收到确认
将自动重传该帧

当接收方检测到某个数据帧出错
它会向发送方发送一个否定帧NAK
要求发送方立刻重传该帧



连续ARQ协议

选择重传协议(SR)



SR只重传出现差错的数据帧和计时器超时的数据帧:

当发送方发送的某个数据帧超时仍未收到确认
将自动重传该帧

当接收方检测到某个数据帧出错
它会向发送方发送一个否定帧NAK
要求发送方立刻重传该帧



连续ARQ协议

选择重传协议(SR)



SR只重传出现差错的数据帧和计时器超时的数据帧:

当发送方发送的某个数据帧超时仍未收到确认
将自动重传该帧

当接收方检测到某个数据帧出错
它会向发送方发送一个否定帧NAK
要求发送方立刻重传该帧



连续ARQ协议

选择重传协议(SR)



SR只重传出现差错的数据帧和计时器超时的数据帧:

当发送方发送的某个数据帧超时仍未收到确认
将自动重传该帧

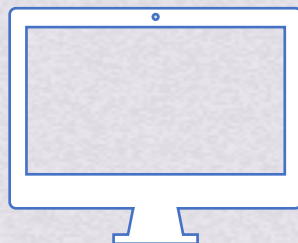
当接收方检测到某个数据帧出错
它会向发送方发送一个否定帧NAK
要求发送方立刻重传该帧



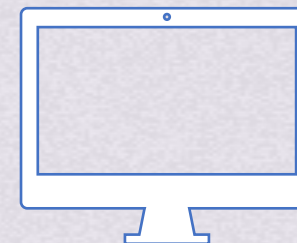
连续ARQ协议

选择重传协议(SR)

发送方



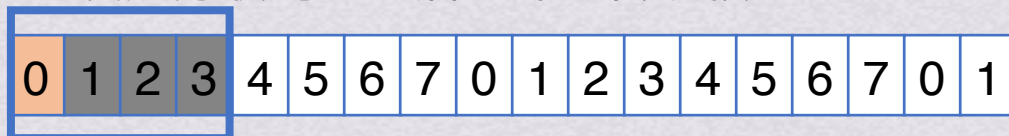
接收方



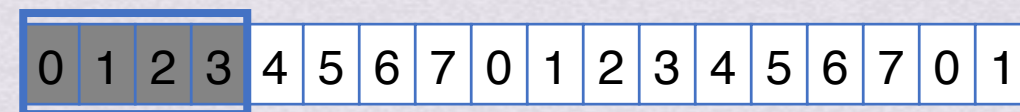
传输过程中帧0丢失
帧1出错

0号帧在超时后会重新发送

发送方收到NAK后立即重传该帧



发送窗口



接收窗口

SR只重传出现差错的数据帧和计时器超时的数据帧：

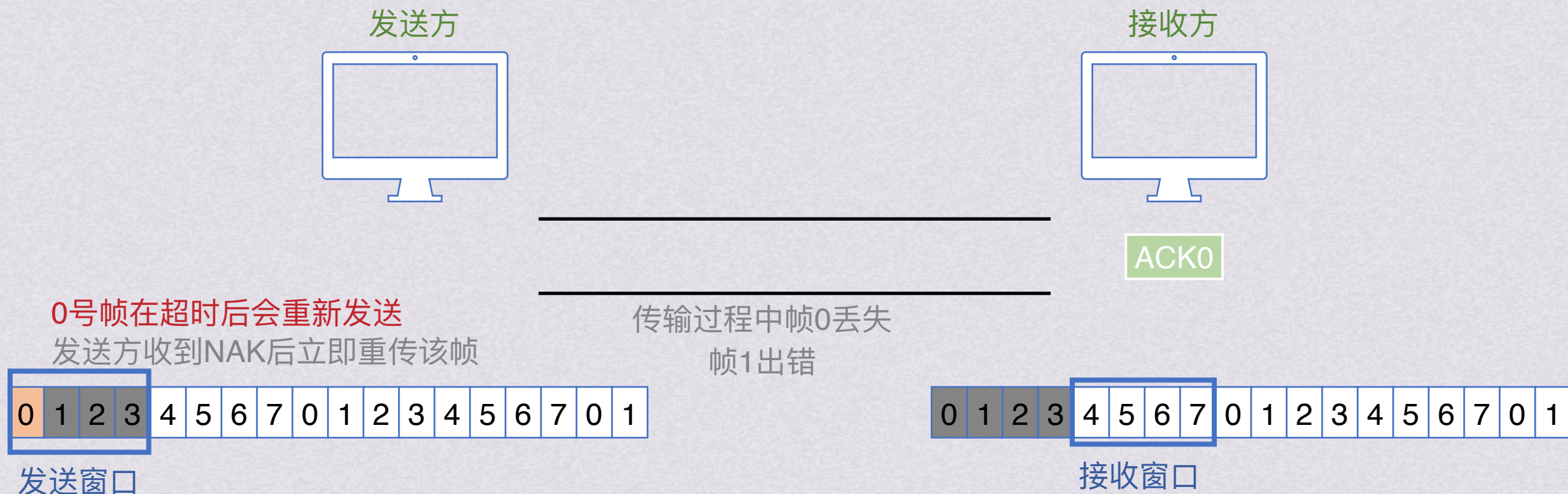
当发送方发送的某个数据帧超时仍未收到确认
将自动重传该帧

当接收方检测到某个数据帧出错
它会向发送方发送一个否定帧NAK
要求发送方立刻重传该帧



连续ARQ协议

选择重传协议(SR)



SR只重传出现差错的数据帧和计时器超时的数据帧:

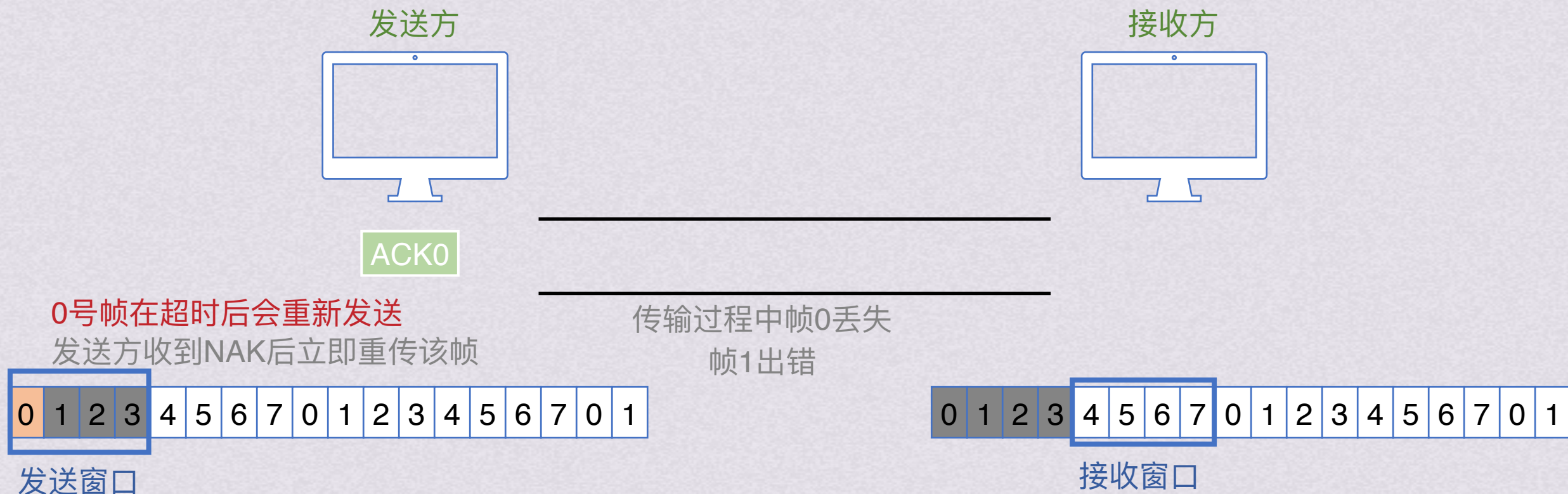
当发送方发送的某个数据帧超时仍未收到确认
将自动重传该帧

当接收方检测到某个数据帧出错
它会向发送方发送一个否定帧NAK
要求发送方立刻重传该帧



连续ARQ协议

选择重传协议(SR)



SR只重传出现差错的数据帧和计时器超时的数据帧:

当发送方发送的某个数据帧超时仍未收到确认
将自动重传该帧

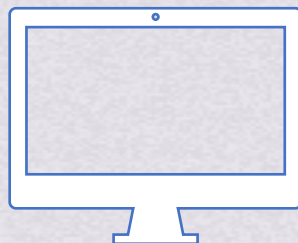
当接收方检测到某个数据帧出错
它会向发送方发送一个否定帧NAK
要求发送方立刻重传该帧



连续ARQ协议

选择重传协议(SR)

发送方



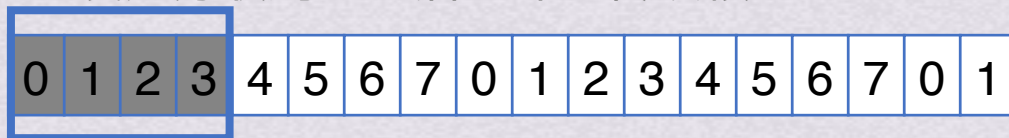
接收方



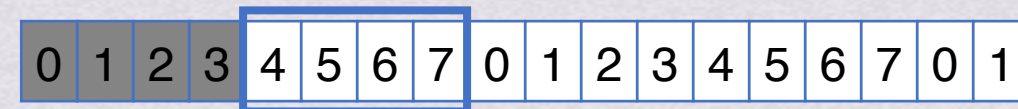
传输过程中帧0丢失
帧1出错

0号帧在超时后会重新发送

发送方收到NAK后立即重传该帧



发送窗口



接收窗口

SR只重传出现差错的数据帧和计时器超时的数据帧：

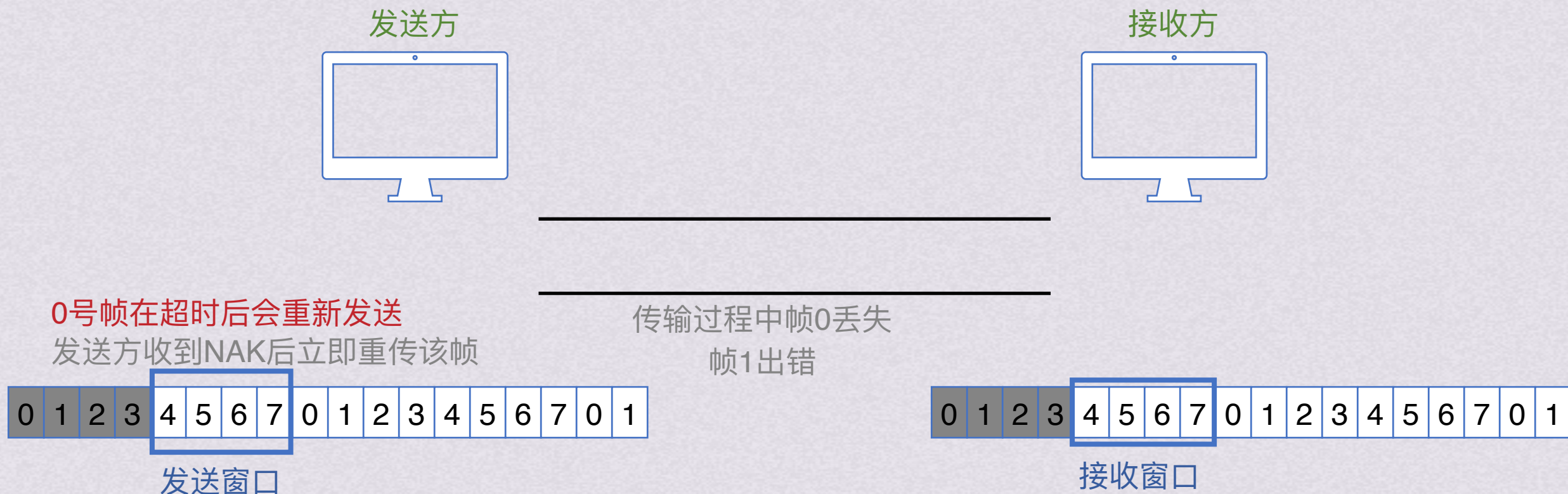
当发送方发送的某个数据帧超时仍未收到确认
将自动重传该帧

当接收方检测到某个数据帧出错
它会向发送方发送一个否定帧NAK
要求发送方立刻重传该帧



连续ARQ协议

选择重传协议(SR)



SR只重传出现差错的数据帧和计时器超时的数据帧:

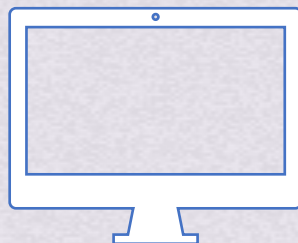
当发送方发送的某个数据帧超时仍未收到确认
将自动重传该帧

当接收方检测到某个数据帧出错
它会向发送方发送一个否定帧NAK
要求发送方立刻重传该帧

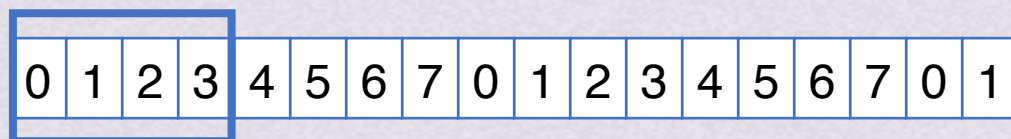
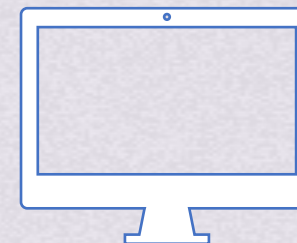


选择重传协议(SR)

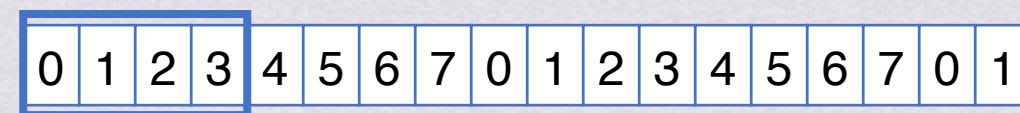
发送方



接收方



发送窗口



接收窗口

因为要让发送方只重传那些丢失/出错的帧
所以不再使用累计确认，改为逐帧确认

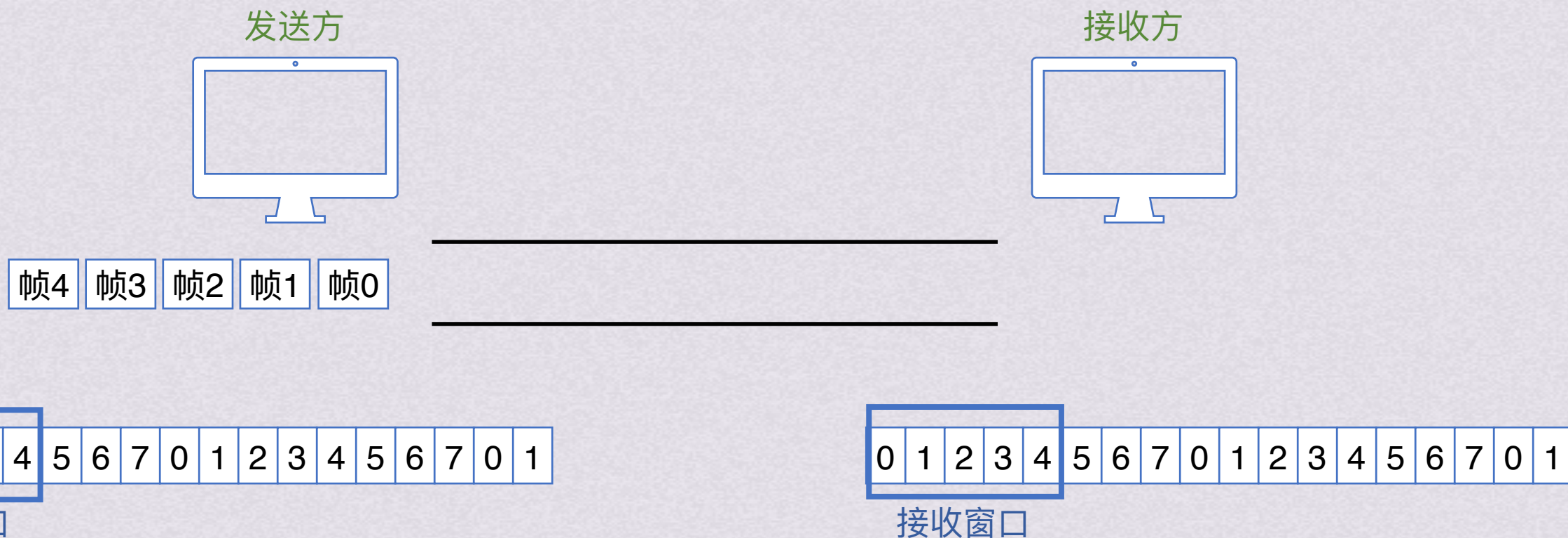
发送窗口和接收窗口范围都变为了 $1 \leq W \leq 2^{n-1}$
假设用3比特给帧编号，则发送窗口和接收窗口都最大为4

按照作死惯例，再来试一下窗口改成5会怎样



连续ARQ协议

选择重传协议(SR)



因为要让发送方只重传那些丢失/出错的帧
所以不再使用累计确认，改为逐帧确认

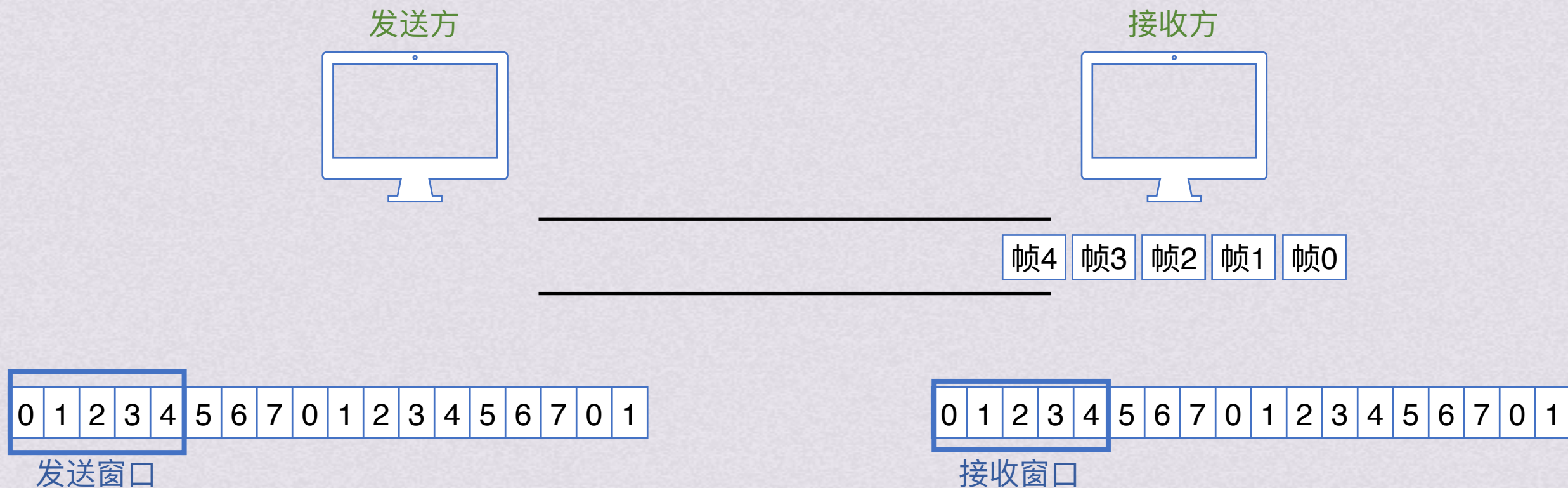
发送窗口和接收窗口范围都变为了 $1 \leq W \leq 2^{n-1}$
假设用3比特给帧编号，则发送窗口和接收窗口都最大为4

按照作死惯例，再来试一下窗口改成5会怎样



连续ARQ协议

选择重传协议(SR)



因为要让发送方只重传那些丢失/出错的帧
所以不再使用累计确认，改为逐帧确认

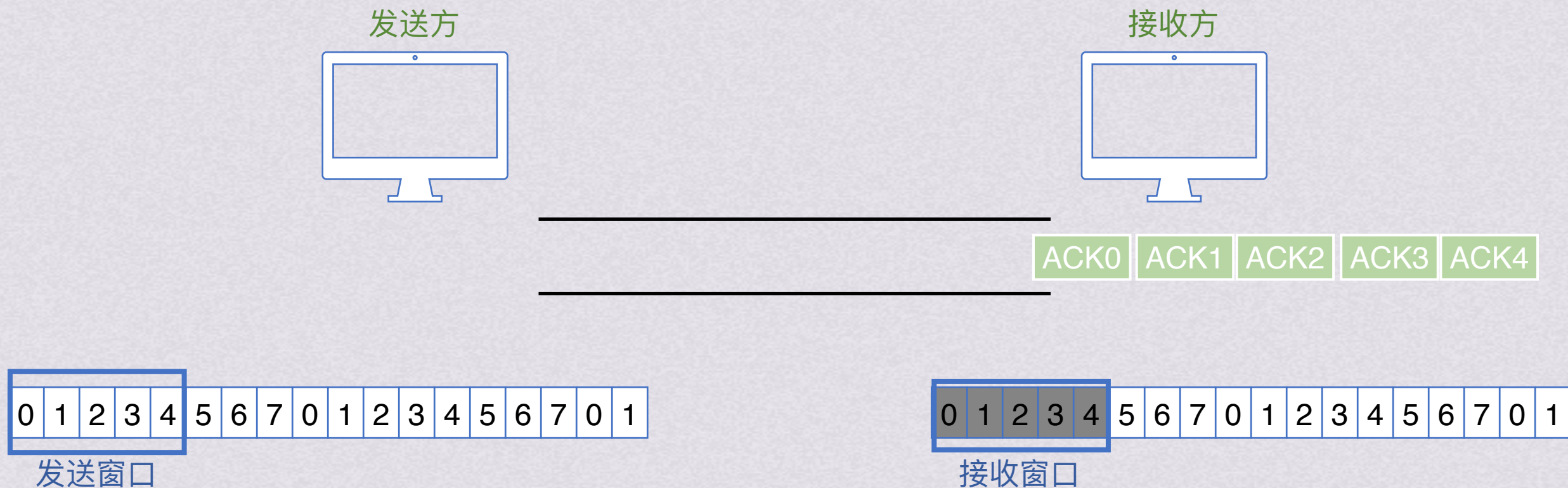
发送窗口和接收窗口范围都变为了 $1 \leq W \leq 2^{n-1}$
假设用3比特给帧编号，则发送窗口和接收窗口都最大为4

按照作死惯例，再来试一下窗口改成5会怎样



连续ARQ协议

选择重传协议(SR)



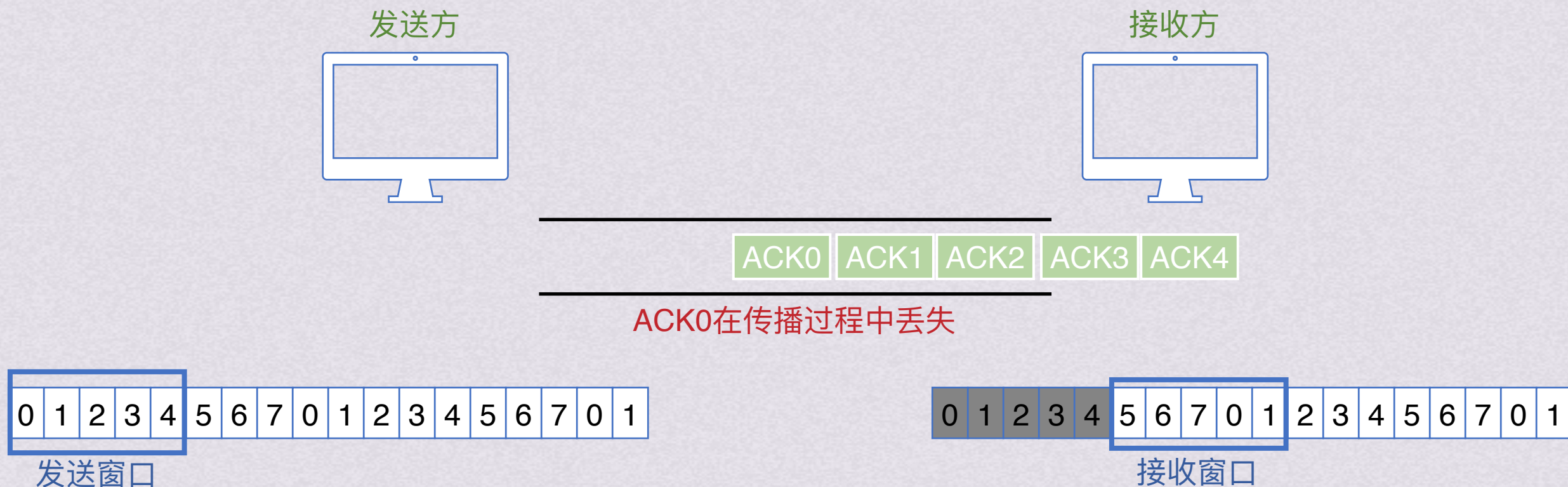
因为要让发送方只重传那些丢失/出错的帧
所以不再使用累计确认，改为逐帧确认

发送窗口和接收窗口范围都变为了 $1 \leq W \leq 2^{n-1}$
假设用3比特给帧编号，则发送窗口和接收窗口都最大为4

按照作死惯例，再来试一下窗口改成5会怎样



选择重传协议(SR)



因为要让发送方只重传那些丢失/出错的帧
所以不再使用累计确认，改为逐帧确认

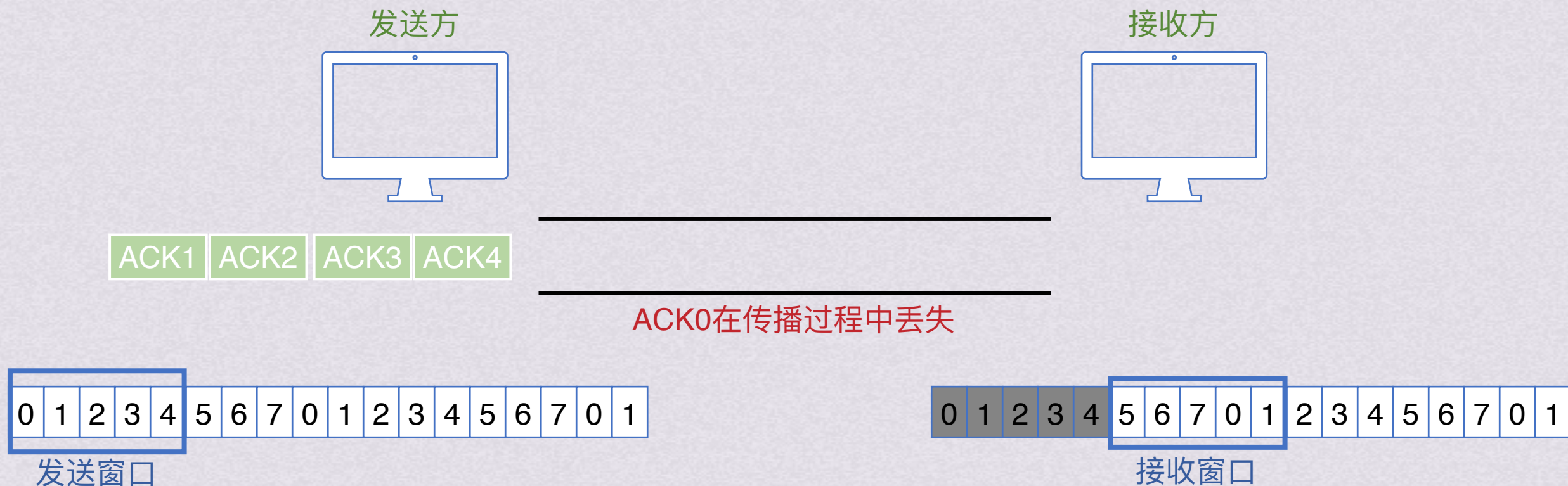
发送窗口和接收窗口范围都变为了 $1 \leq W \leq 2^{n-1}$
假设用3比特给帧编号，则发送窗口和接收窗口都最大为4

按照作死惯例，再来试一下窗口改成5会怎样



连续ARQ协议

选择重传协议(SR)



因为要让发送方只重传那些丢失/出错的帧
所以不再使用累计确认，改为逐帧确认

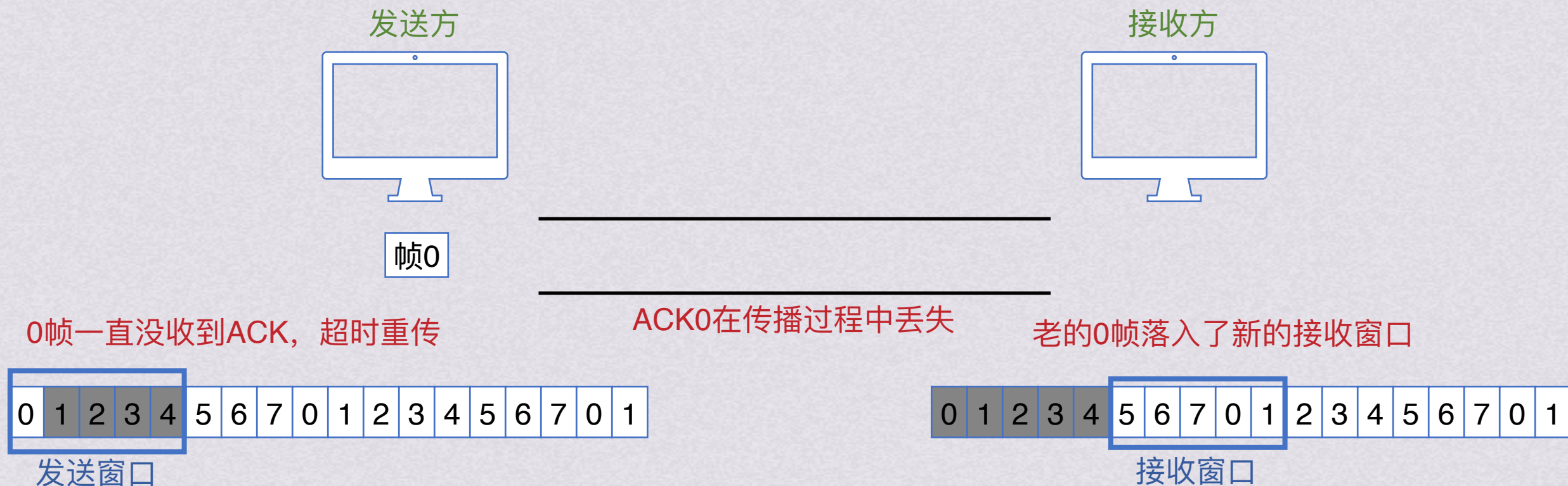
发送窗口和接收窗口范围都变为了 $1 \leq W \leq 2^{n-1}$
假设用3比特给帧编号，则发送窗口和接收窗口都最大为4

按照作死惯例，再来试一下窗口改成5会怎样



连续ARQ协议

选择重传协议(SR)



因为要让发送方只重传那些丢失/出错的帧
所以不再使用累计确认, 改为逐帧确认

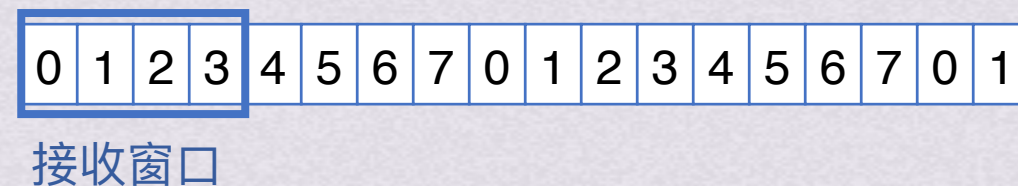
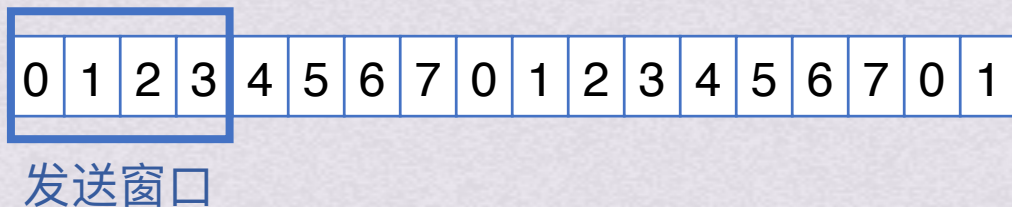
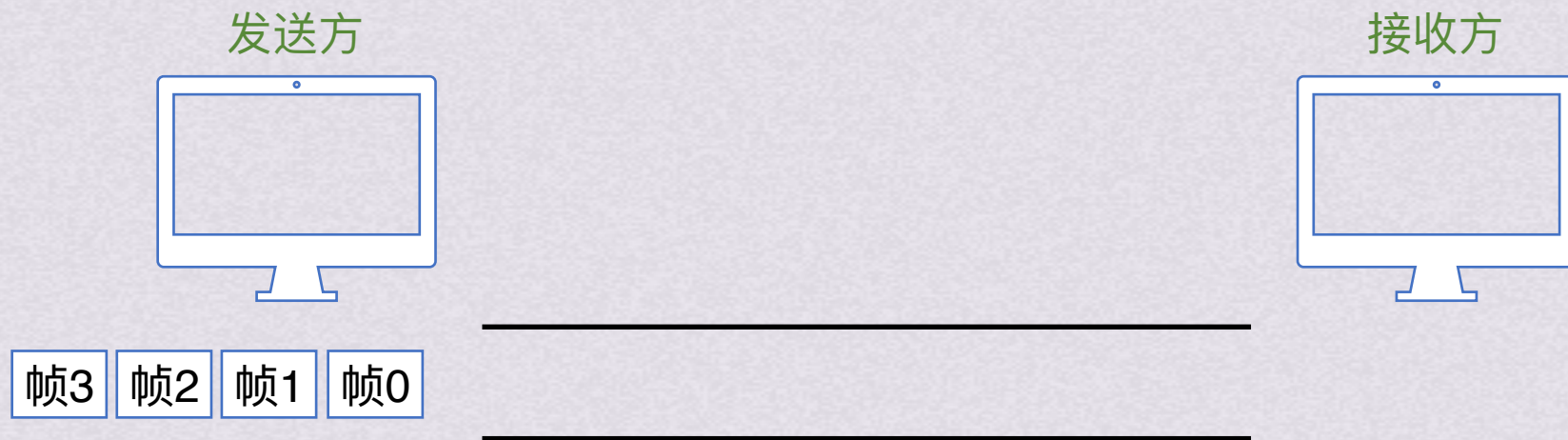
发送窗口和接收窗口范围都变为了 $1 \leq W \leq 2^{n-1}$
假设用3比特给帧编号, 则发送窗口和接收窗口都最大为4

按照作死惯例, 再来试一下窗口改成5会怎样



连续ARQ协议

选择重传协议(SR)

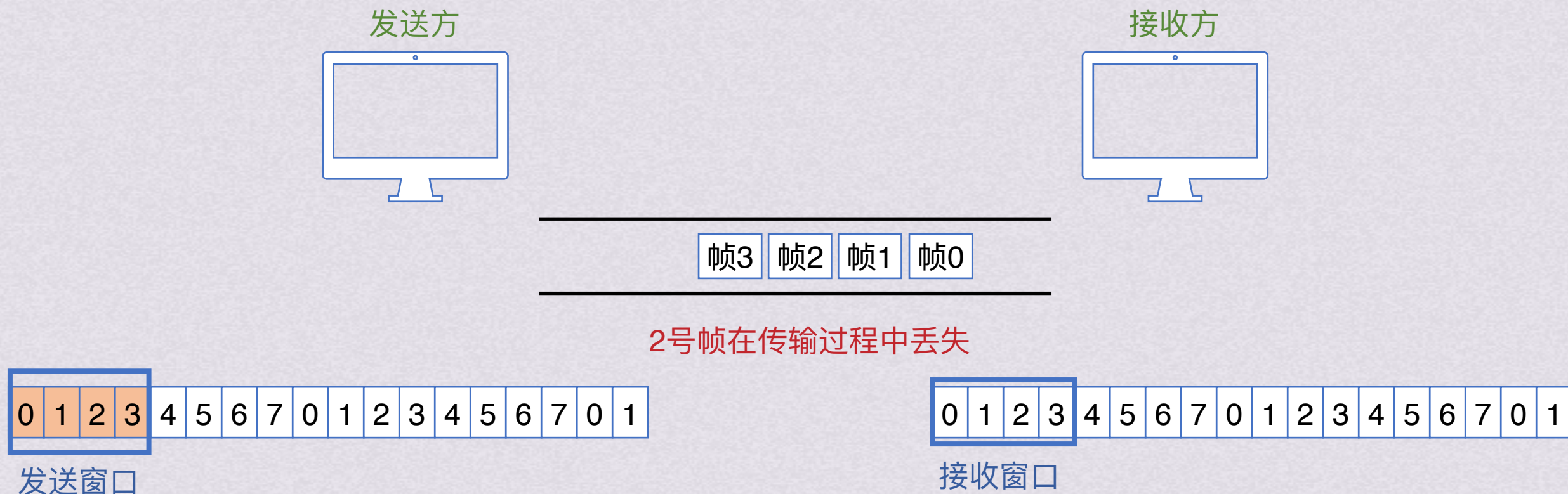


发送窗口和接收窗口范围都变为了 $1 \leq W \leq 2^{n-1}$
假设用3比特给帧编号, 则发送窗口和接收窗口都最大为4



连续ARQ协议

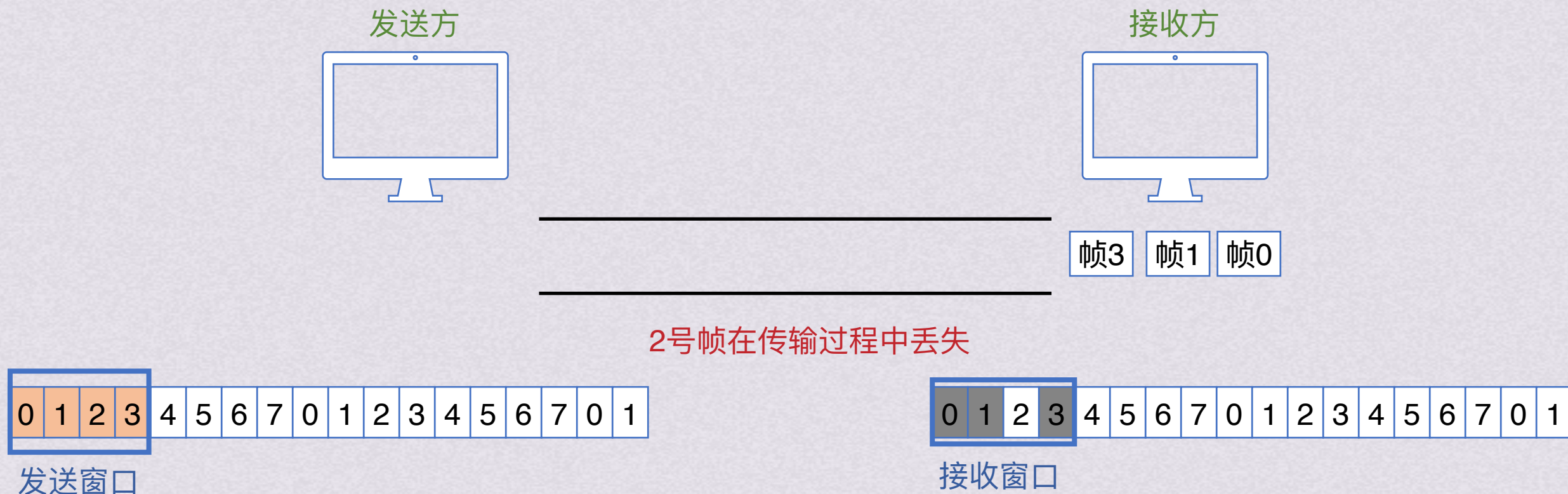
选择重传协议(SR)



发送窗口和接收窗口范围都变为了 $1 \leq W \leq 2^{n-1}$
假设用3比特给帧编号，则发送窗口和接收窗口都最大为4



选择重传协议(SR)



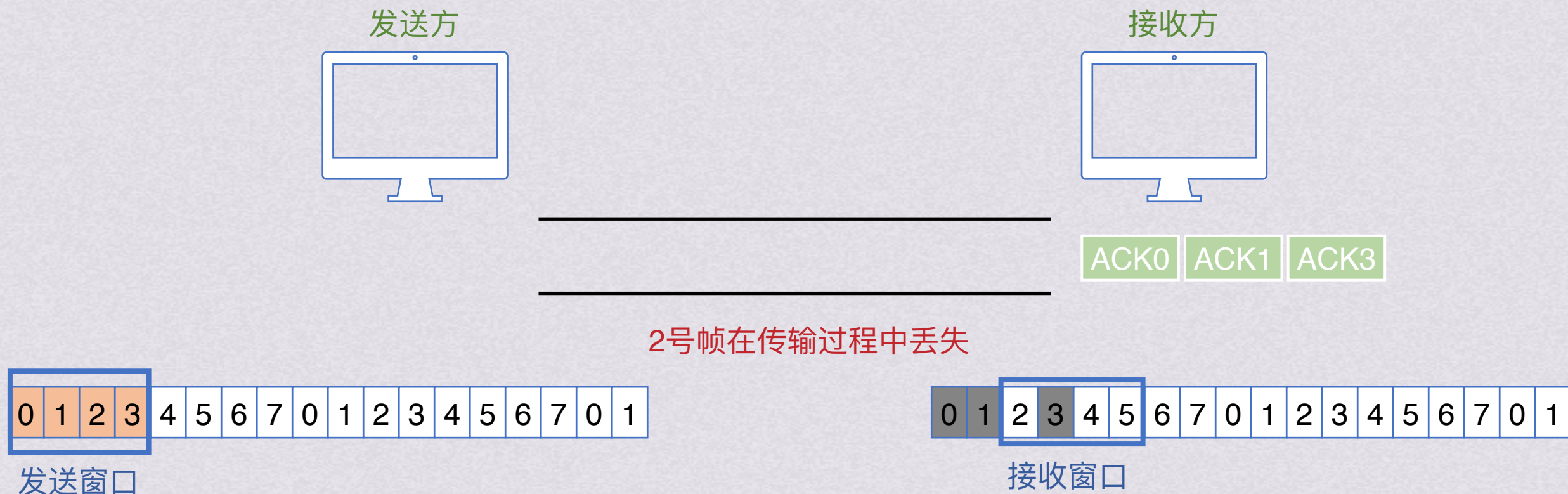
接收方会发送确认给发送方，并且根据有序接收的帧来移动接收窗口
这里3并不是按序接收的，所以接收窗口会移动到2开始的地方，3号帧暂时缓存等待前面的帧一起上交

发送窗口和接收窗口范围都变为了 $1 \leq W \leq 2^{n-1}$
假设用3比特给帧编号，则发送窗口和接收窗口都最大为4



连续ARQ协议

选择重传协议(SR)



接收方会发送确认给发送方，并且根据有序接收的帧来移动接收窗口
这里3并不是按序接收的，所以接收窗口会移动到2开始的地方，3号帧暂时缓存等待前面的帧一起上交

发送窗口和接收窗口范围都变为了 $1 \leq W \leq 2^{n-1}$
假设用3比特给帧编号，则发送窗口和接收窗口都最大为4

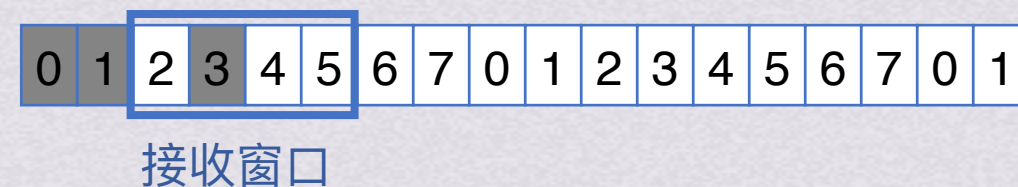
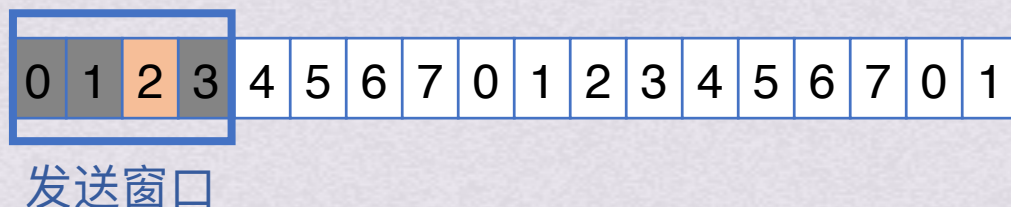


连续ARQ协议

选择重传协议(SR)



2号帧在传输过程中丢失

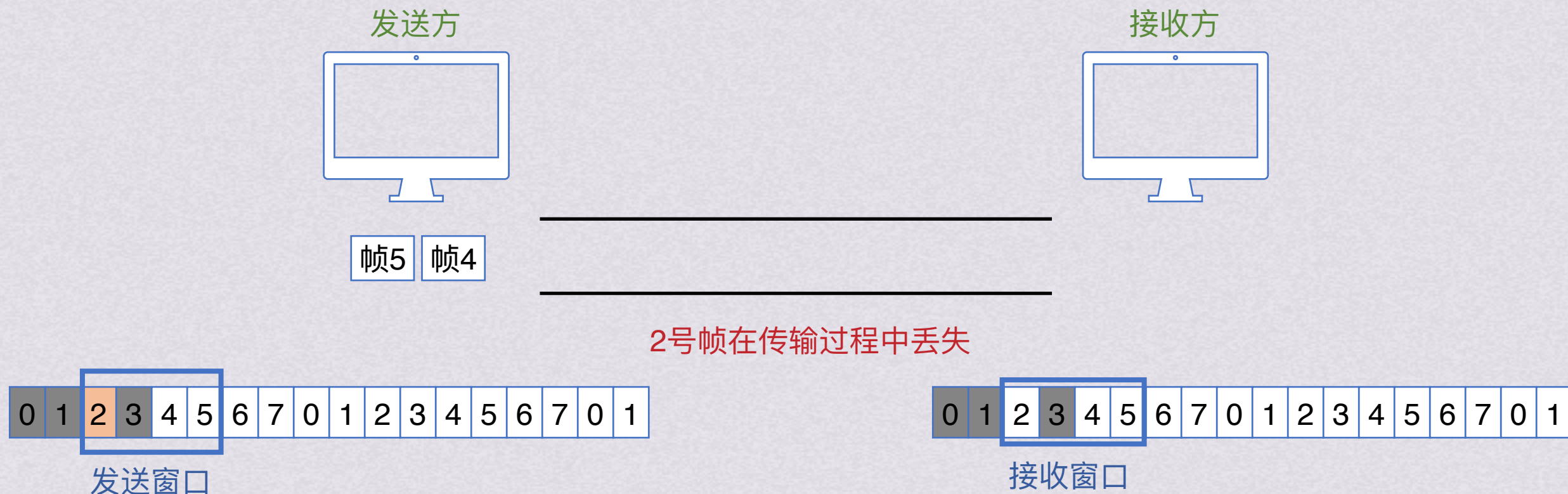


接收方会发送确认给发送方，并且根据有序接收的帧来移动接收窗口
这里3并不是按序接收的，所以接收窗口会移动到2开始的地方，3号帧暂时缓存等待前面的帧一起上交

发送窗口和接收窗口范围都变为了 $1 \leq W \leq 2^{n-1}$
假设用3比特给帧编号，则发送窗口和接收窗口都最大为4



选择重传协议(SR)



假设此刻2帧没有超时，那么发送方会将4,5号帧发送出去，等待2号帧超时了再重新传送2号帧

接收方会发送确认给发送方，并且根据有序接收的帧来移动接收窗口

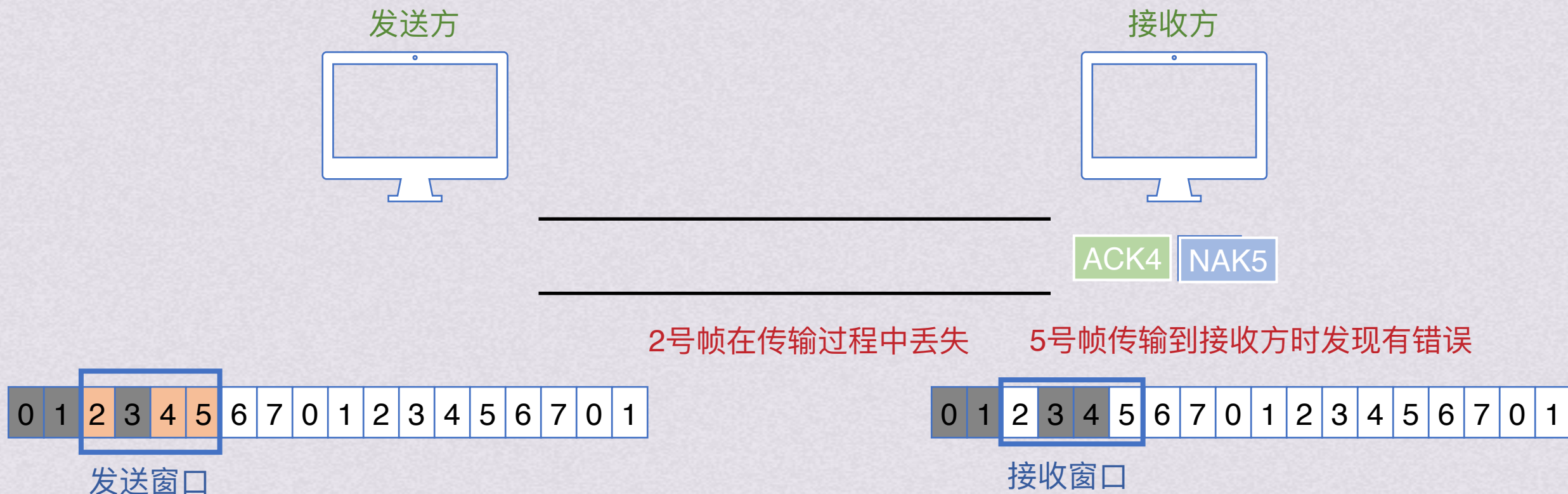
这里3并不是按序接收的，所以接收窗口会移动到2开始的地方，3号帧暂时缓存等待前面的帧一起上交
发送窗口和接收窗口范围都变为了 $1 \leq W \leq 2^{n-1}$

假设用3比特给帧编号，则发送窗口和接收窗口都最大为4



连续ARQ协议

选择重传协议(SR)



假设此刻2帧没有超时，那么发送方会将4,5号帧发送出去，等待2号帧超时了再重新传送2号帧

接收方会发送确认给发送方，并且根据有序接收的帧来移动接收窗口

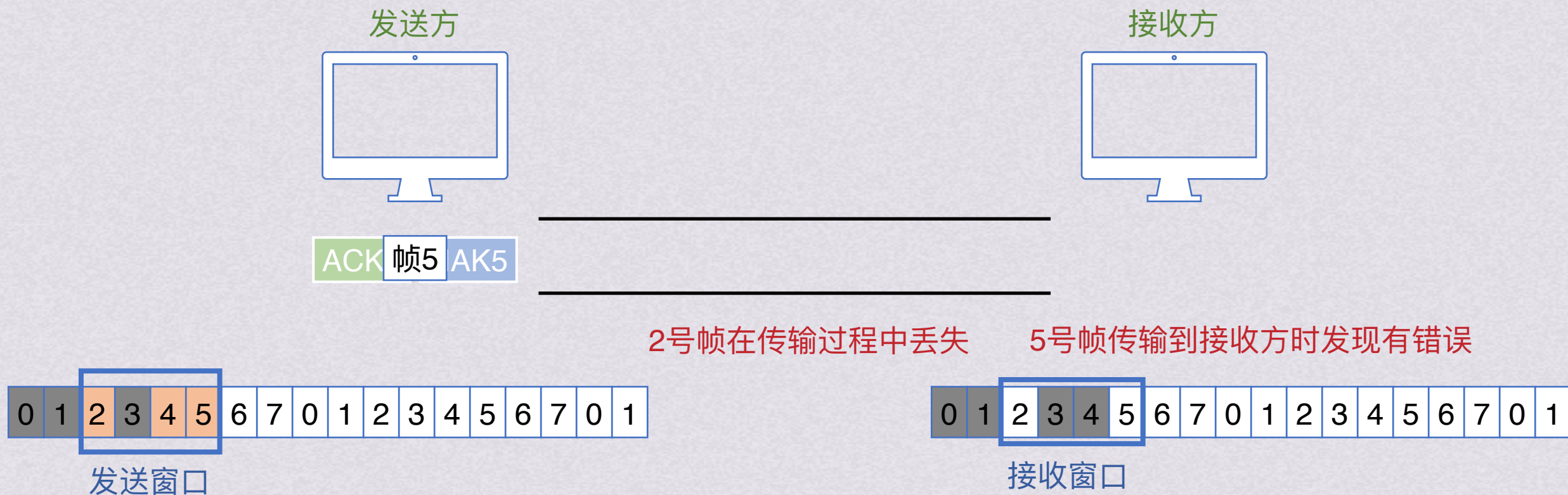
这里3并不是按序接收的，所以接收窗口会移动到2开始的地方，3号帧暂时缓存等待前面的帧一起上交
发送窗口和接收窗口范围都变为了 $1 \leq W \leq 2^{n-1}$

假设用3比特给帧编号，则发送窗口和接收窗口都最大为4



连续ARQ协议

选择重传协议(SR)



假设此刻2帧没有超时，那么发送方会将4,5号帧发送出去，等待2号帧超时了再重新传送2号帧

接收方会发送确认给发送方，并且根据有序接收的帧来移动接收窗口

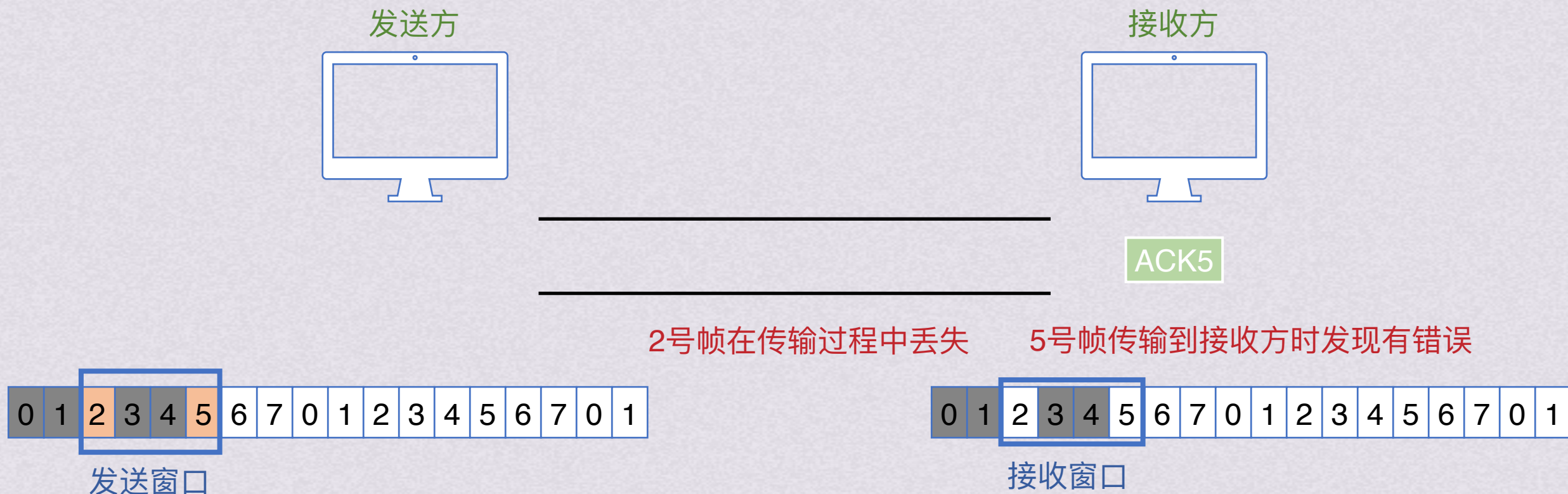
这里3并不是按序接收的，所以接收窗口会移动到2开始的地方，3号帧暂时缓存等待前面的帧一起上交
发送窗口和接收窗口范围都变为了 $1 \leq W \leq 2^{n-1}$

假设用3比特给帧编号，则发送窗口和接收窗口都最大为4



连续ARQ协议

选择重传协议(SR)



假设此刻2帧没有超时，那么发送方会将4,5号帧发送出去，等待2号帧超时了再重新传送2号帧

接收方会发送确认给发送方，并且根据有序接收的帧来移动接收窗口

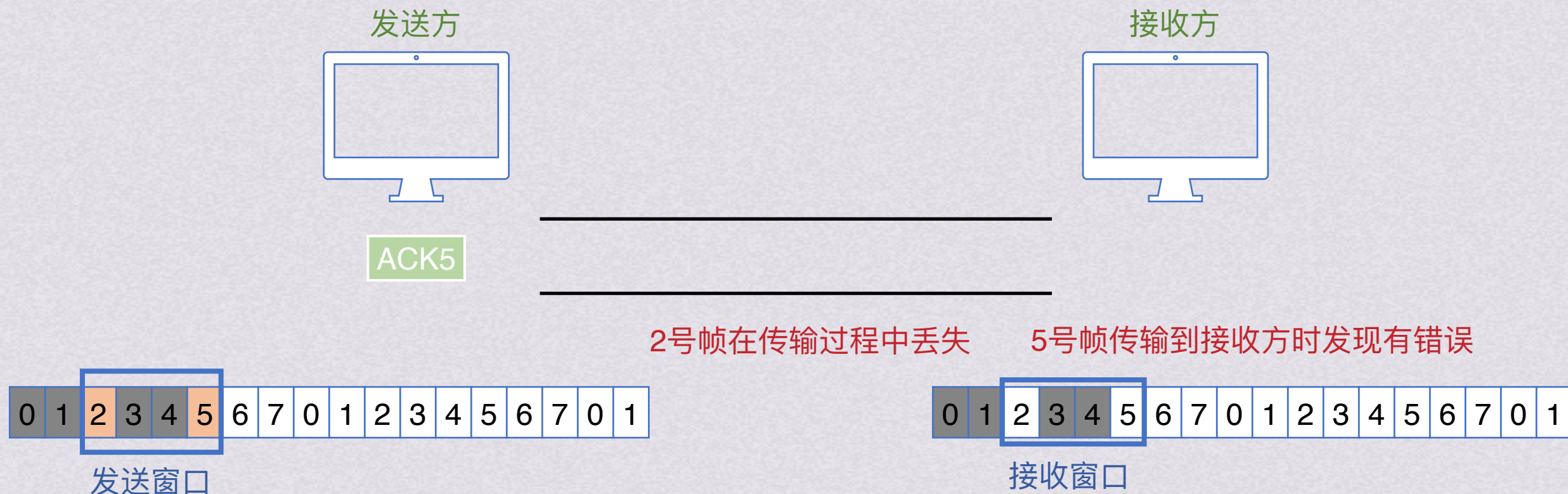
这里3并不是按序接收的，所以接收窗口会移动到2开始的地方，3号帧暂时缓存等待前面的帧一起上交

发送窗口和接收窗口范围都变为了 $1 \leq W \leq 2^{n-1}$

假设用3比特给帧编号，则发送窗口和接收窗口都最大为4



选择重传协议(SR)



假设此刻2帧没有超时，那么发送方会将4,5号帧发送出去，等待2号帧超时了再重新传送2号帧

接收方会发送确认给发送方，并且根据有序接收的帧来移动接收窗口

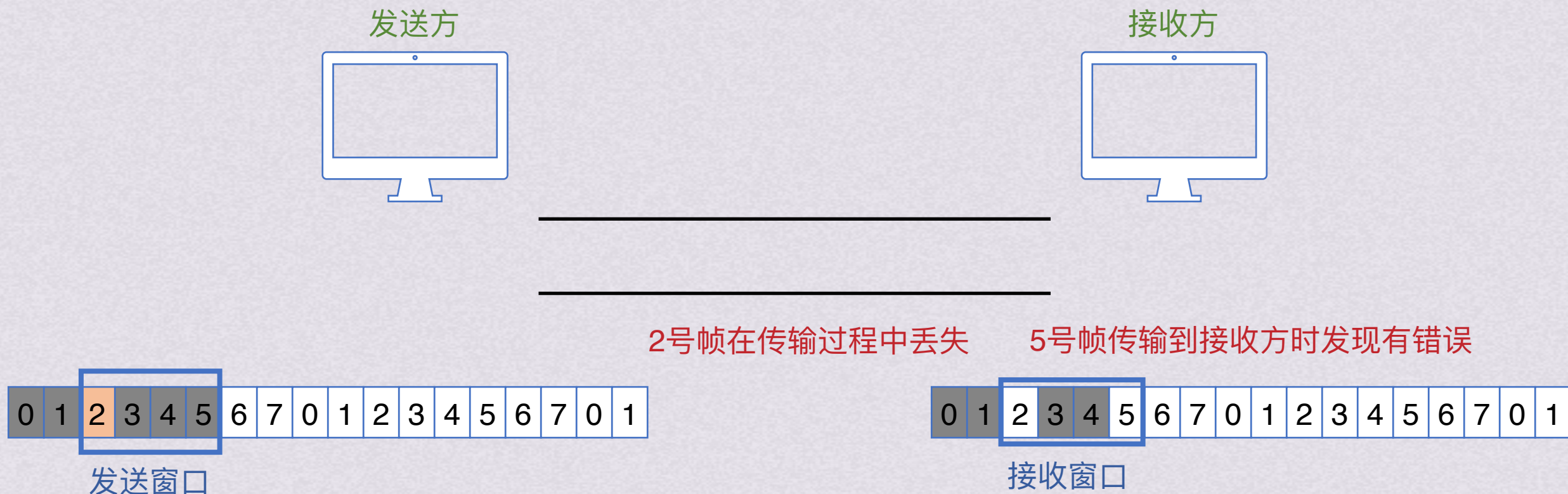
这里3并不是按序接收的，所以接收窗口会移动到2开始的地方，3号帧暂时缓存等待前面的帧一起上交
发送窗口和接收窗口范围都变为了 $1 \leq W \leq 2^{n-1}$

假设用3比特给帧编号，则发送窗口和接收窗口都最大为4



连续ARQ协议

选择重传协议(SR)



假设此刻2帧没有超时，那么发送方会将4,5号帧发送出去，等待2号帧超时了再重新传送2号帧

接收方会发送确认给发送方，并且根据有序接收的帧来移动接收窗口

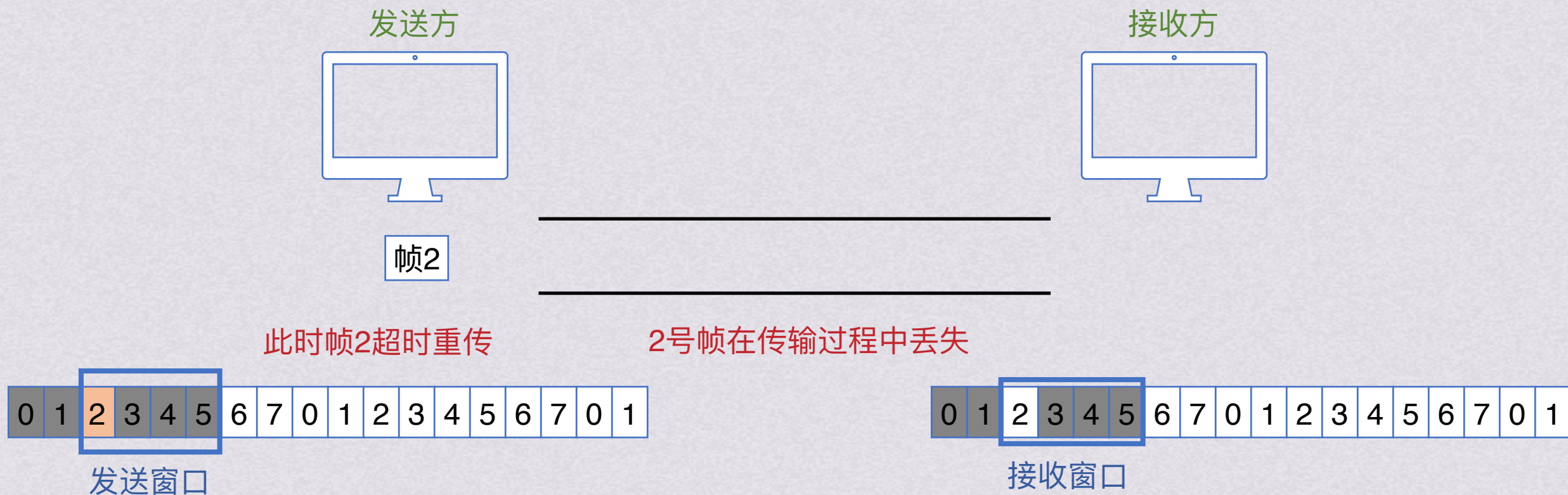
这里3并不是按序接收的，所以接收窗口会移动到2开始的地方，3号帧暂时缓存等待前面的帧一起上交
发送窗口和接收窗口范围都变为了 $1 \leq W \leq 2^{n-1}$

假设用3比特给帧编号，则发送窗口和接收窗口都最大为4



连续ARQ协议

选择重传协议(SR)



假设此刻2帧没有超时，那么发送方会将4,5号帧发送出去，等待2号帧超时了再重新传送2号帧

接收方会发送确认给发送方，并且根据有序接收的帧来移动接收窗口

这里3并不是按序接收的，所以接收窗口会移动到2开始的地方，3号帧暂时缓存等待前面的帧一起上交

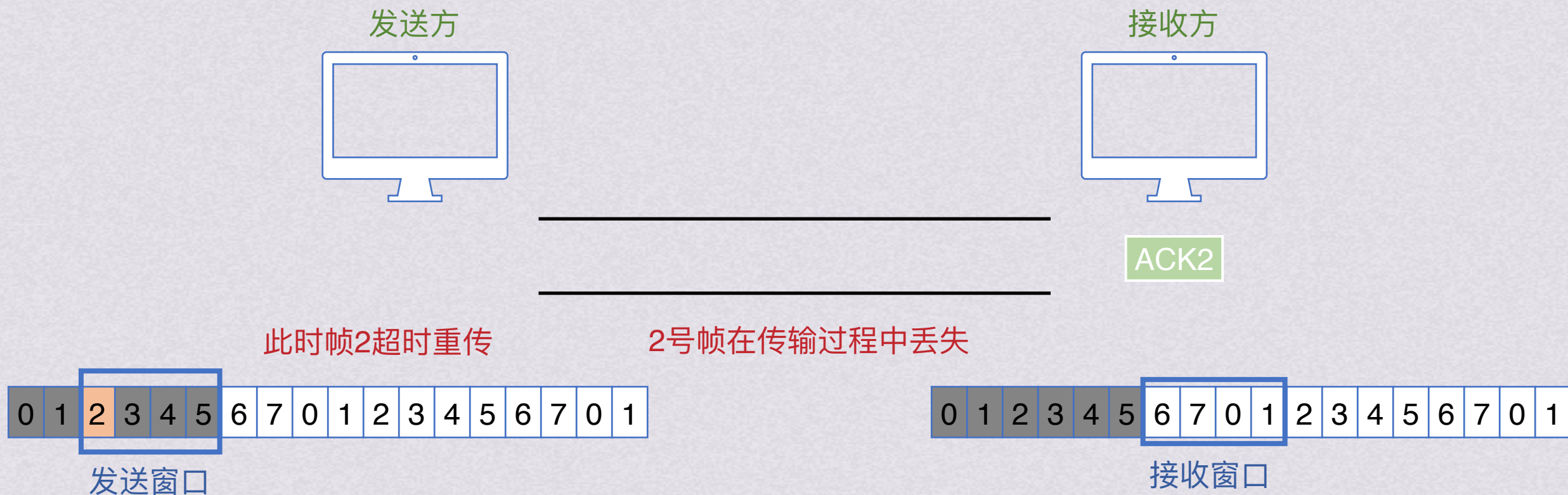
发送窗口和接收窗口范围都变为了 $1 \leq W \leq 2^{n-1}$

假设用3比特给帧编号，则发送窗口和接收窗口都最大为4



连续ARQ协议

选择重传协议(SR)



假设此刻2帧没有超时，那么发送方会将4,5号帧发送出去，等待2号帧超时了再重新传送2号帧

接收方会发送确认给发送方，并且根据有序接收的帧来移动接收窗口

这里3并不是按序接收的，所以接收窗口会移动到2开始的地方，3号帧暂时缓存等待前面的帧一起上交

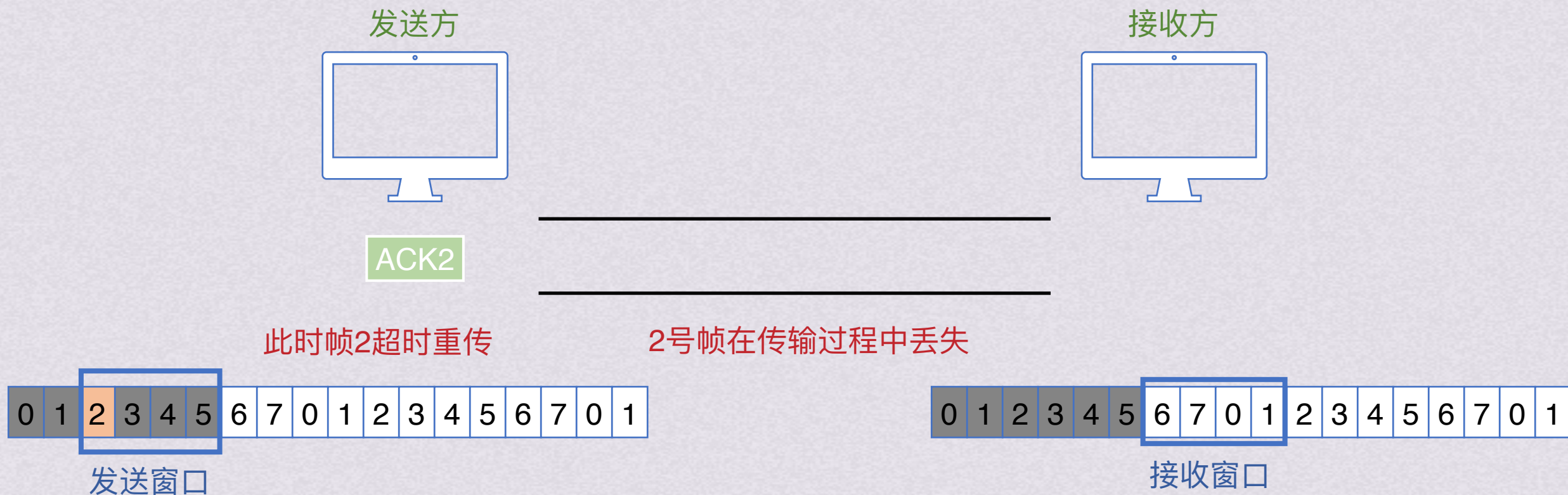
发送窗口和接收窗口范围都变为了 $1 \leq W \leq 2^{n-1}$

假设用3比特给帧编号，则发送窗口和接收窗口都最大为4



连续ARQ协议

选择重传协议(SR)



假设此刻2帧没有超时，那么发送方会将4,5号帧发送出去，等待2号帧超时了再重新传送2号帧

接收方会发送确认给发送方，并且根据有序接收的帧来移动接收窗口

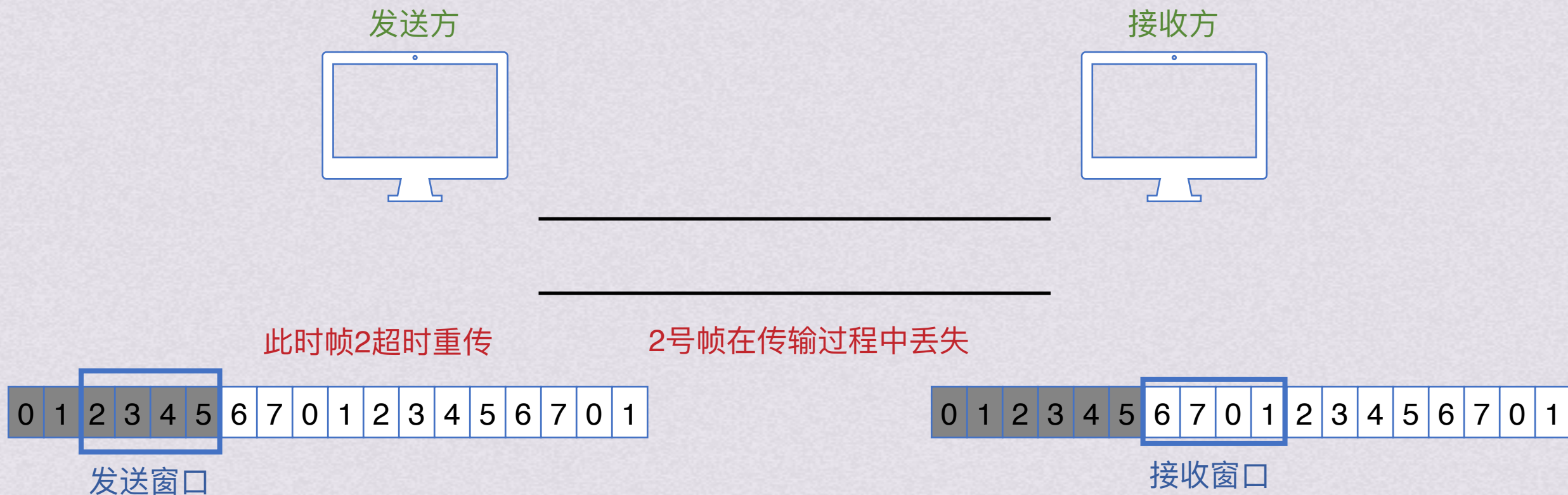
这里3并不是按序接收的，所以接收窗口会移动到2开始的地方，3号帧暂时缓存等待前面的帧一起上交
发送窗口和接收窗口范围都变为了 $1 \leq W \leq 2^{n-1}$

假设用3比特给帧编号，则发送窗口和接收窗口都最大为4



连续ARQ协议

选择重传协议(SR)



假设此刻2帧没有超时，那么发送方会将4,5号帧发送出去，等待2号帧超时了再重新传送2号帧

接收方会发送确认给发送方，并且根据有序接收的帧来移动接收窗口

这里3并不是按序接收的，所以接收窗口会移动到2开始的地方，3号帧暂时缓存等待前面的帧一起上交

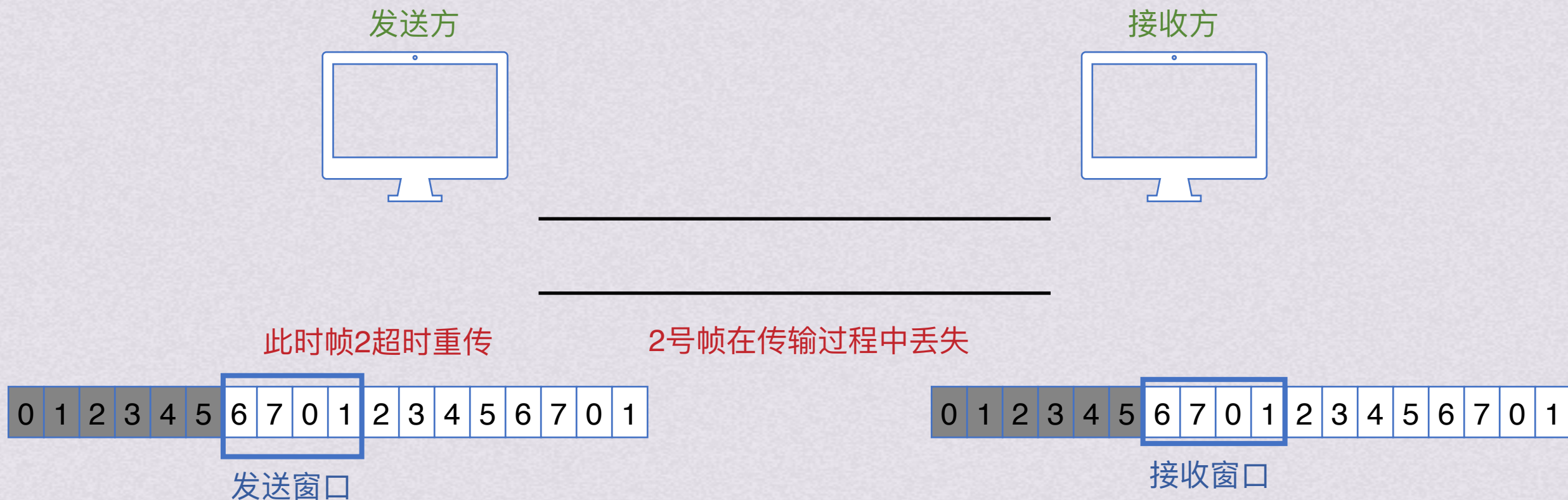
发送窗口和接收窗口范围都变为了 $1 \leq W \leq 2^{n-1}$

假设用3比特给帧编号，则发送窗口和接收窗口都最大为4



连续ARQ协议

选择重传协议(SR)



假设此刻2帧没有超时，那么发送方会将4,5号帧发送出去，等待2号帧超时了再重新传送2号帧

接收方会发送确认给发送方，并且根据有序接收的帧来移动接收窗口

这里3并不是按序接收的，所以接收窗口会移动到2开始的地方，3号帧暂时缓存等待前面的帧一起上交

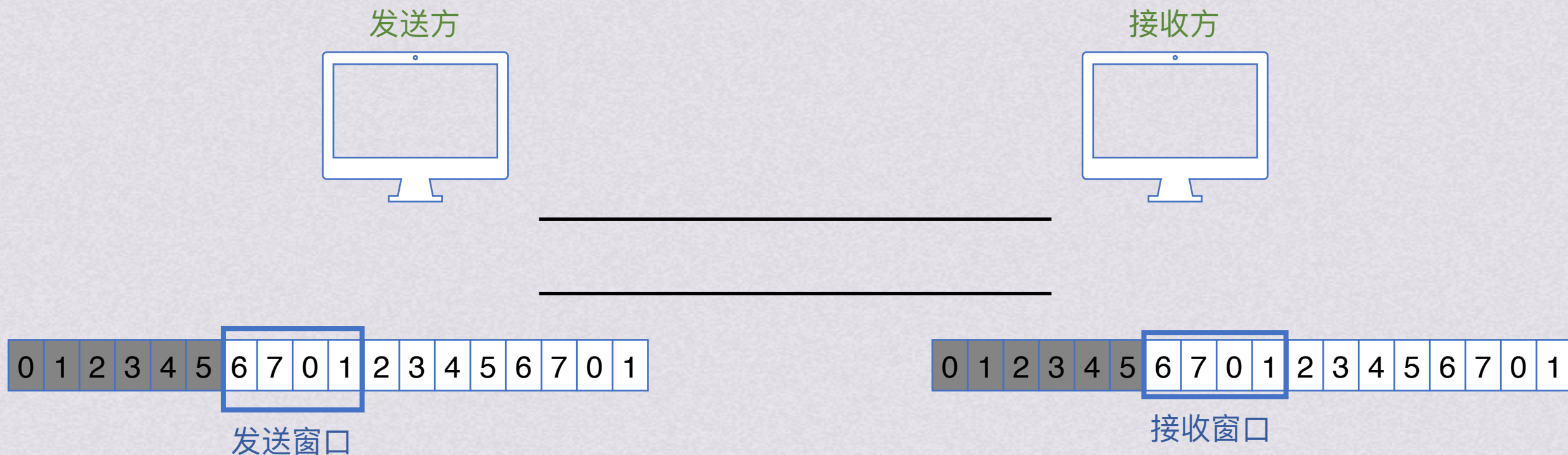
发送窗口和接收窗口范围都变为了 $1 \leq W \leq 2^{n-1}$

假设用3比特给帧编号，则发送窗口和接收窗口都最大为4

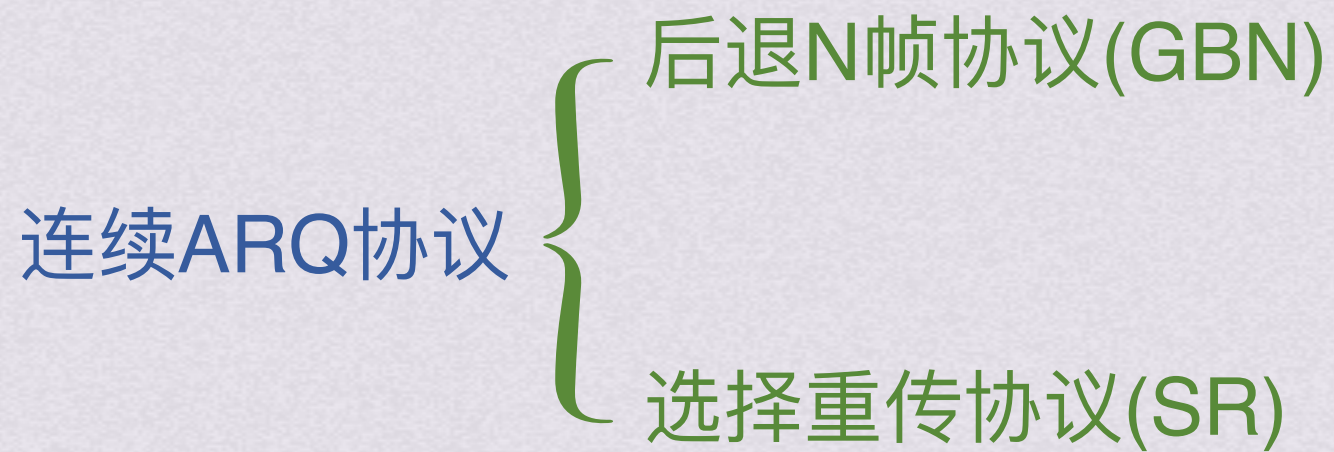


连续ARQ协议

选择重传协议(SR)



- 停止等待协议：发送窗口 $W_t=1$ ，接收窗口 $W_r=1$ 。
- 后退N帧协议：发送窗口 $W_t>1$ ，接收窗口 $W_r=1$ 。
- 选择重传协议：发送窗口 $W_t>1$ ，接收窗口 $W_r>1$ 。

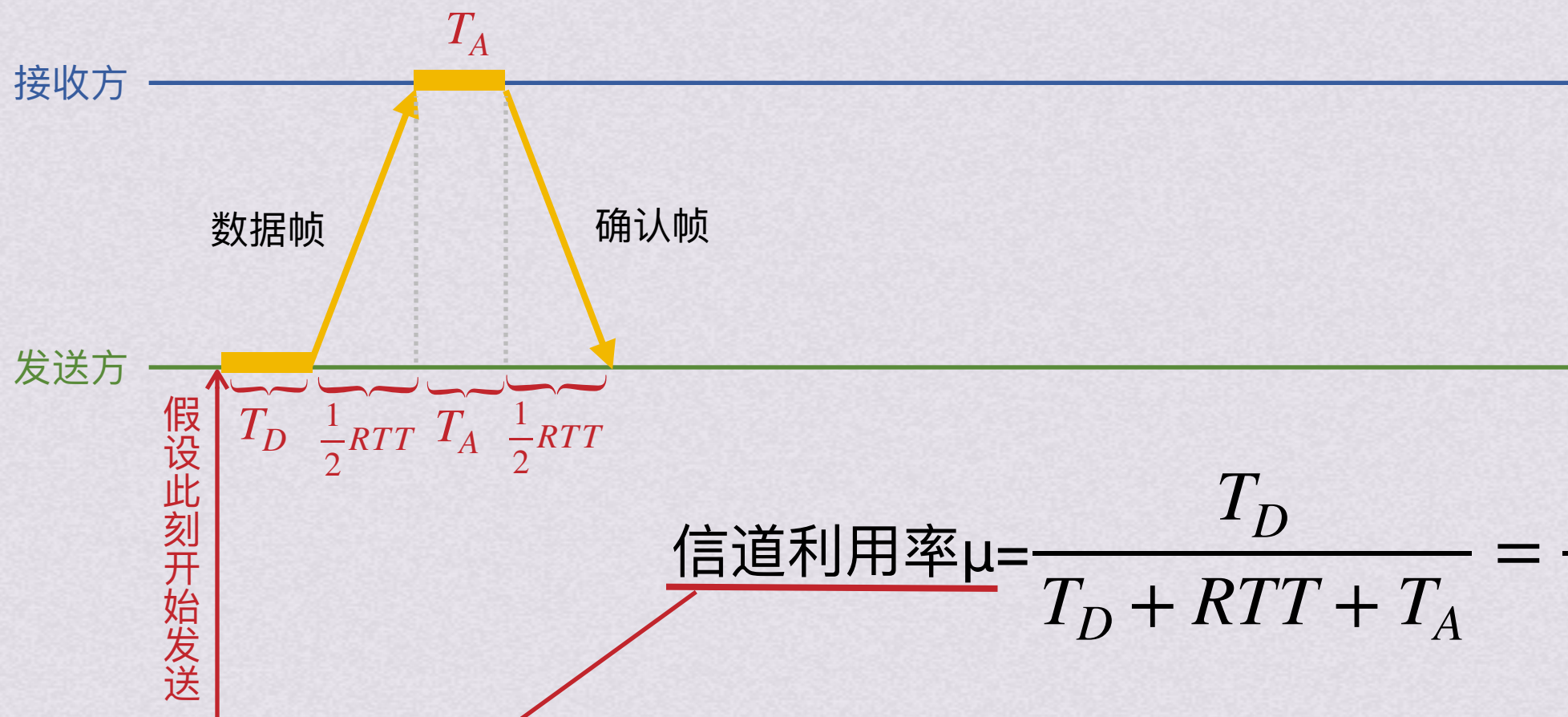




信道利用率



信道利用率

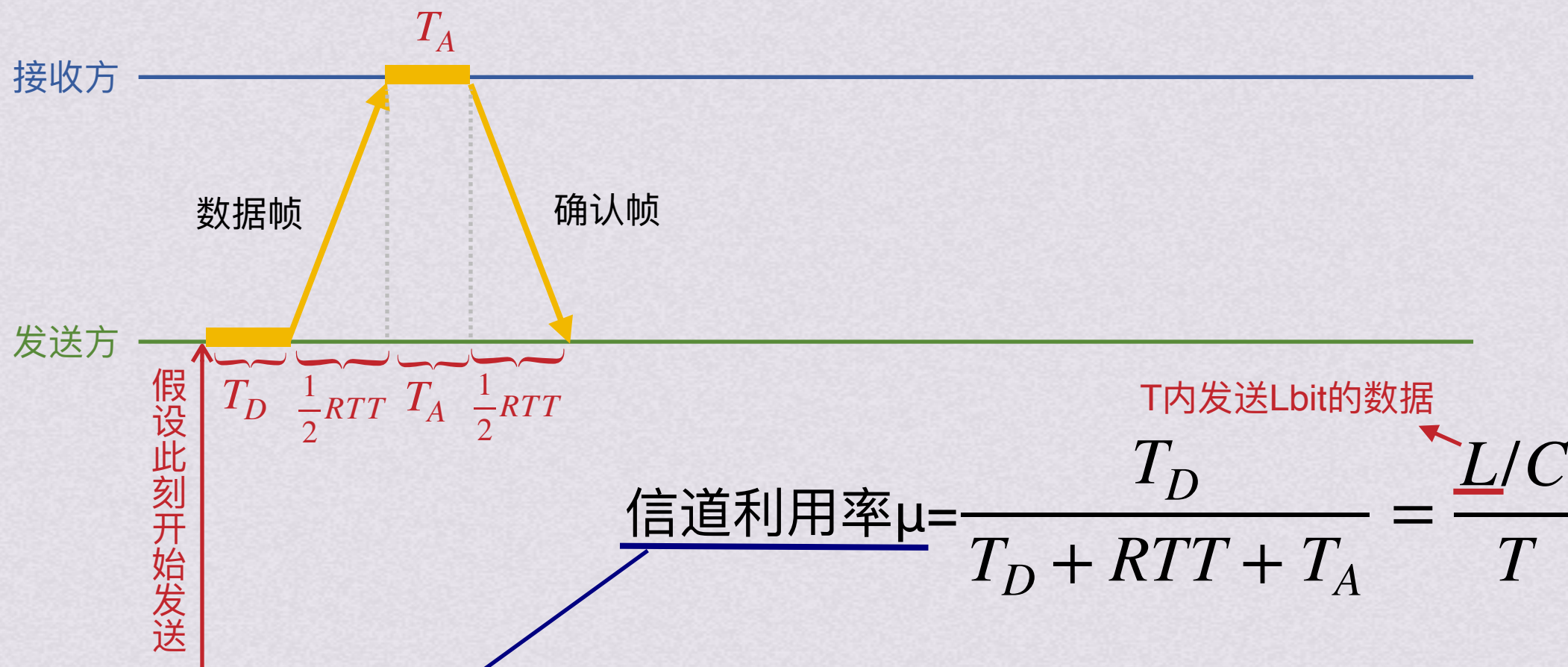


信道利用率 $\mu = \frac{T_D}{T_D + RTT + T_A} = \frac{L/C}{T}$

在一个发送周期内，
有效地发送数据所需时间
占整个发送周期的比例



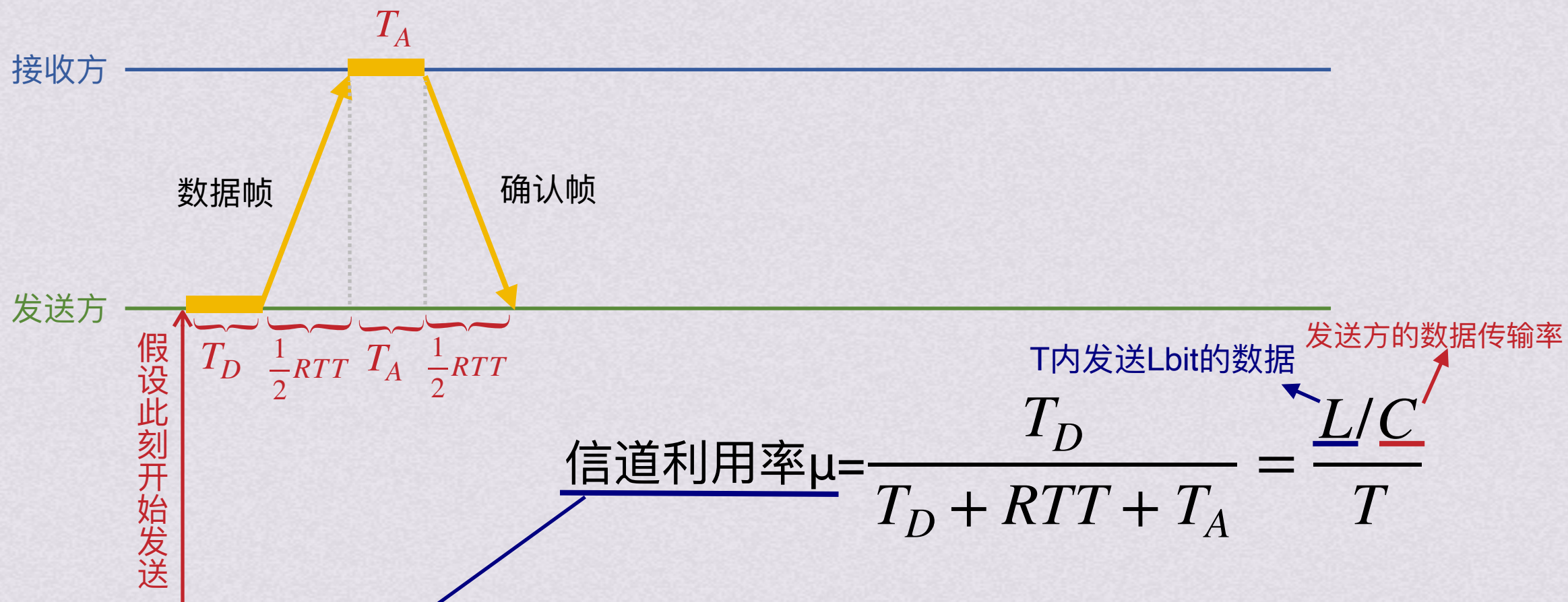
信道利用率



在一个发送周期内，
有效地发送数据所需时间
占整个发送周期的比例



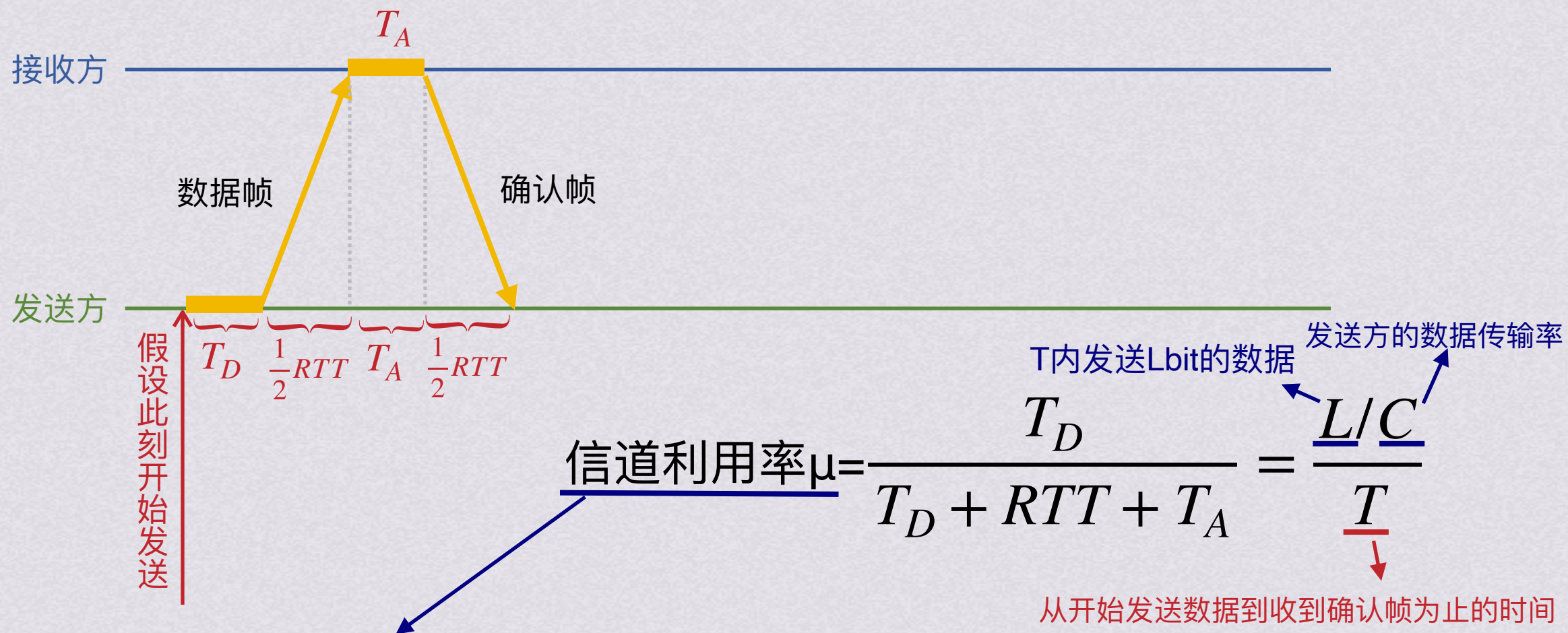
信道利用率



在一个发送周期内，
有效地发送数据所需时间
占整个发送周期的比例



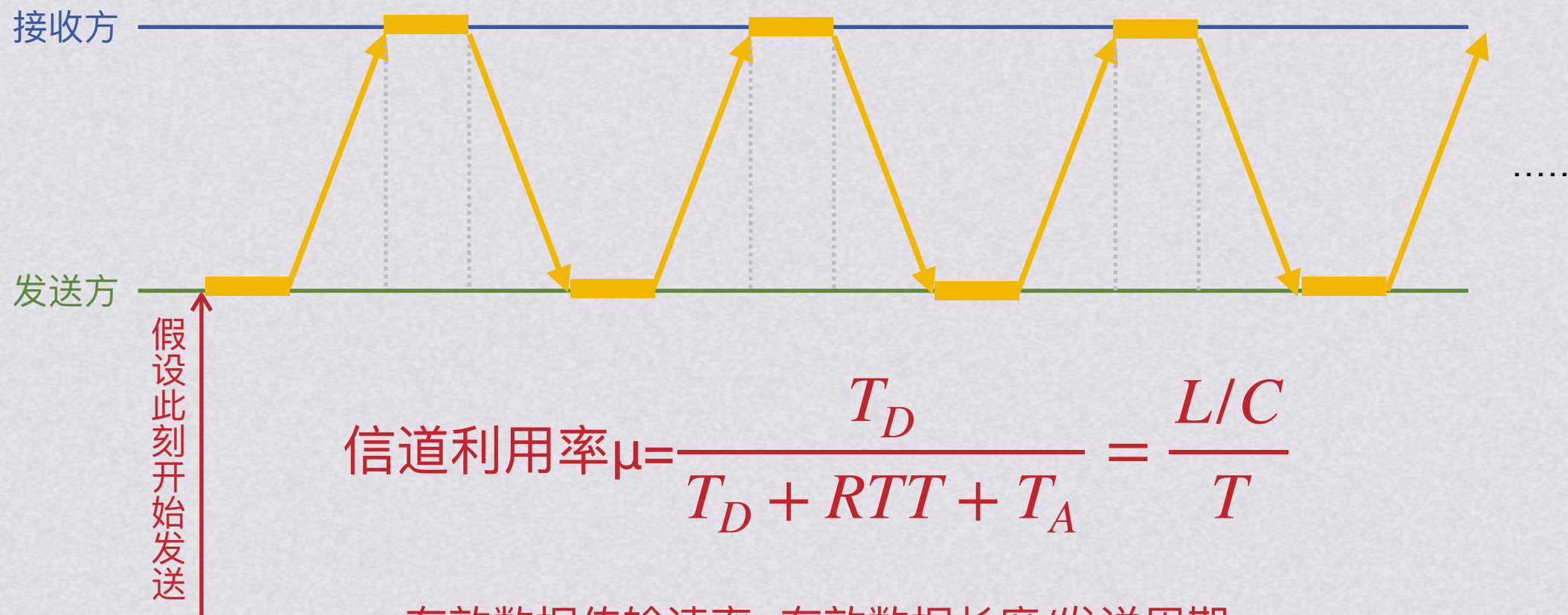
信道利用率



在一个发送周期内，
有效地发送数据所需时间
占整个发送周期的比例



信道利用率



有效数据传输速率=有效数据长度/发送周期

信道吞吐率=信道利用率*发送方的发送速率(数据传输速率)